어셈블리 영상

01 ASSEMBLY 영상

왼다리 (L)



오른다리 (R)



01

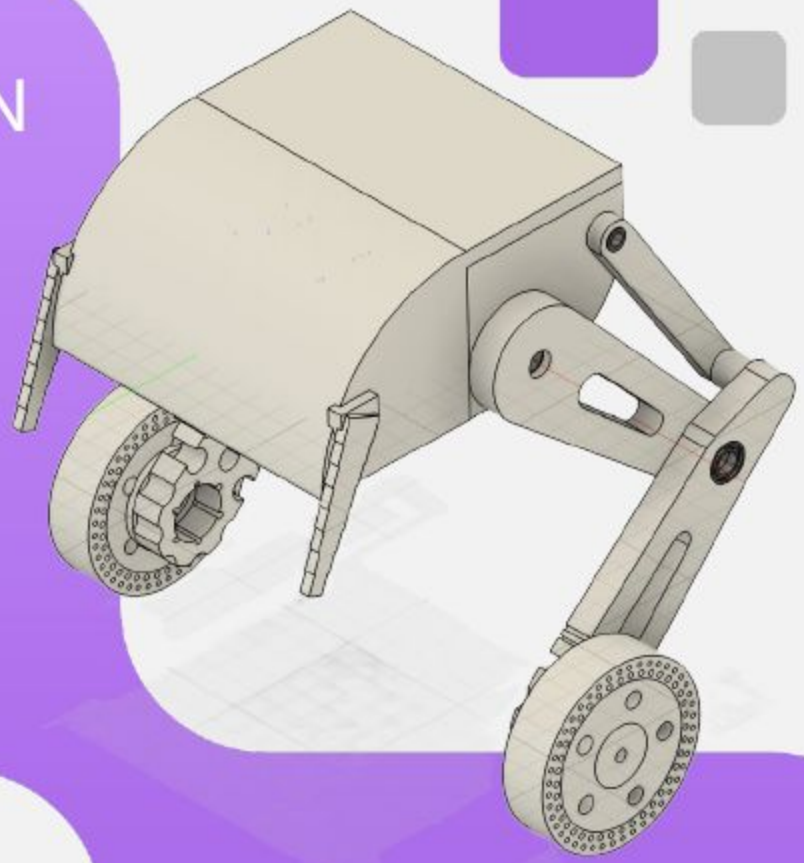
링크부

a · 허벅지 / 종아리 / 커플러 배치

2026 CAPSTONE DESIGN PRESENTATION

캡스톤 디자인

2 학기 하드웨어 통합 보고
+ CAN 구동 · 해설 · 운용



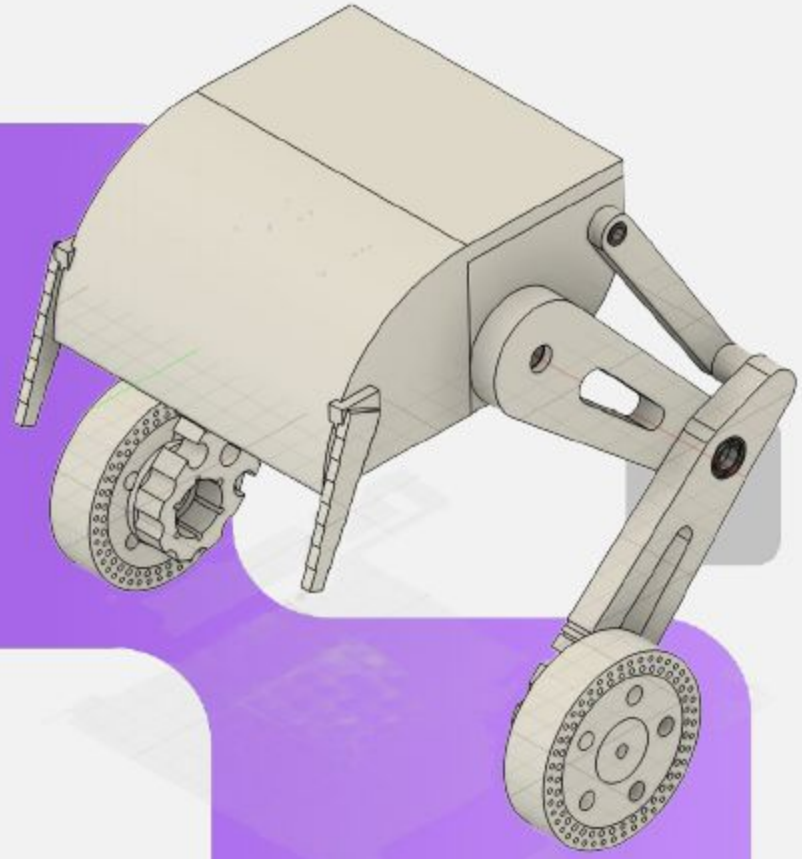
Project by. 2 조 2022121223
문은혁

Date. 2026.09 – 10

목차

CONTENTS

목차 보기



01 **시뮬레이션** 05p
MATLAB Simulink · Motion Simulation

02 **링크부 체결** 12p
Bushing & Shaft

03 **하드웨어 설계 · 제작** 18p
Hardware Design & Build

04 **구동각 · 역학 해석** 27p
Kinematics & Dynamics

05 **전장 구동 테스트** 41p
Electronics Bring-up

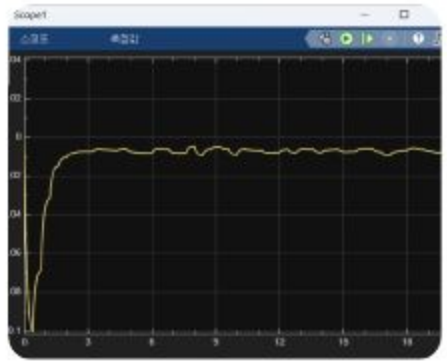
06 **CAN 통합 구동 계획** NEW 55p
CAN Integration Pipeline

07 **코드 해설 · 원리 · 이론** NEW 77p
Code, Principles & Theory

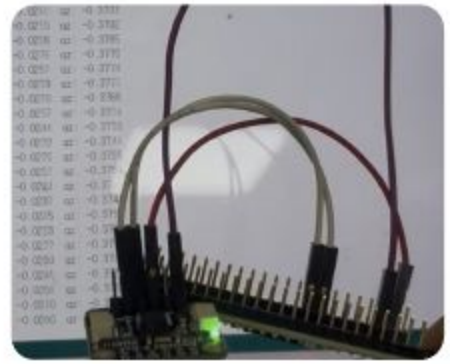
08 **운용 방법** NEW 99p
Operation Manual

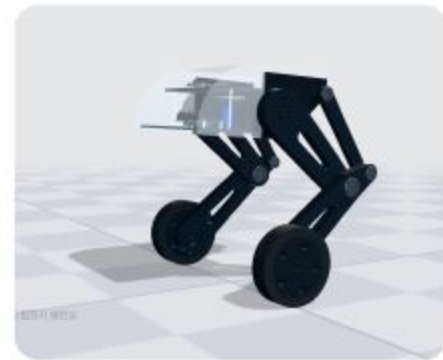
진행 과정

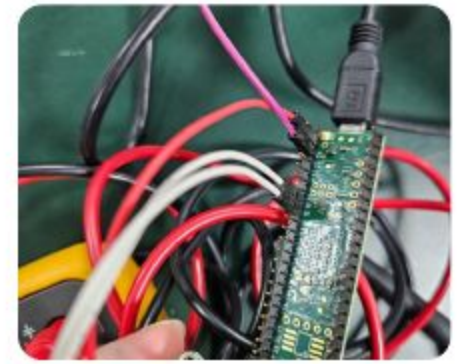
00 1~3 주차 발표에서 CAN 통합까지 — 이 발표의 흐름











09 / 07
1 주차

Simulink 스쿼트 · 주행 · 부시 / 샤프트 · 서보 · 인휠 구동 영상 · 제작 계획

→ 1 · 2 · 5 장

09 / 14
2 주차

CoM · 피치 · 모션 시뮬레이션 · 부시 사양 · 링크 3D 프린팅 · 모듈 구동

→ 1 · 2 · 3 장

09 / 21
3 주차

구동각 한계 · 정역학 · 동역학 · LattePanda · BNO085 IMU

→ 3 · 4 · 5 장

10 / 06
영상 · 부품 확정

쇼케이스 · 어셈블리 v2 · Cura 영상 , 고관절 DM4340P 확정 · BOM 1,184,795 원

→ 3 장

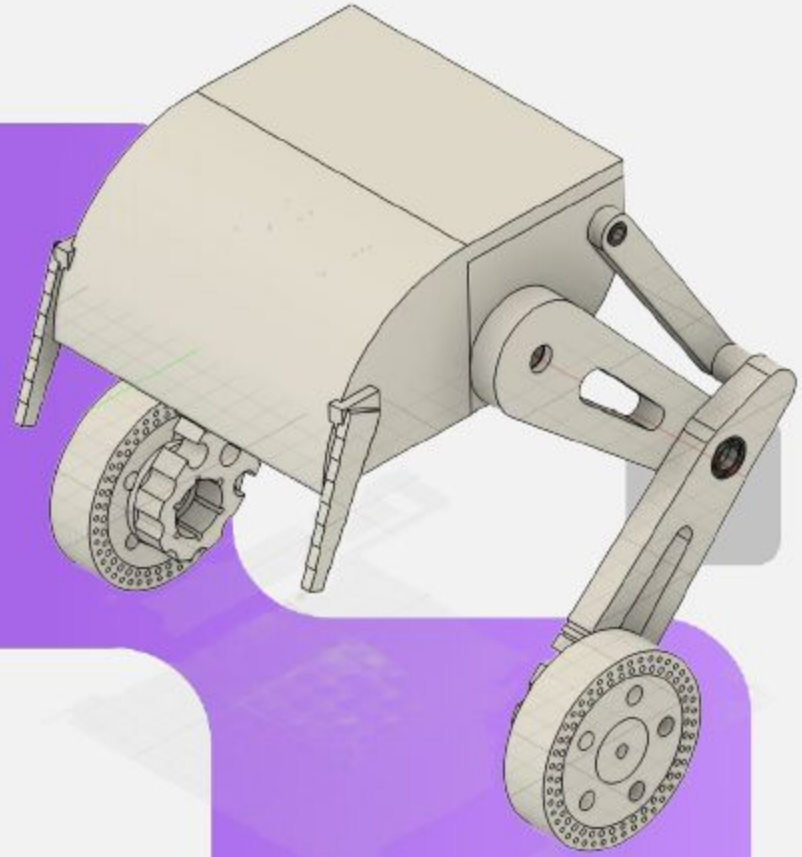
다음
CAN 통합 · 해설 · 운용

DM4340P + ODrive + IMU → Teensy 500 Hz → ROS 2, 원리와 운용 방법

→ 6 · 7 · 8 장 (신규)

PART 01

MATLAB SIMULINK · MOTION SIMULATION



시뮬레이션

스쿼트 · 주행 동작에서 무게중심과 피치 응답, 서보 필요 토크를 시뮬레이션으로 확인

Simulink — 스쿼트 · 주행 · 피치 6–8p

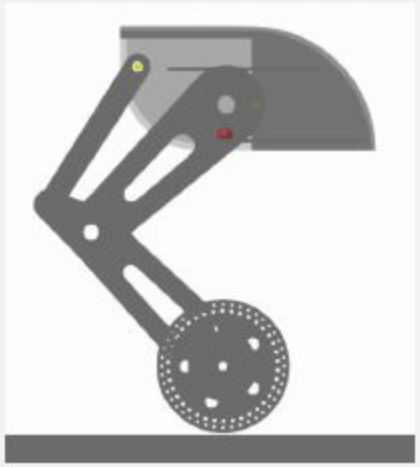
서보모터 필요 토크 9p

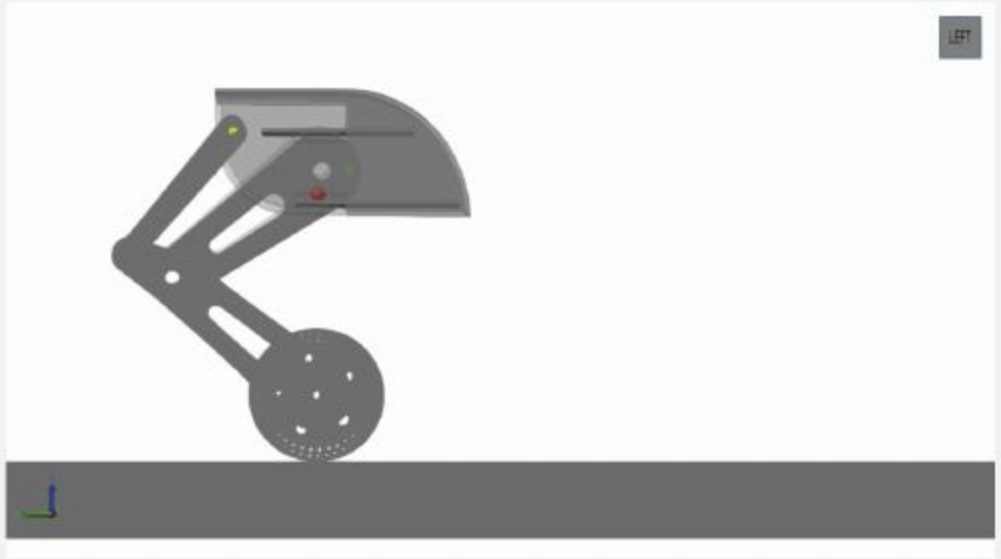
모션 시뮬레이션 10–11p

MATLAB SIMULINK

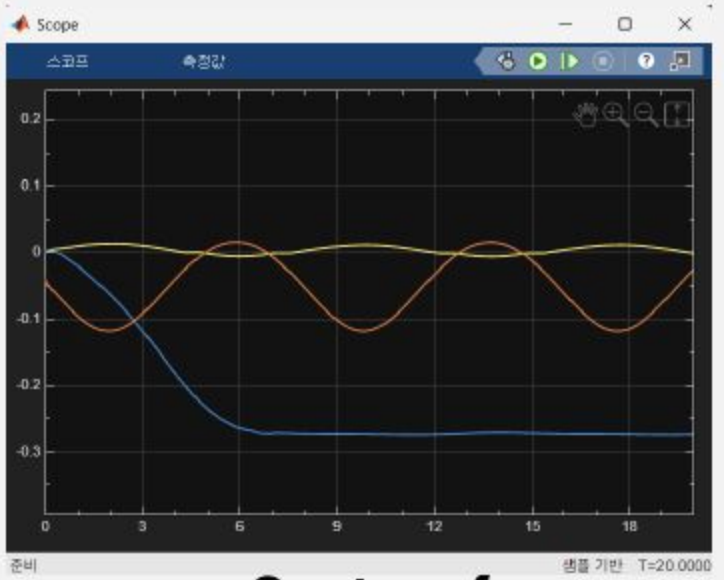
01 Matlab 시뮬레이션

- 스쿼트 모션 영상 및 피치 스코프

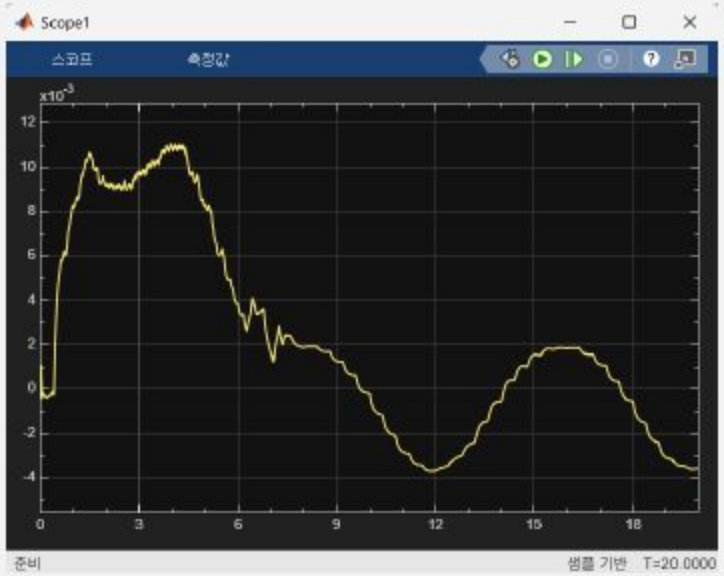




스쿼트 모션 영상



Center of



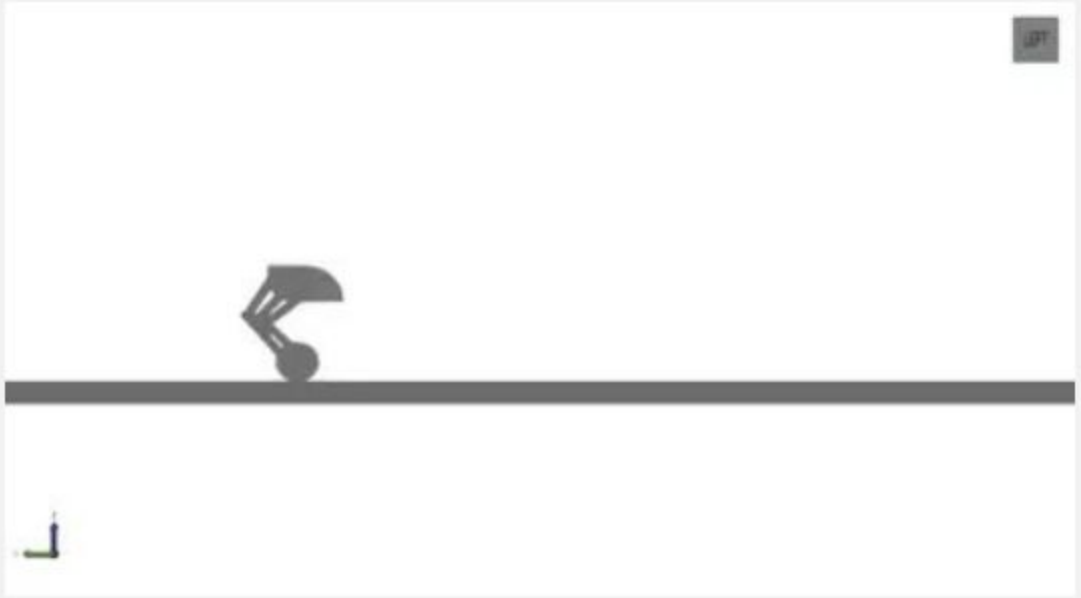
pitch

목차

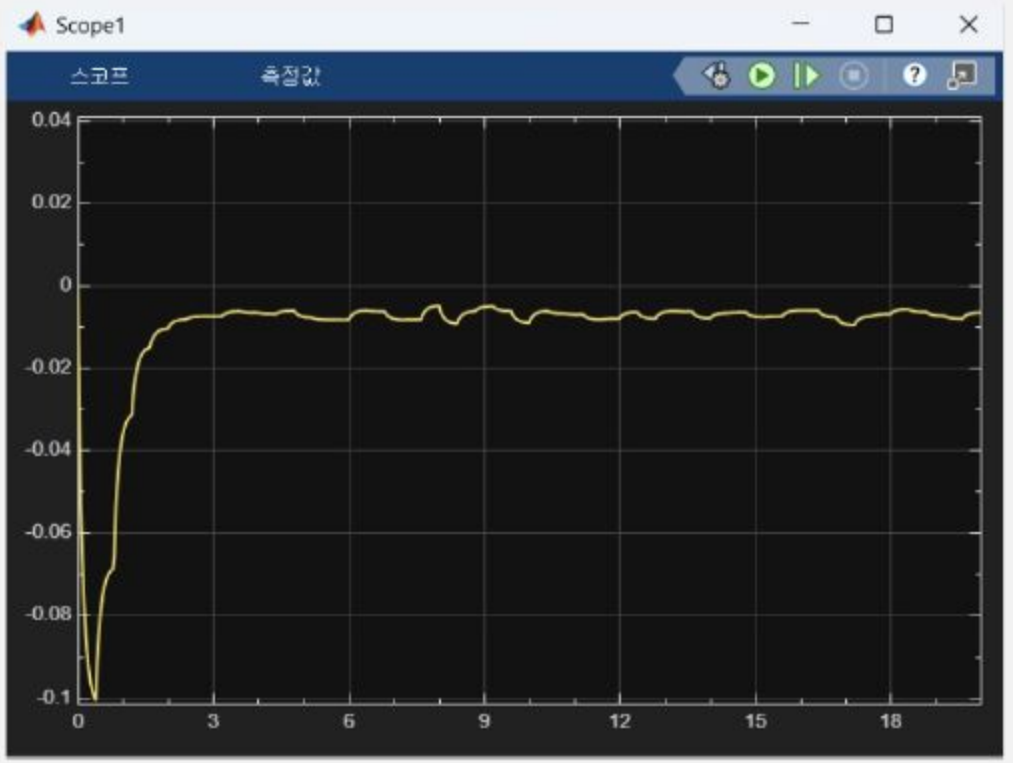
MATLAB SIMULINK

01 **Matlab 시뮬레이션**

주행 영상 및 피치 스코프



주행 영상



pitch scope

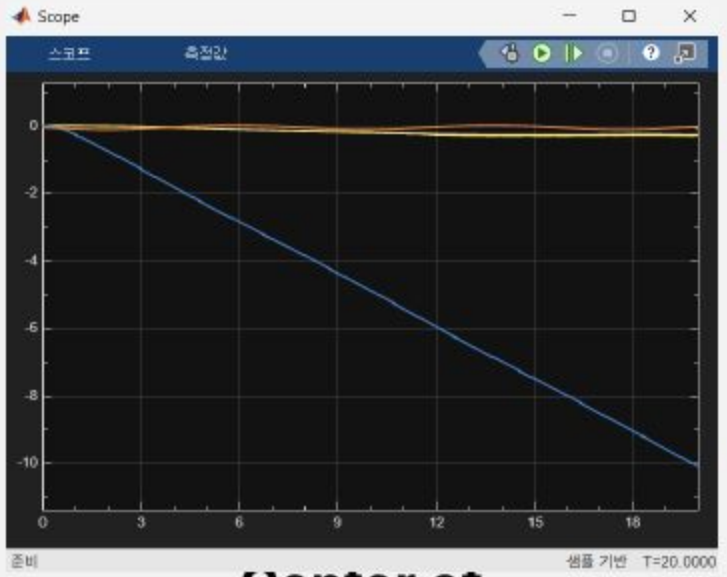
MATLAB SIMULINK

01 Matlab 시뮬레이션

주행 영상 및 피치 스코프



주행 + 스쾃 영상



Center of



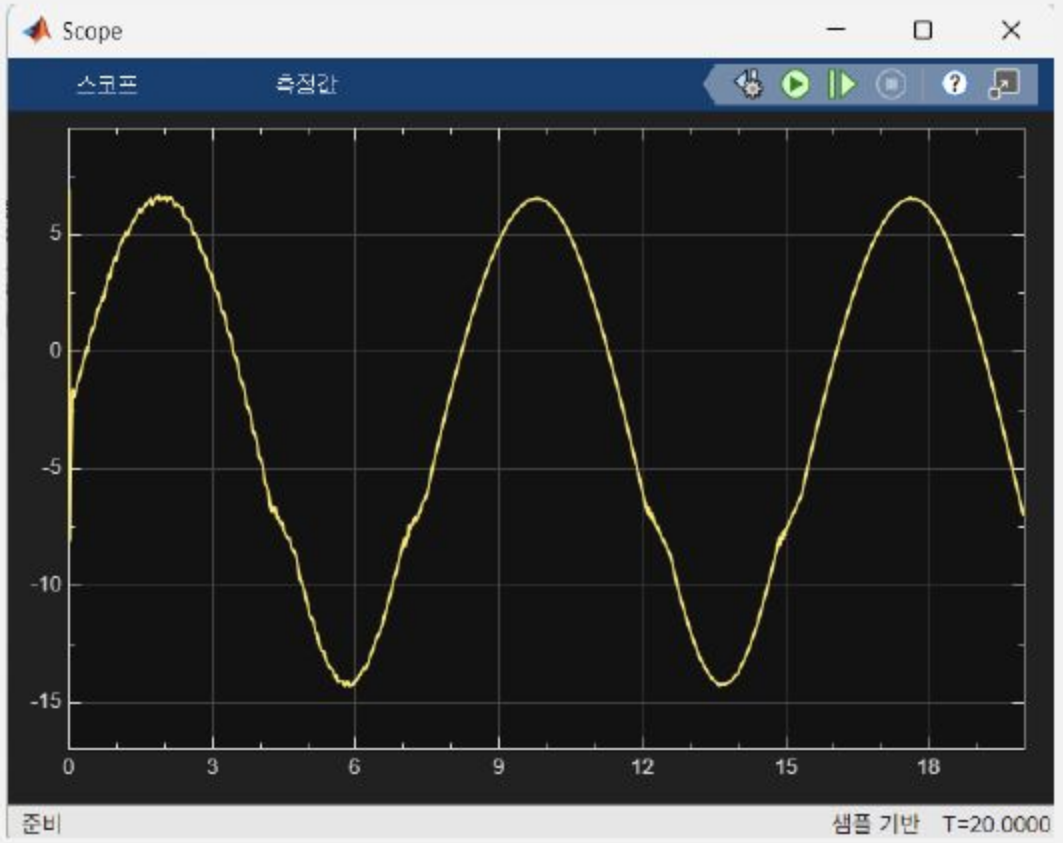
Pitch

목차

MATLAB SIMULINK

01 Matlab 시뮬레이션으로 구한 서보모터 필요 토크



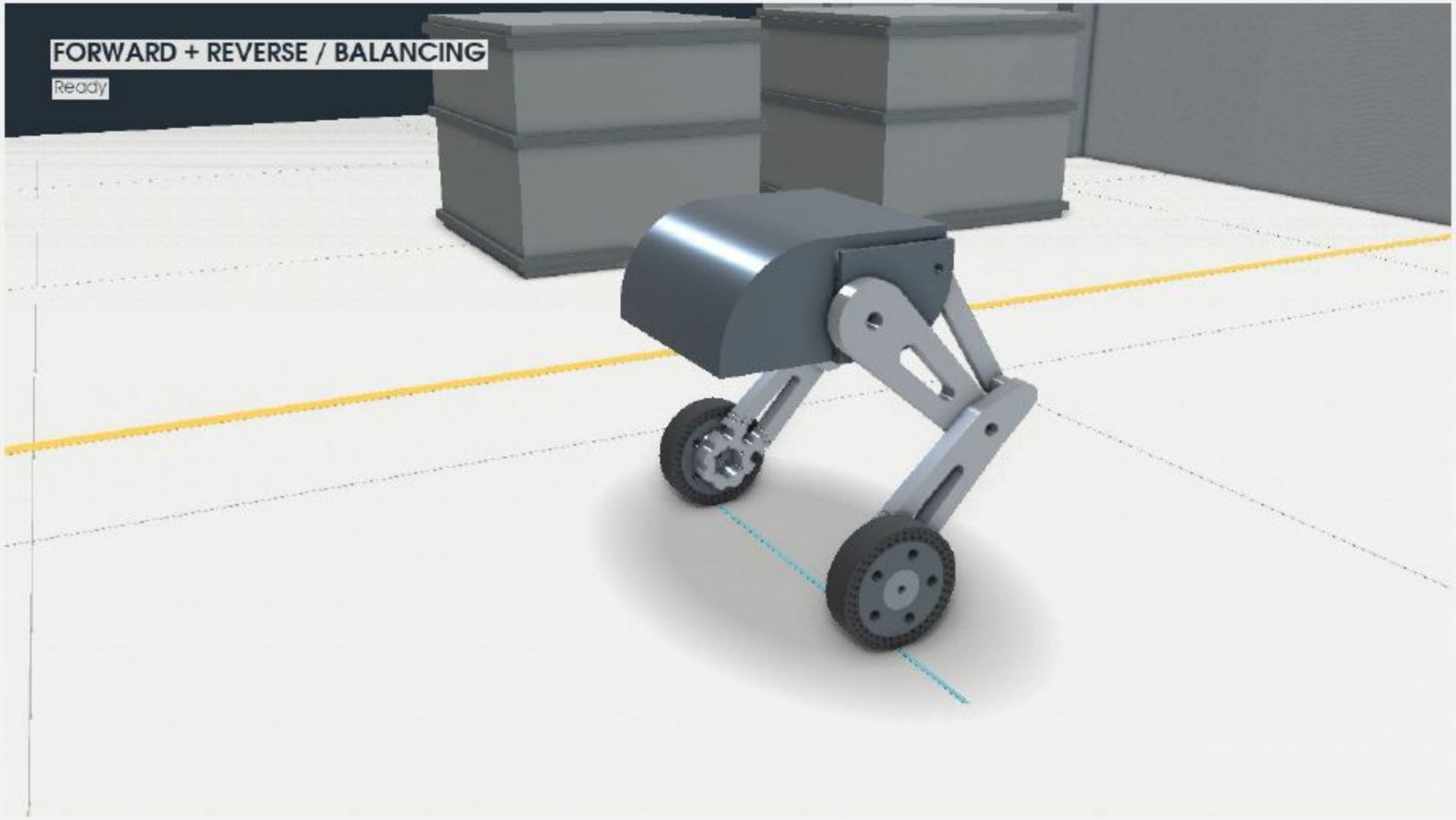


MOTION SIMULATION



목차

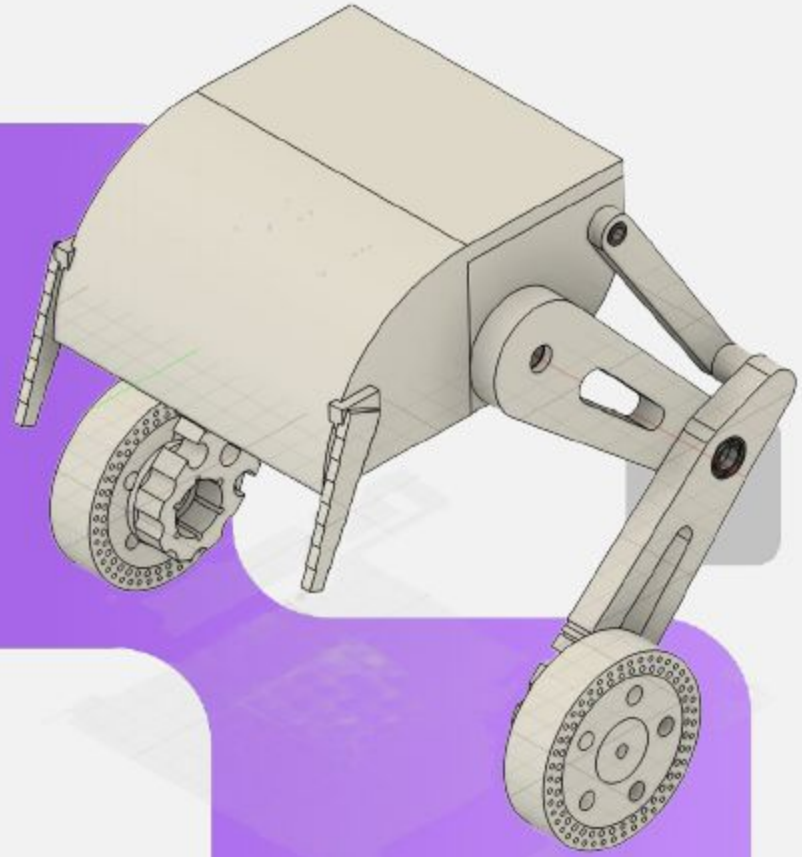
MOTION SIMULATION



목차

PART 02

BUSHING & SHAFT



링크부 체결

MISUMI 부시 · 샤프트로 4 절 링크 조인트를 결합하고 조인트별 최소 길이와 주문 현황 정리

MISUMI 부시 · 샤프트 결합 13p

부싱 · 샤프트 사양 14–15p

조인트별 부시 최소 길이 16p

부싱 · 샤프트 주문 현황 17p

링크부 체결

02 **MisuMi 사 부시 및 샤프트를 이용하여 결합**

- 부시와 샤프트를 체결하여 결합

MiSUMi

미스미 (Misumi) 는 산업용 기계 부품 분야에서 높은 신뢰도와 인지도를 가진 매우 유명한 일본계 기업







플랜지붙이 리니어부시 **리니어 샤프트** **부시 , 샤프트 체결**

https://kr.misumi-ec.com/vona2/detail/110300027150/?list=PageCategory - 리니어 부시
https://kr.misumi-ec.com/vona2/detail/110310559889/?clkid=clkid_pdct_kr_ec_45189_105 - 샤프트

링크부 체결

02 MisuMi 사 부시 및 샤프트를 이용하여 결합

- 부시와 샤프트를 체결하여 결합



부싱

■치수표

dr	허용공차	D	표면처리 없음 허용공차	L	허용공차	H	(T)	d	d_1	t	P.C.D.	W	F	A	K	편심 μm	볼 조열	* 직각도	기본정격하중 C(동)N	기본정격하중 Co(정)N	참고질량(g) 원형플랜지	참고질량(g) 각형플랜지	참고질량(g) 2면플랜지
5	0 -0.008	10	0 -0.011	15	±0.6	26	5	3.5	6	3.1	18	21	-	18	16	24	4	0.024	167	206	18	13	15
6	0 -0.009	12	0 -0.013	19		28					20	22	-	20	18				206	265	24	18	21
8		15		24		32					24	25	-	24	21				265	380	37	29	33
10		19		29		40					29	30	-	29	25				372	549	72	52	64
12		21	0 -0.016	30		42	6	4.5	7.5	4.1	32	32	-	32	27				412	598	76	57	68
13		23		32		43					33	34	-	33	29				510	784	88	72	81
16		28		37		48					38	37	22	31	34		5		775	1180	120	104	112
20	0 -0.010	32	0 -0.019	42		54	8	5.5	9	5.1	43	42	24	36	38	30		0.030	882	1370	180	145	167
25		40		59		62					51	50	32	40	46				980	1570	340	300	325
30		45		64		74	10	6.6	11	6.1	60	58	35	49	51		6		1570	2740	470	375	388
35	0 -0.012	52	0 -0.022	70		82					67	64	38	55	60	40		0.040	1670	3140	650	560	575
40		60		80		96	13	9	14	8.1	78	75	45	64	70				2160	4020	1060	880	913
(50)		80		100		116					98	92	-	-	-				3820	7940	2200	2000	-

* D부와 플랜지면의 직각도

[!]dr( )사양은 원형플랜지·각형플랜지만 선택할 수 있습니다.

[!]납품시 방청유가 도포되어있습니다.

[!]플랜지형 높이조정 스페이서는 >>본 링크에서 선택 하실 수 있습니다.

[!]샤프트-하우징공차는『축경-하우징 직경 권장 공차』를 확인해 주십시요

https://kr.misumi-ec.com/vona2/detail/110300027150/?list=PageCategory - 리니어 부시

https://kr.misumi-ec.com/vona2/detail/110310559889/?clkid=clkid_pdct_kr_ec_45189_105 - 샤프트

목차

링크부 체결

02 **MisuMi 사 부시 및 샤프트를 이용하여 결합**

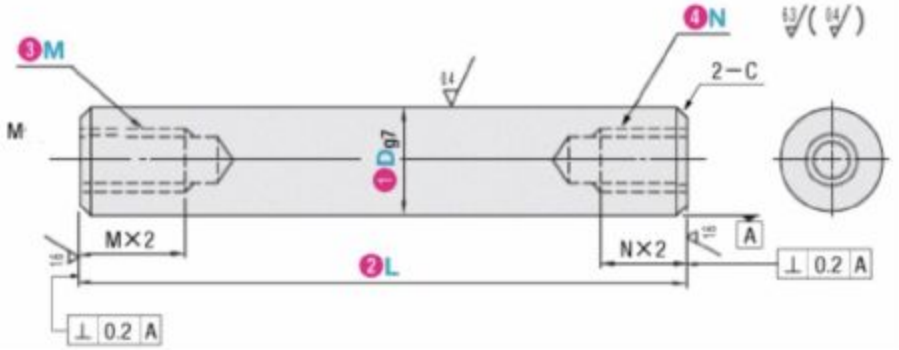
- 부시와 샤프트를 체결하여 결합



샤프트

외형도/규격표

치수도



■ 치수 규격

형식		❷L 지정 단위 1mm	❸M(보통 나사) ❹N(보통 나사) 선택
❶Type	❷D	표준형	
	4	20~300	2
	5	20~400	2.6 3
	6	20~600	3
	8	20~1000	3 4 5
표준형	10	20~ 1000	3 4 5 6
(D 공차 g7)	12	20~1200	4 5 6 8
E-SFJW	13	25~1200	4 5 6 8
E-SSFJW	15	25~1200	4 5 6 8 10
E-PSFJW	16	30~1200	4 5 6 8 10
E-PSSFJW	20	30~1200	4 5 6 8 10 12
(D 공차 f8)	25	35~1200	4 5 6 8 10 12 16
E-PSFGW	30	35~1500	6 8 10 12 16 20
	35	35~1500	8 10 12 16 20 24
	40	50~1500	10 12 16 20 24 30
	50	65~1500	12 16 20 24 30

❶L치수는 M×2+N×2≤L를 충족해야 합니다. M×2.5+4+N×2.5+4≤L일 때 나사 하단 구멍이 관통될 수 있습니다.
❶D치수 35,40,50은 E-PSFJW에만 적용 가능
❶D치수 4, 5는 E-PSFGW에 적용되지 않음

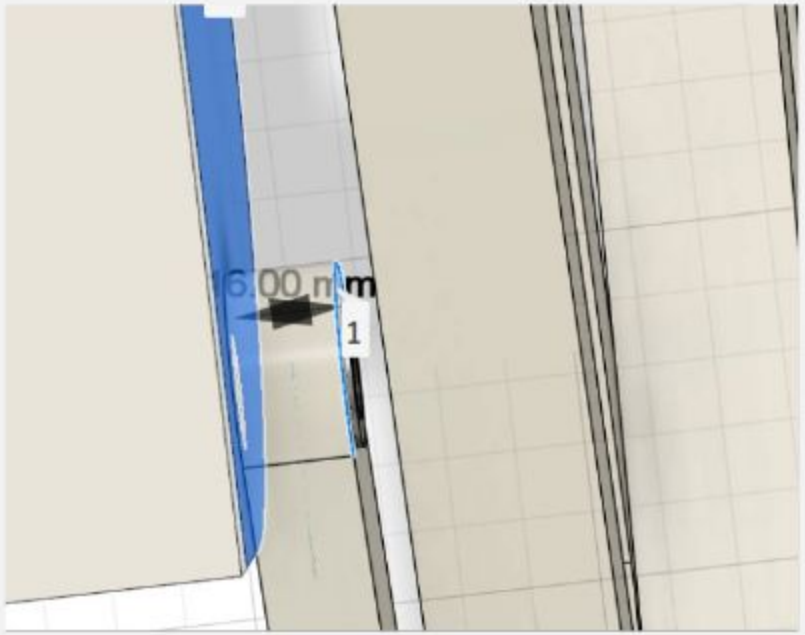
https://kr.misumi-ec.com/vona2/detail/110300027150/?list=PageCategory - 리니어 부시
https://kr.misumi-ec.com/vona2/detail/110310559889/?clkid=clkid_pdct_kr_ec_45189_105 - 샤프트

목차

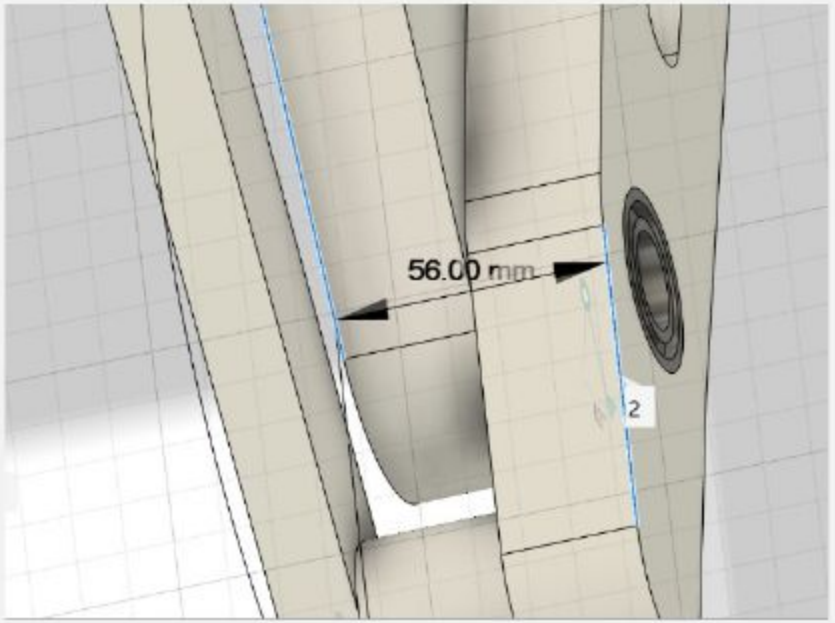
링크부 체결

02 **MisuMi 사 부시 및 샤프트를 이용하여 결합**

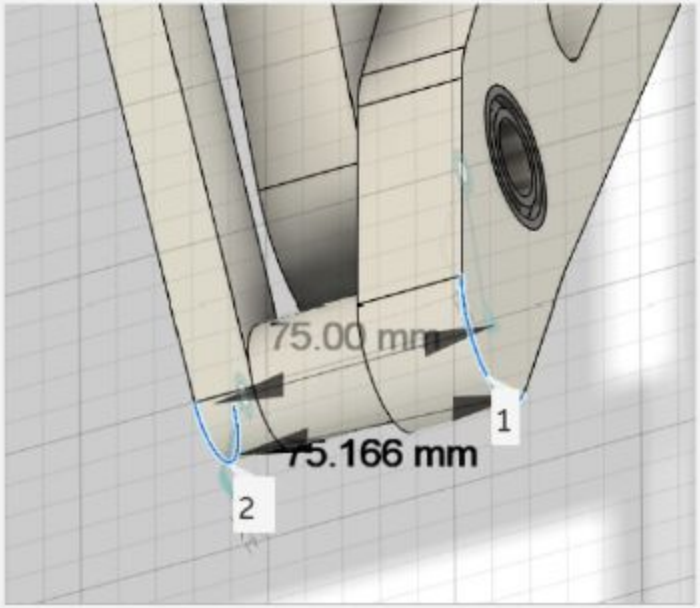
- 각 링크별 부시의 필요한 최소길이



뒷 허벅지와 토르소 사이



허벅지와 무릎 사이



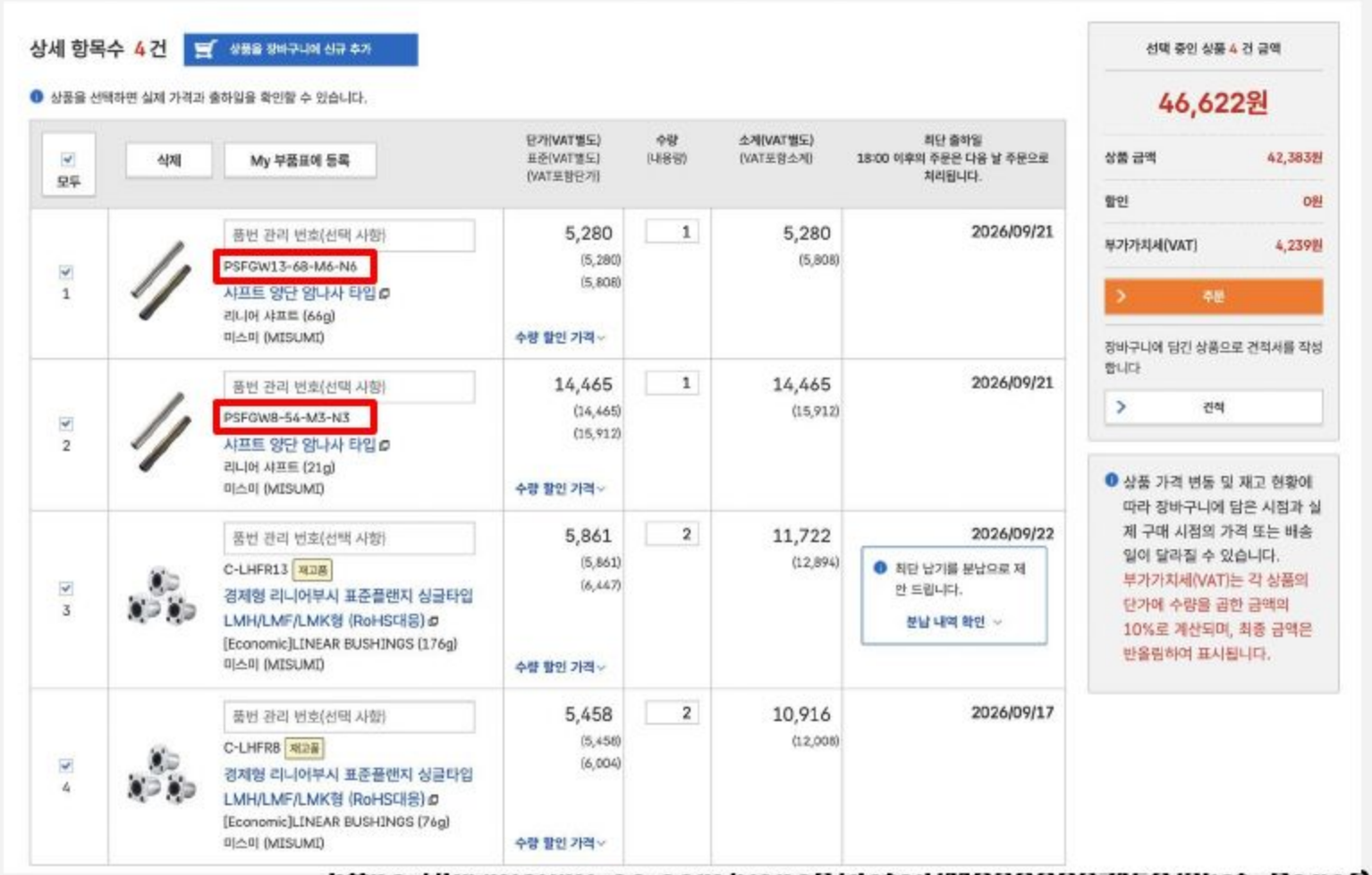
뒷 허벅지와 무릎 사이

링크부 체결

02

부싱 , 샤프트 주문 현황

- 부시와 샤프트를 체결하여 결합



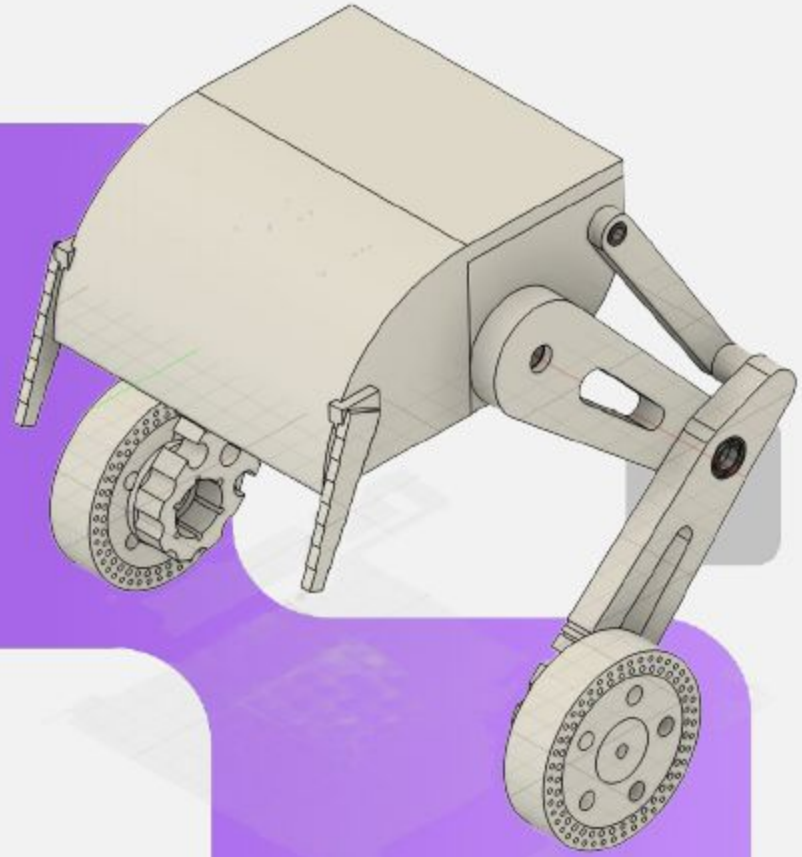
https://kr.misumi-ec.com/vona2/detail/110300027150/?list=PageCategory - 리니어 부시

https://kr.misumi-ec.com/vona2/detail/110310559889/?clkid=clkid_pdct_kr_ec_45189_105 - 샤프트

목차

PART 03

HARDWARE DESIGN & BUILD



하드웨어 설계 · 제작

서포터 → 어셈블리 → 링크 · 토르소 3D 프린팅 → 아크릴 → 모션 쇼케이스

서포터 설계 19p

어셈블리 영상 v1 (v2 는 1p 오프닝) 20p

3D 프린팅 — 계획 · 링크부 · 토르소 21–24p

토르소 아크릴 25p

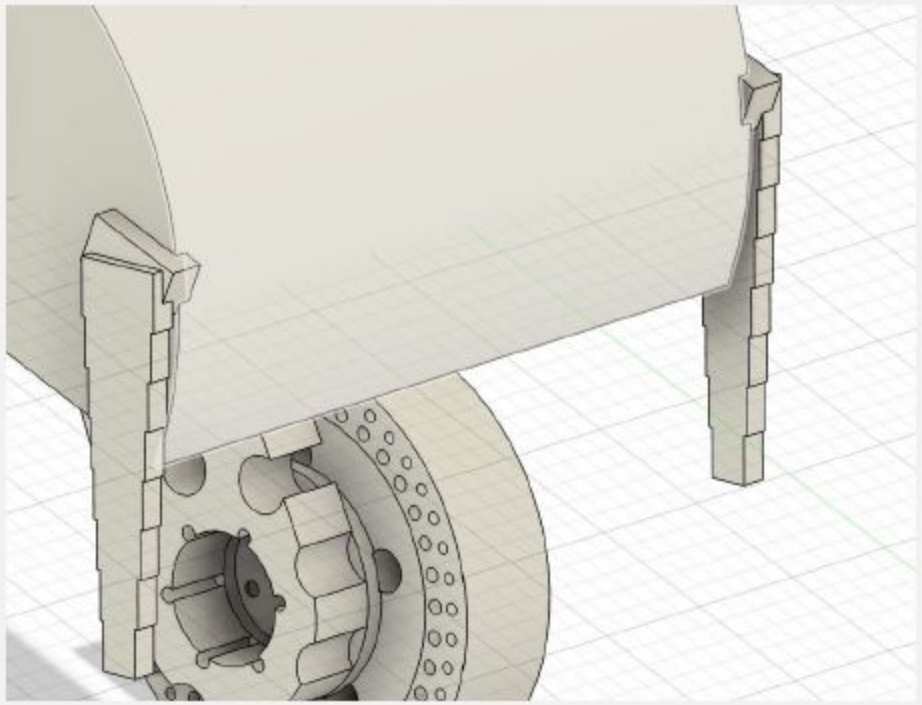
모션 쇼케이스 26p

HARDWARE DESIGN & ASSEMBLY

03

Supporter Desgin

- Supporter 를 통한 토르소 및 센서 보호



서포터와 하우징 프레임이
외력을 대신 지지하여
축의 직접적인 파손을 방지 .

Supporter 디자인

Supporter 설계 이유

ASSEMBLY VIDEO



목차

하드웨어 제작 계획

05 **3D printing**

- 교내 시설 (드론앤커센터 or 기게공작실) 이용 예정



드론앤커센터 3D printer 실



Ultimaker S5 Pro 3D printer

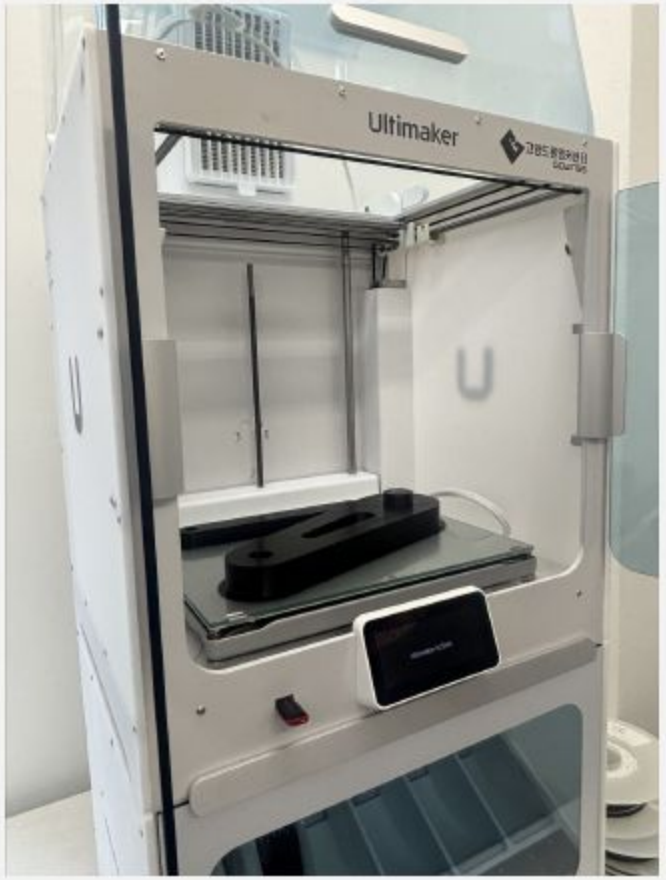


Markforged Mark 2

3D 프린팅

01 링크부 3D 프린팅

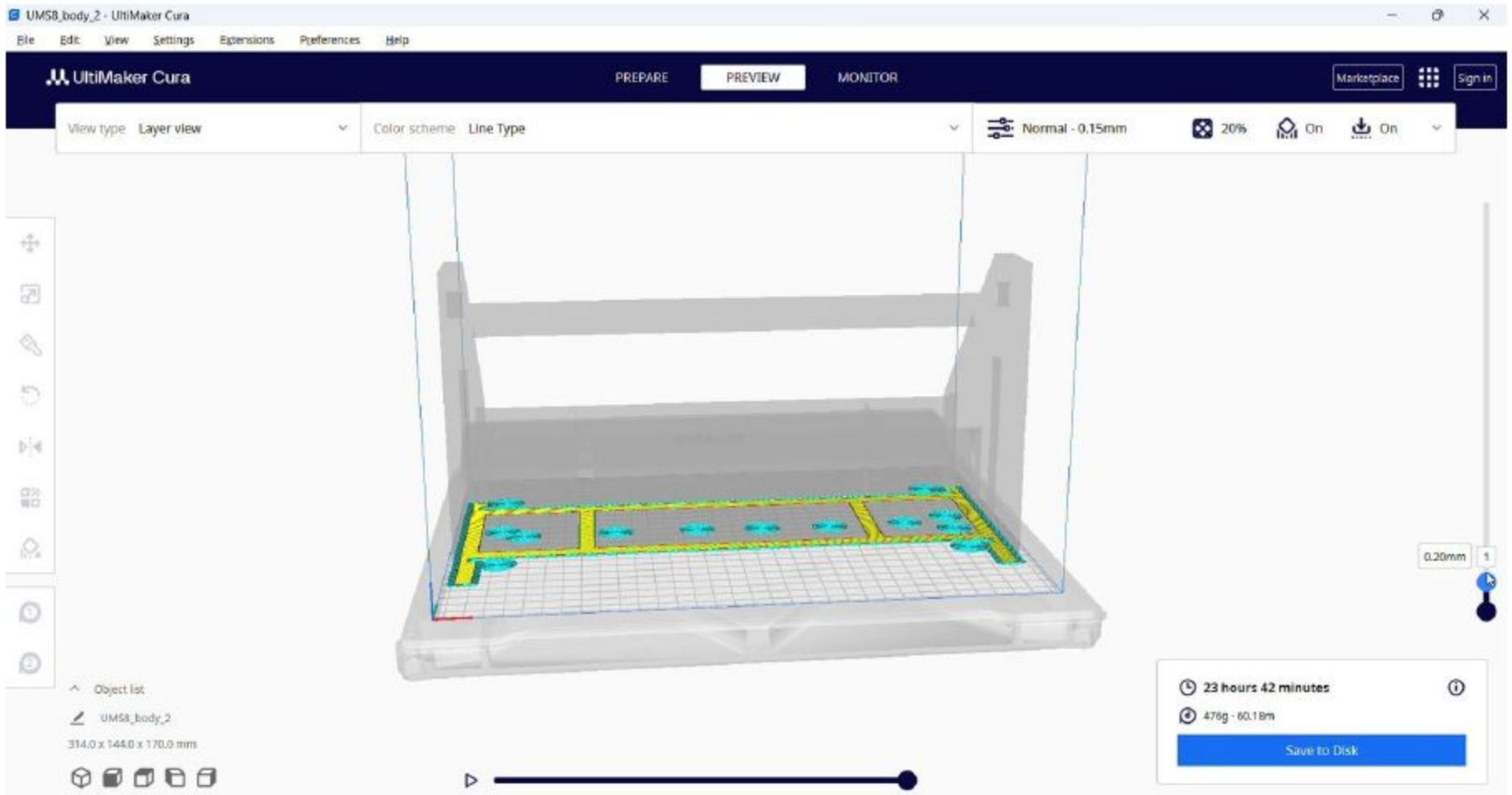




고양시 드론앤커센터에서 출력
비용 : 84000 원

3D 프린팅

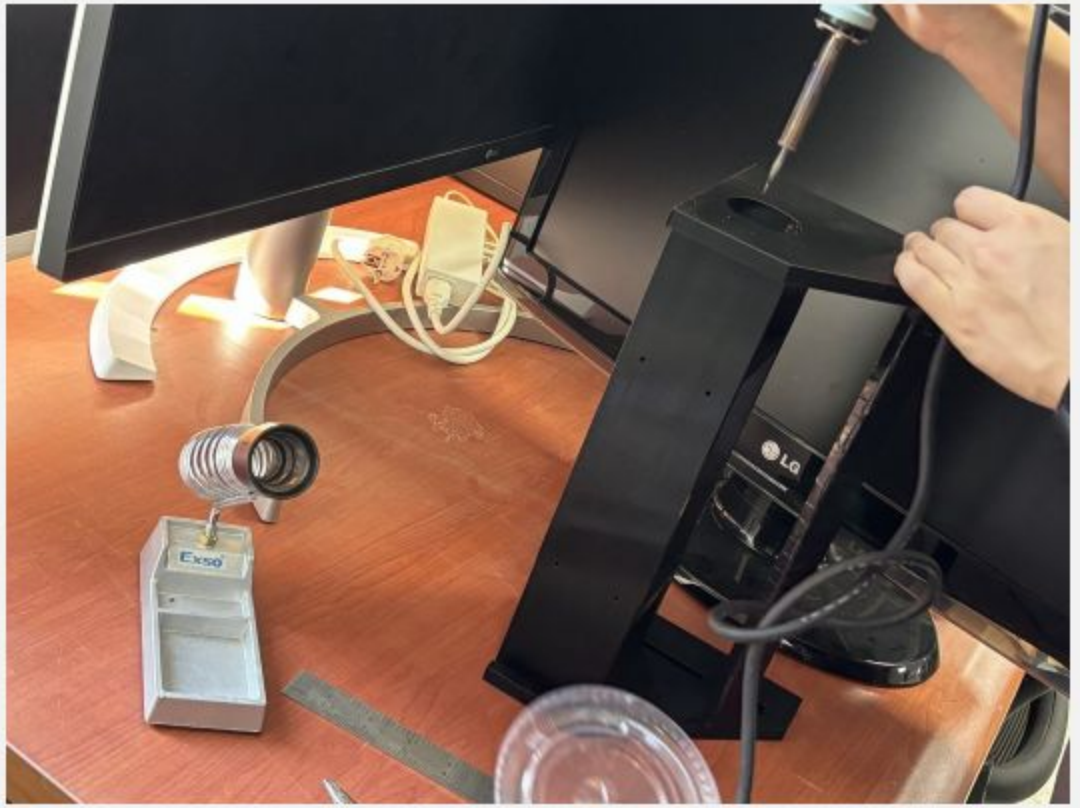
00 토르소 3D 프린팅 시뮬레이션



하드웨어 제작 현황

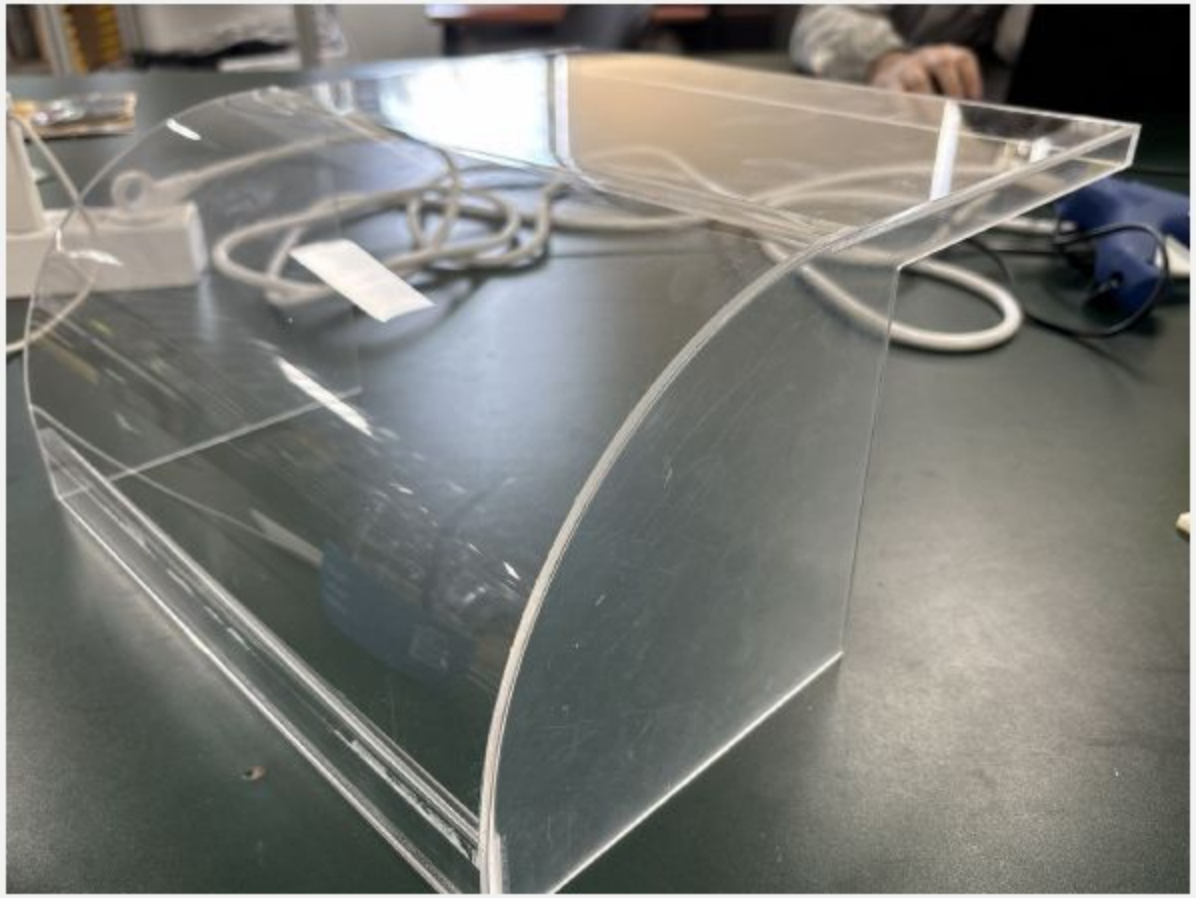
01 토르소 본체 제작 현황





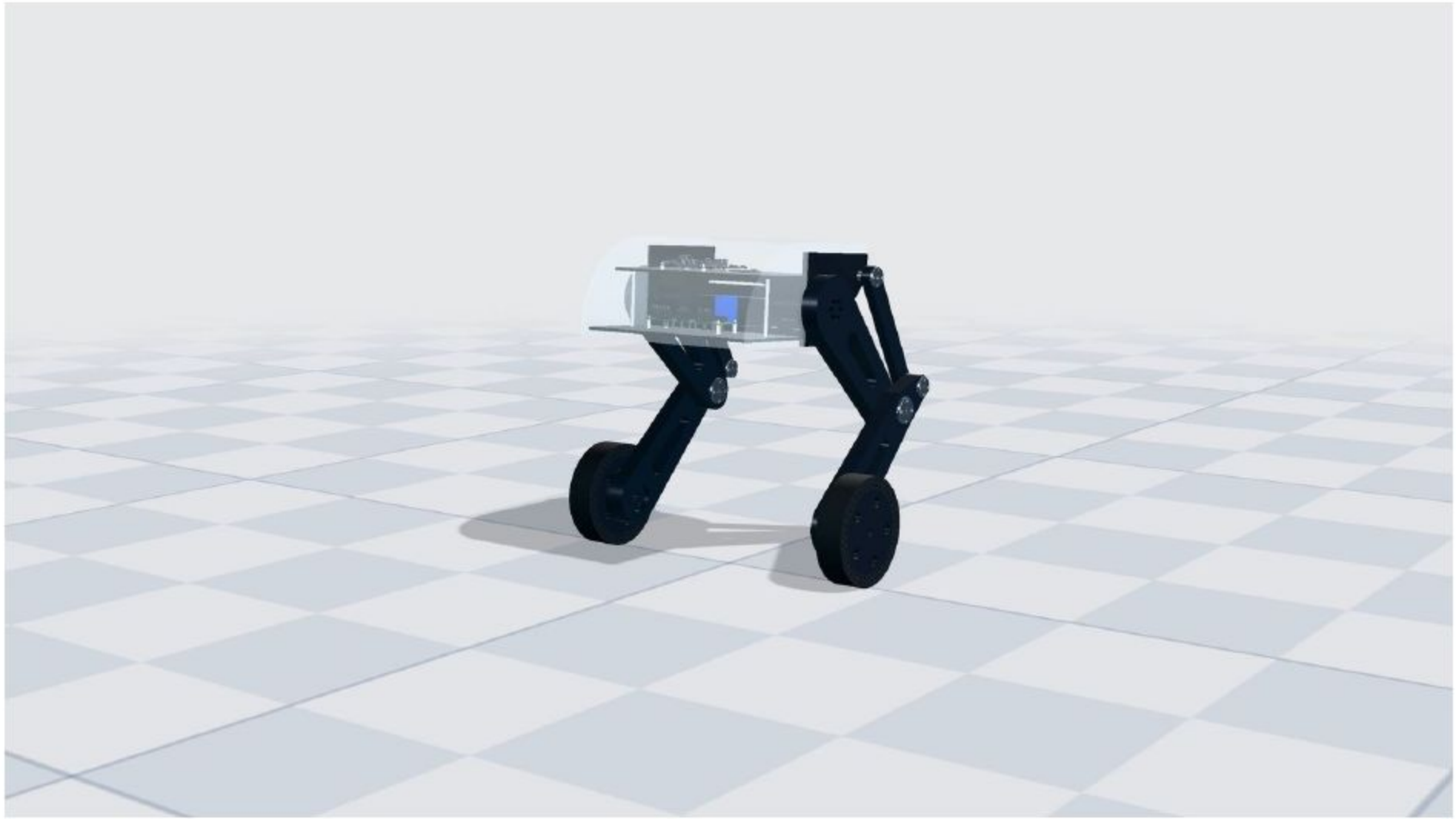
하드웨어 제작 현황

01 토르소 아크릴 제작 현황



모션 쇼케이스

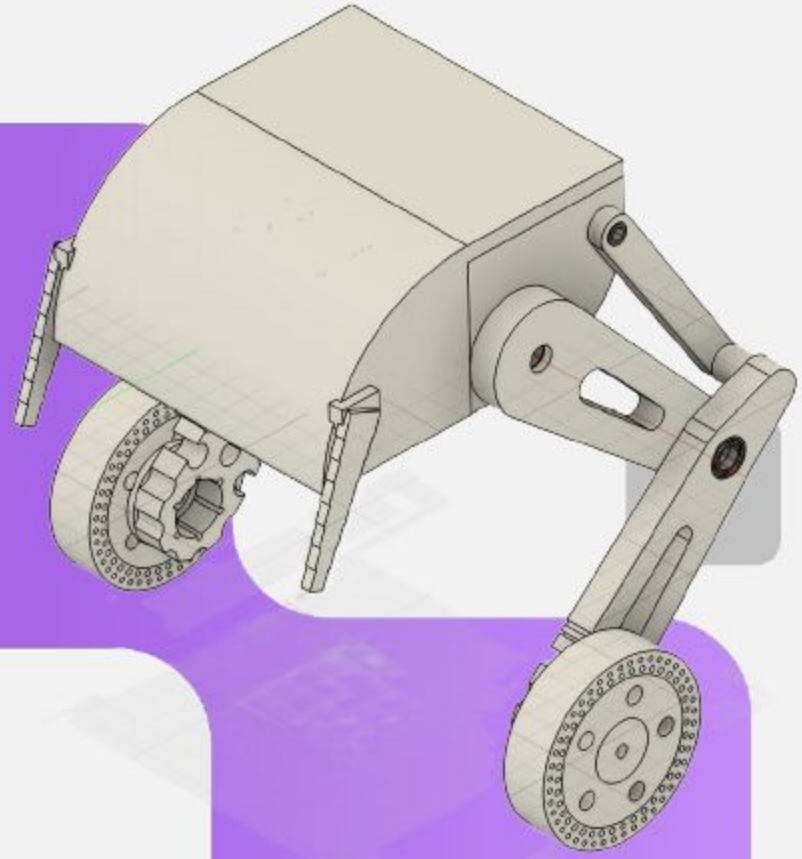
02 MOTION / 쇼케이스 영상



목차

PART 04

KINEMATICS & DYNAMICS



구동각 · 역학 해석

조립 · 간섭 · 전달각 한계로 운용범위 (+20.9° / −21.6°) 를 정하고
정역학 · 동역학으로 서보 토크 검증

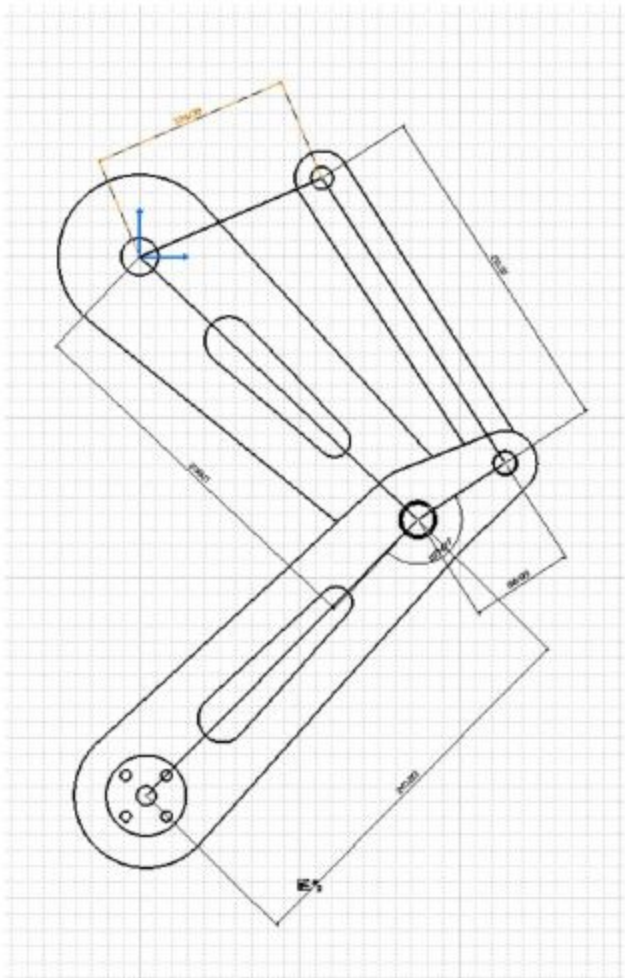
서보 구동각 제한 — 조립 · 간섭 · 전달각 28–34p

정역학 해석 35–38p

기구학 · 동역학 39–40p

서보모터의 구동각 제한

00 구동각 제한 설정 기준



1. 조립 한계에 의한 구동각 제한
-> 무릎 레버와 보조링크가 일직선이 되어 링크가 더 이상 연결될 수 없는 각도

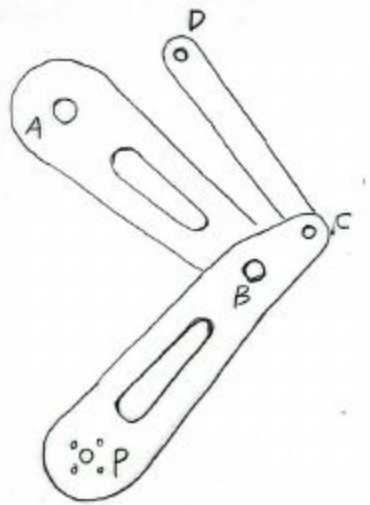
2. 간섭 한계에 의한 구동각 제한
→RecurDyn 해석으로 확인한 , 링크 부품끼리 물리적으로 충돌하는 각도

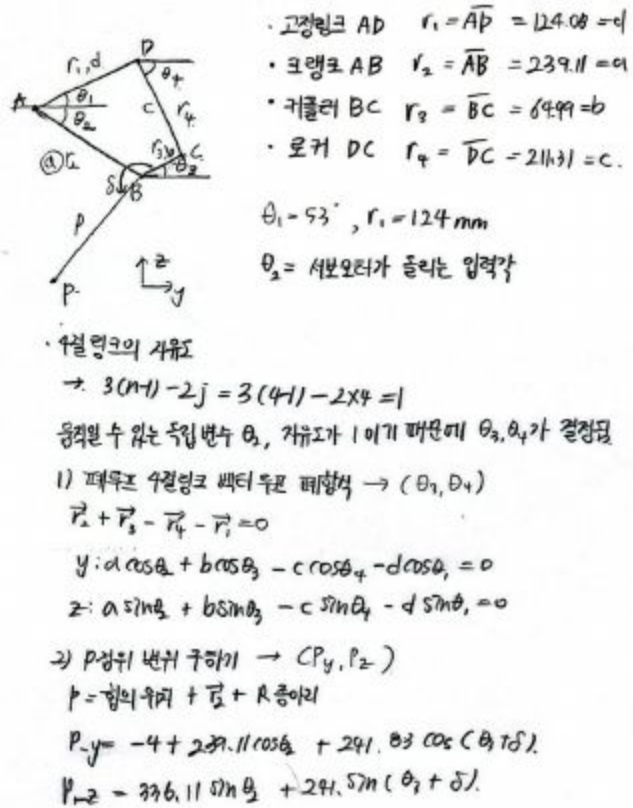
3. 전달각 한계에 의한 구동각 제한
-> 보조링크의 힘이 레버를 효율적으로 돌릴 수 있는 전달각 40°~140° 유지 구간

3 가지 조건을 모두 만족하는 구간을 구동 가능 구간으로 선정

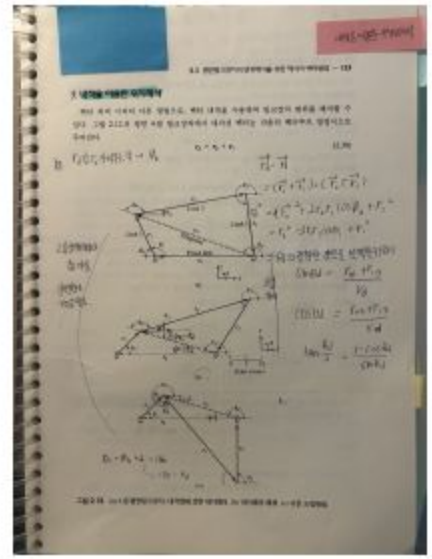
서보모터의 구동각 제한

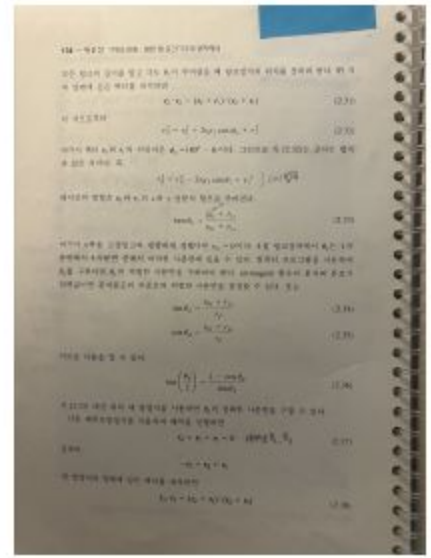
01 조립 한계에 의한 구동각 제한

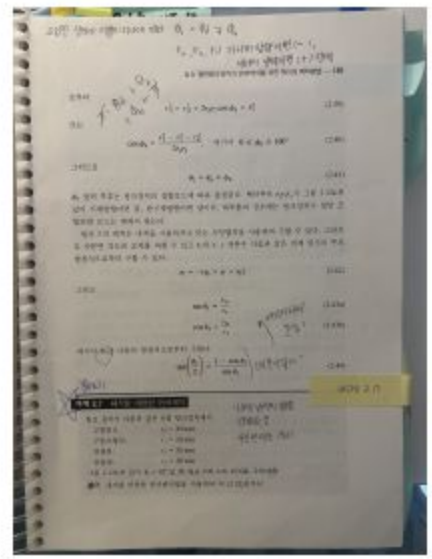


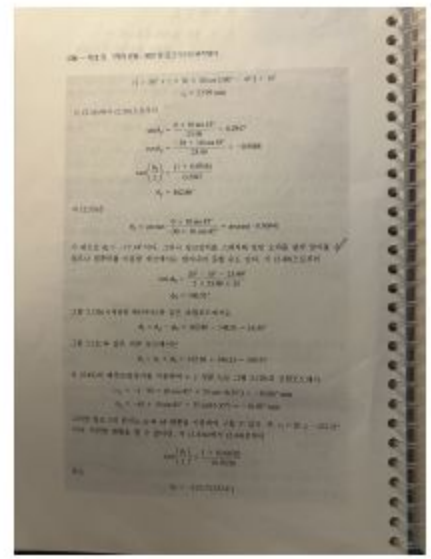


허벅지와 무릎 사이







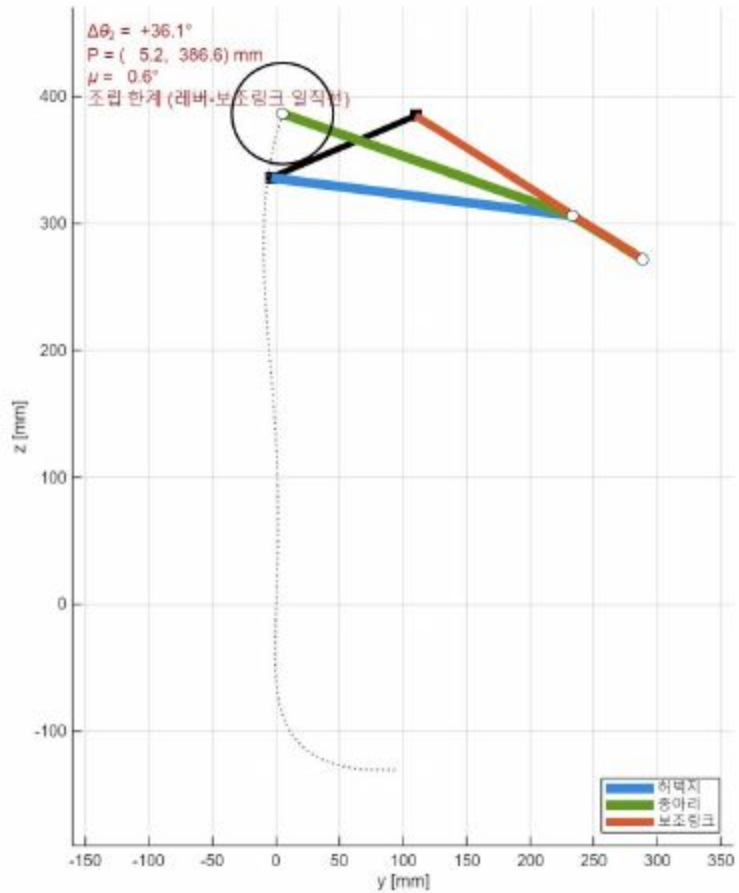


*Charles E. Wilson 의 『 Kinematics and Dynamics of Machinery 』 **133~136p**

목차

서보모터의 구동각 제한

01 조립 한계에 의한 구동각 제한



[삼각형 BCD 가 성립하는 조건]

접힘 한계 : BD 가 최소
→ 레버가 보조링크 위로 접혀 겹친 일직선 ($\mu = 0°$)

펴짐 한계 : BD 가 최대
→ 레버와 보조링크가 쭉 펴진 일직선 ($\mu = 180°$)

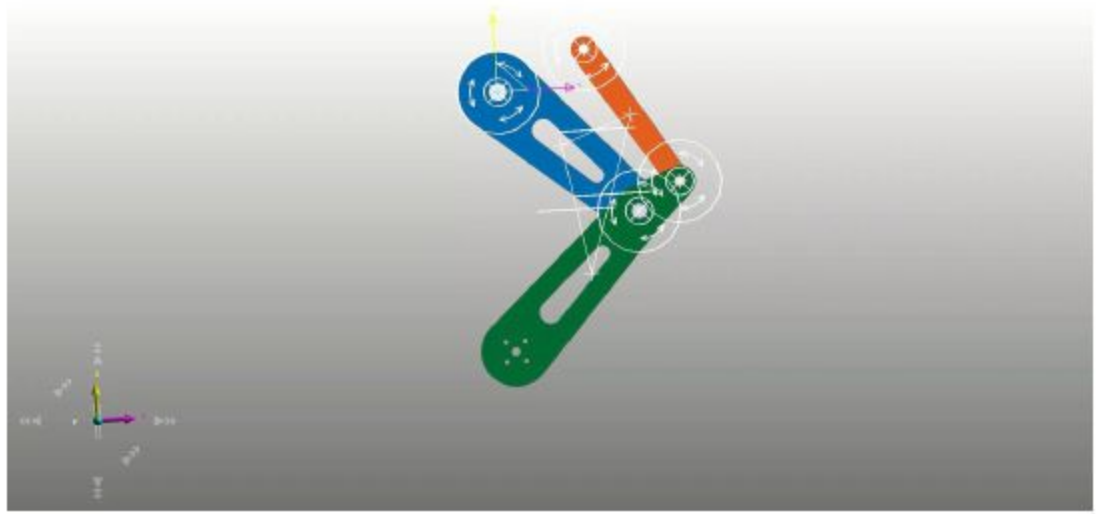
접힘 한계 (BD = c − b): $\cos\varphi = (a^2 + d^2 - (c-b)^2) / (2ad) = 0.8622$
→ $\varphi = 30.44°$

펴짐 한계 (BD = b + c): $\cos\varphi = (a^2 + d^2 - (b+c)^2) / (2ad) = -0.0636$
→ $\varphi = 93.63°$

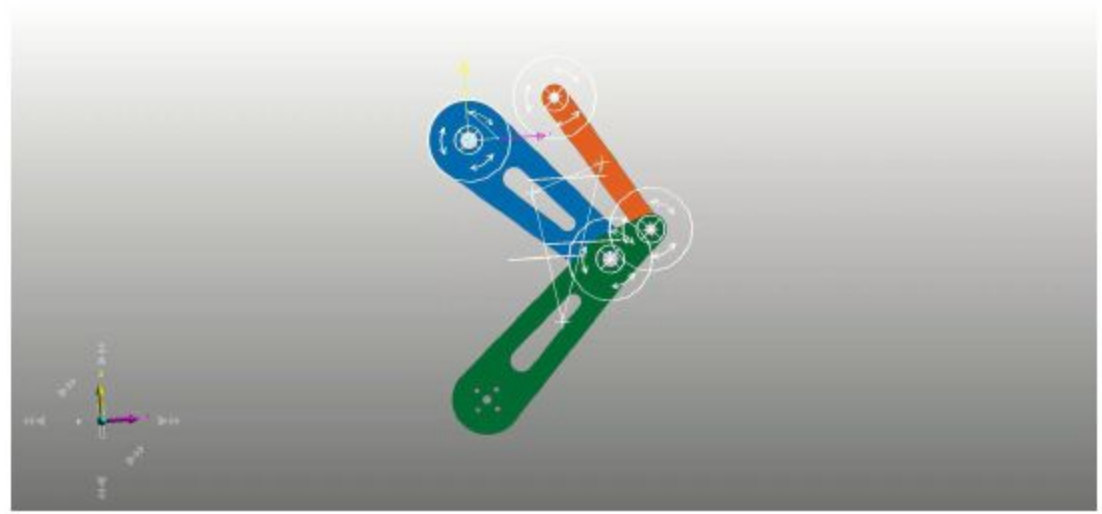
$\Delta\varphi$_ 접힘 = −36.12°
$\Delta\varphi$_ 펴짐 = +27.07°

서보모터의 구동각 제한

02 간섭 한계에 의한 구동각 제한 - RECURDYN



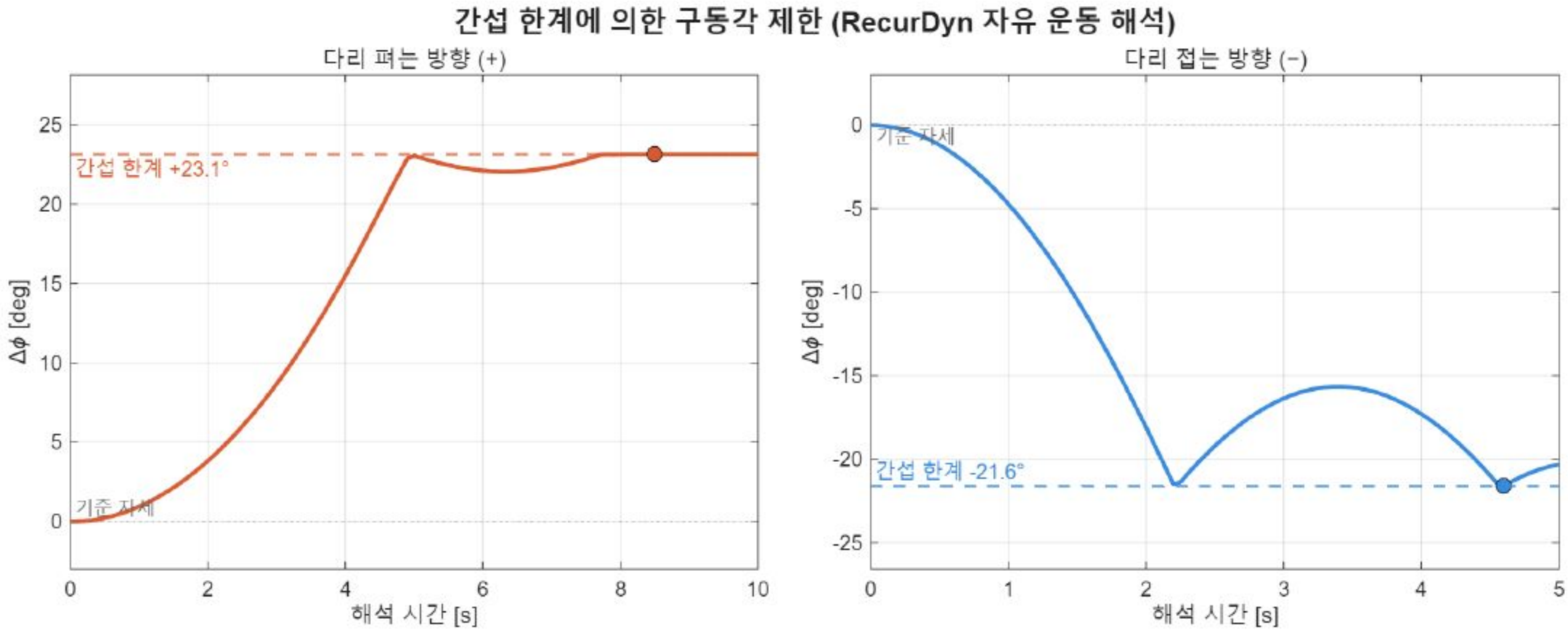
Unfold motion



fold motion

서보모터의 구동각 제한

02 간섭 한계에 의한 구동각 제한 - RECURDYN

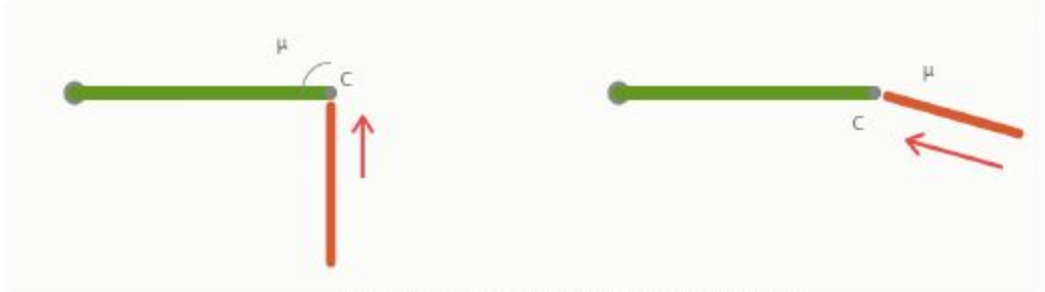


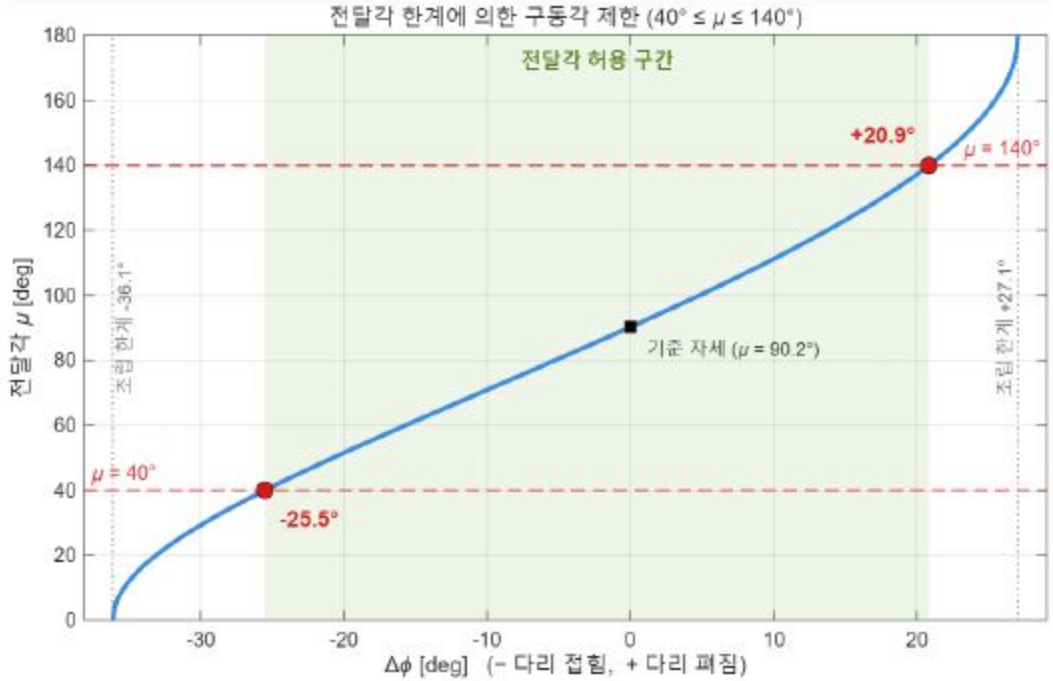
$\Delta\varphi_{\text{접힘}} = -21.6° \;/\; \Delta\varphi_{\text{펴짐}} = +23.1°$

목차

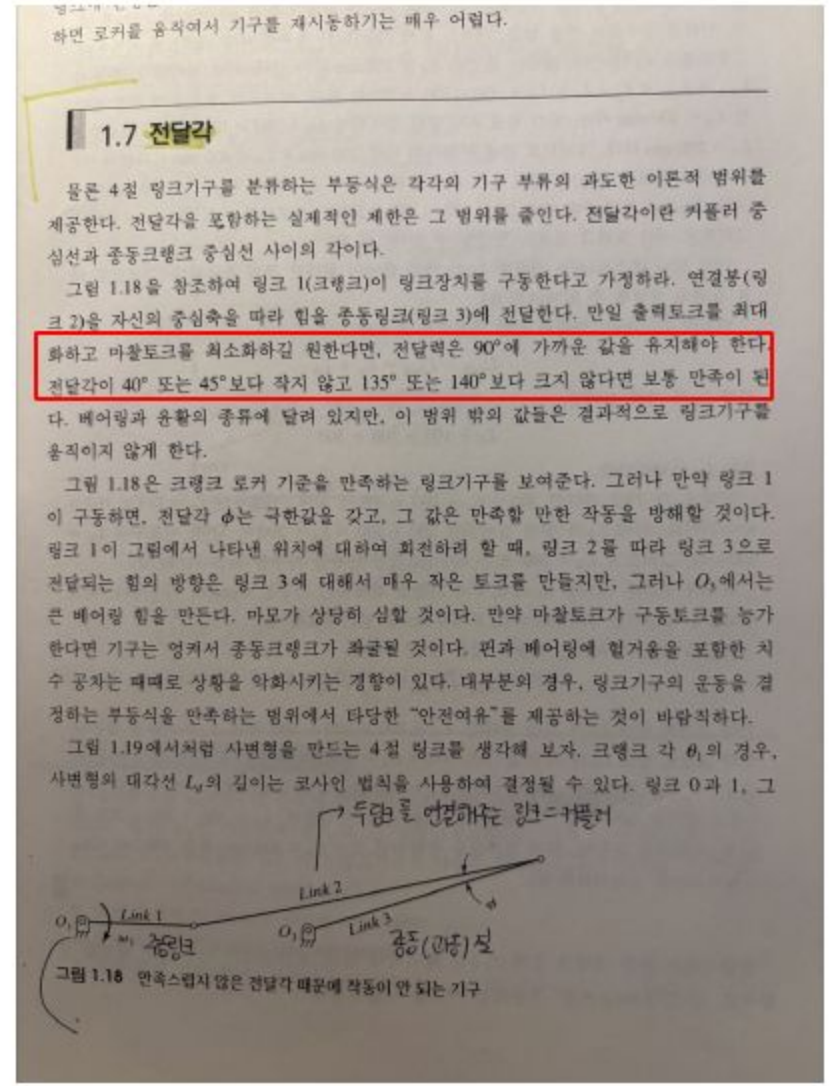
서보모터의 구동각 제한

03 전달각 한계에 의한 구동각 제한





Δφ_ 접힘 = −25.5° / Δφ_ 펴짐 = +20.9°



하면 로커를 움직여서 기구를 재시동하기는 매우 어렵다.

1.7 전달각

물론 4절 링크기구를 분류하는 부등식은 각각의 기구 부류의 과도한 이론적 범위를 제공한다. 전달각을 포함하는 실제적인 제한은 그 범위를 줄인다. 전달각이란 커플러 중심선과 종동크랭크 중심선 사이의 각이다.

그림 1.18을 참조하여 링크 1(크랭크)이 링크장치를 구동한다고 가정하라. 연결봉(링크 2)을 자신의 중심축을 따라 힘을 종동링크(링크 3)에 전달한다. 만일 출력토크를 최대화하고 마찰토크를 최소화하길 원한다면, 전달력은 90°에 가까운 값을 유지해야 한다. 전달각이 40° 또는 45°보다 작지 않고 135° 또는 140°보다 크지 않다면 보통 만족이 된다. 베어링과 윤활의 종류에 달려 있지만, 이 범위 밖의 값들은 결과적으로 링크기구를 움직이지 않게 한다.

그림 1.18은 크랭크 로커 기준을 만족하는 링크기구를 보여준다. 그러나 만약 링크 1이 구동하면, 전달각 ϕ는 극한값을 갖고, 그 값은 만족할 만한 작동을 방해할 것이다. 링크 1이 그림에서 나타낸 위치에 대하여 회전하려 할 때, 링크 2를 따라 링크 3으로 전달되는 힘의 방향은 링크 3에 대해서 매우 작은 토크를 만들지만, 그러나 O_4에서는 큰 베어링 힘을 만든다. 마모가 상당히 심할 것이다. 만약 마찰토크가 구동토크를 능가한다면 기구는 엉켜서 종동크랭크가 좌굴될 것이다. 핀과 베어링에 헐거움을 포함한 치수 공차는 때때로 상황을 악화시키는 경향이 있다. 대부분의 경우, 링크기구의 운동을 결정하는 부등식을 만족하는 범위에서 타당한 "안전여유"를 제공하는 것이 바람직하다.

그림 1.19에서처럼 사변형을 만드는 4절 링크를 생각해 보자. 크랭크 각 θ_1의 경우, 사변형의 대각선 L_d의 길이는 코사인 법칙을 사용하여 결정될 수 있다. 링크 0과 1, 그

커플러를 연결하는 링크

Link 2
Link 1
Link 3
O_2
O_4
종동링크
종동(리듬)링

그림 1.18 만족스럽지 않은 전달각 때문에 작동이 안 되는 기구

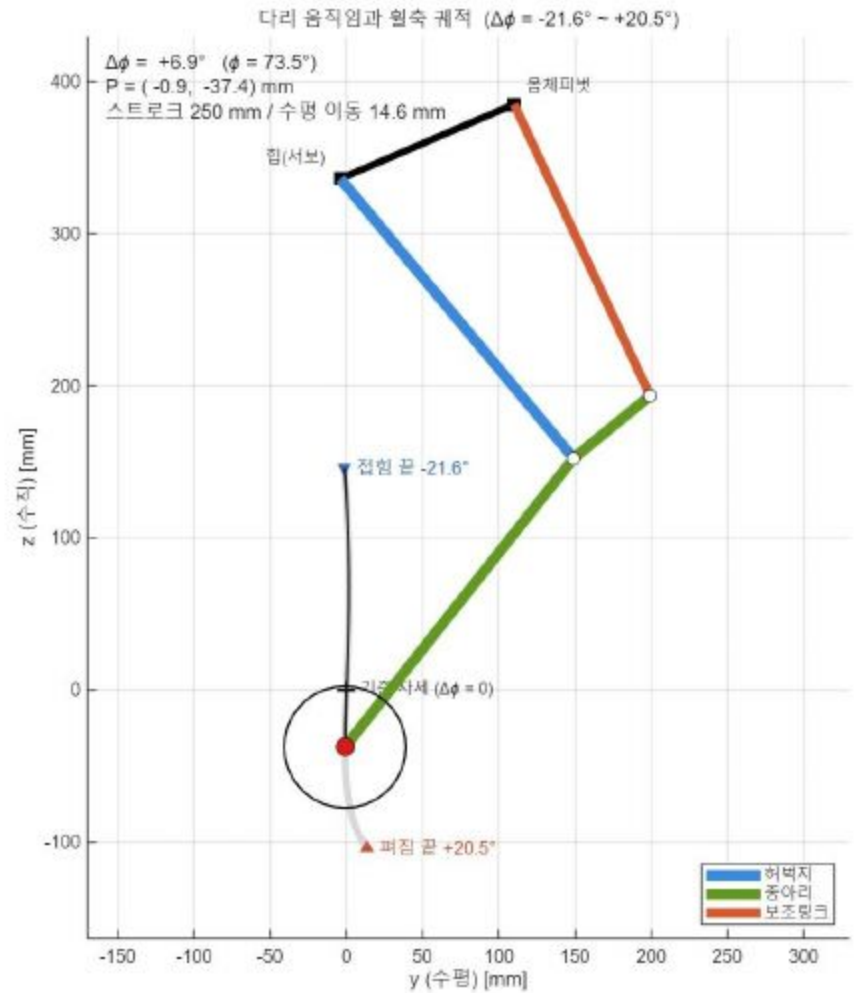
*Charles E. Wilson 의 『 Kinematics and Dynamics of Machinery 』 **128p**

목차

서보모터의 구동각 제한

04 결론

기준	펴는 쪽	접는 쪽
기준 자세	0°	
조립 한계	+27.07°	-36.12°
간섭 한계	+23.1°	-21.6°
전달각 한계	+20.9°	-25.5°
권장 운용 범위	+20.9°	-21.6°



Δφ_ 접힘 = −21.6° / Δφ_ 펴짐 = +20.9°

목차

정역학 해석

00 정역학 해석 목적

➔구동 범위 내 각 자세에서 서보 모터 필요 토크와 , 서보 정격 만족 여부를 검증

➔동역학 해석의 기준값 확보 (저속 동작 시 동역학 결과와 일치해야 함)

정역학 해석

01 각 링크에 대한 정역학 해석

- 해석 범위 : Δφ = −21.6° ~ +20.9° (구동 가능 구간)
- 하중 : 토르소 3 kg → W = 14.7 N (다리 1 개 기준)
- 링크 질량 : 허벅지 0.2 / 종아리 0.2 / 보조링크 0.1 kg
  → 지면 반력 G = 19.6 N

- 가정 : 정적 평형 , 무게중심은 링크 중앙 ,
  마찰 · 스프링 무시

✳ 정역학 해석 - 힘평형 식

기준 ↑ y →

⊕

u: 보조링크 방향 단위벡터

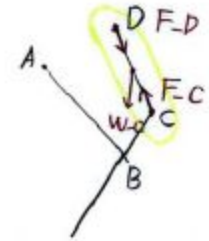
G: 지면 반력 ($G = W + W_{\perp} + W_S + W_C$)

F_K : 허벅지가 종아리에 주는 무릎 핀힘

H: 토르소가 허벅지에 주는 힘 핀힘

T: 서보토크

① 보조링크



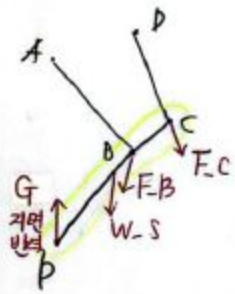
$\Sigma F = 0$

$: F_C + F_B - W_C \hat{j} = 0$

$\Sigma M_D = 0$

$: r_{DC} \times F_C + r_{DG_C} \times (-W_C \hat{j}) = 0$

② 종아리



$\Sigma F = 0$

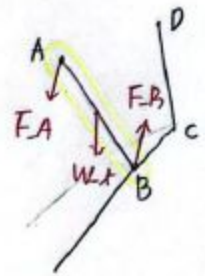
$: G\hat{j} - F_C + F_B - W_S \hat{j} = 0$

$\Sigma M_B = 0$

$: r_{BP} \times G\hat{j} + r_{BC} \times (-F_C) + r_{BG_S} \times (-W_S \hat{j}) = 0$

(우선방향.)

③ 허벅지



$\Sigma F = 0$

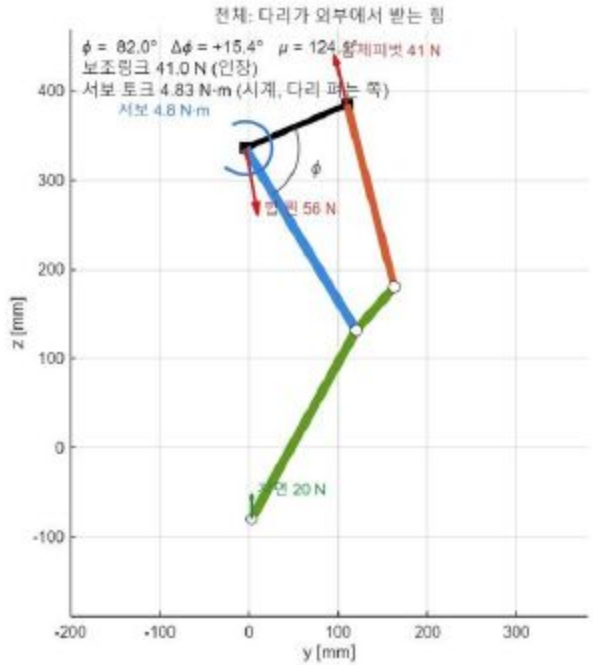
$: F_A - F_B - W_\perp \hat{j} = 0$

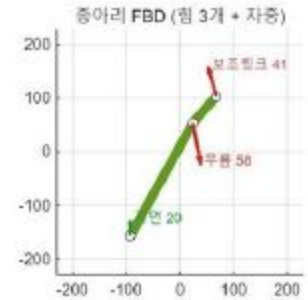
$\Sigma M_A = 0$

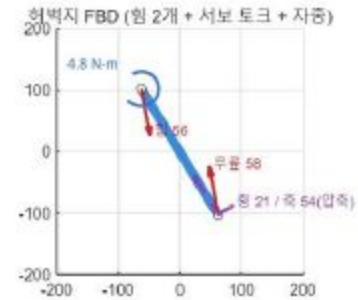
$: T + r_{AB} \times (-F_B) + r_{AG_\perp} \times (-W_\perp \hat{j}) = 0$

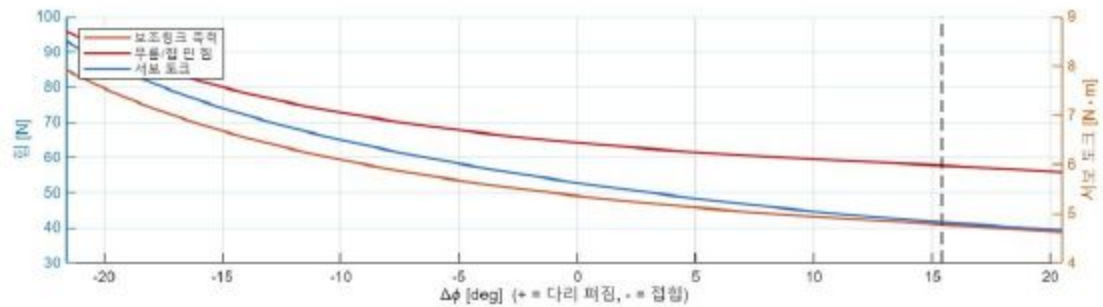
정역학 해석

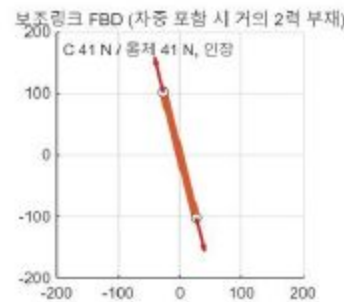
01 각 링크에 대한 정역학 해석











목차

정역학 해석

01 결론

[해석 결과]
- 다리를 접을수록 모든 하중 증가
- 지면 반력은 19.6 N 으로 일정하지만 , 접힘 끝 (Δφ = −21.6°) 에서 보조링크 축력 85 N, 핀 반력 96 N, 서보 토크 8.5 N·m 로 최대
- 압축이 발생하지 않아 좌굴 우려 없음 (수직 하중 조건)

[서보 판정]
- 정적 최대 토크 8.5 N·m < 정격 9 N·m (95%) → 구동 가능
- 여유가 작아 접힌 자세 장시간 유지는 지양

실제 동작 시 관성 하중 추가 → 동역학 해석으로 피크 토크 확인 필요

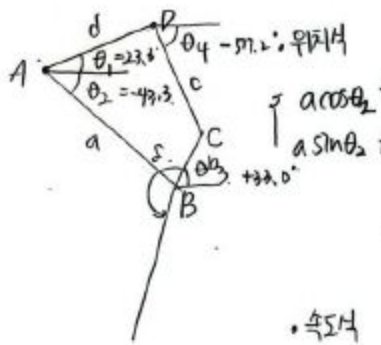
DM-J4340P-2EC Motor Specification Table (24V / 48V)

Rated Voltage	24V / 24-48V
Rated Current	3.0 A (24 V) / 2.5 A (24–48 V)
Peak Current	8 A (24 V) / 9 A (24–48 V)
Rated Torque	9 Nm
Peak Torque	27 Nm
Rated Speed	36 rpm
Max No-Load Speed	52 rpm / 100 rpm
Gear Ratio	40:1
Pole Pairs	14
Phase Inductance	360 μH
Phase Resistance	0.88 Ω
Motor Size	57*56.5 mm
Motor Weight	Approx. 375g
Encoder Resolution	14-bit
Number of Encoders	2
Encoder Type	Magnetic; Dual Encoders
Control Interface	CAN@1Mbps
Tuning Interface	UART@921600bps
MIT Mode / Speed Mode / Position Mode / Force-Position Hybrid Mode	
Driver Overtemperature Protection	Exits Enable Mode when ≥120°C
Motor Overtemperature Protection	User-defined; recommended ≤100°C
Overvoltage Protection	≤32V (24V) / ≤52V (48V); exits Enable Mode
Undervoltage Protection	Exits Enable Mode if voltage <15V
Overcurrent Protection	Recommended ≤9.8A; exits Enable Mode
Communication Loss Protection	Exits Enable Mode if no CAN command received within timeout period

기구학 – 속도 / 가속도

01 주기에 따른 링크의 각속도와 각가속도 계산 (T=5.0)

✳ 기구학해석 - 속도 / 가속도 ✳



$$\begin{cases} a\cos\theta_2 + b\cos\theta_3 - c\cos\theta_4 - d\cos\theta_1 = 0 \\ a\sin\theta_2 + b\sin\theta_3 - c\sin\theta_4 - d\sin\theta_1 = 0 \end{cases}$$

한번미분 ↓ 속도식

• 속도식

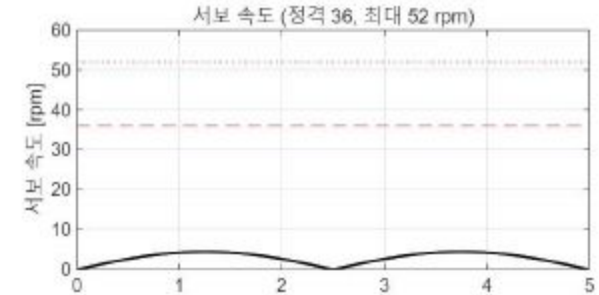
$$\begin{cases} -b\sin\theta_3 \cdot \omega_3 + c\sin\theta_4 \cdot \omega_4 = a\omega_2 \cdot \sin\theta_2 \\ b\cos\theta_3 \cdot \omega_3 - c\cos\theta_4 \cdot \omega_4 = -a\omega_2\cos\theta_2 \end{cases}$$

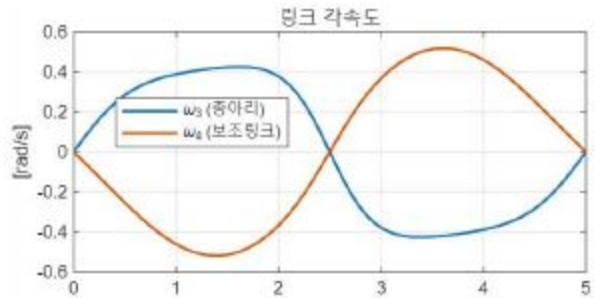
↓

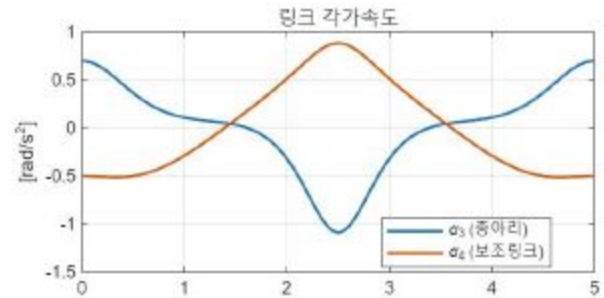
• 가속도식

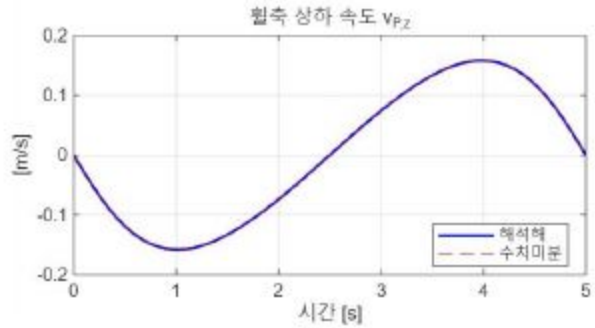
$$\begin{cases} -b\sin\theta_3 \cdot \alpha_3 + c\sin\theta_4 \cdot \alpha_4 = a\alpha_2\sin\theta_2 + a\omega_2^2\cos\theta_2 \\ \qquad\qquad + b\omega_3^2\cos\theta_3 - c\omega_4^2\cos\theta_4 \\ b\cos\theta_3 \cdot \alpha_3 - c\cos\theta_4 \cdot \alpha_4 = -a\alpha_2\cos\theta_2 + a\omega_2^2\sin\theta_2 \\ \qquad\qquad + b\omega_3^2\sin\theta_3 - c\omega_4^2\sin\theta_4 \end{cases}$$

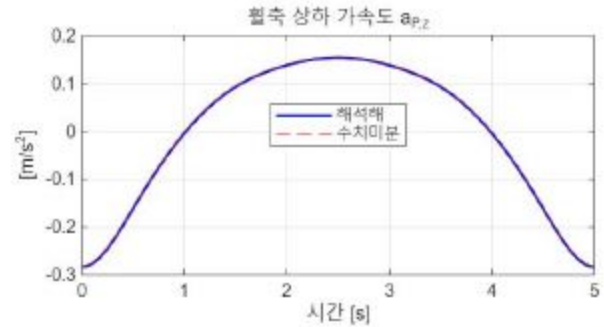












목차

동역학 – 주행 없이 스쿼트

01 T=5.0 일때 토르소 관성과 링크 관성을 고려한 동역학 해석

✳ 동역학 해석 (뉴턴 오일러 방정식)

$\Sigma F = m a_M$ (무게중심)

$\Sigma M = I \cdot \alpha$

① 지면 반력

$G_z = (m_T + m_t + m_s + m_c)\cdot g + m_T \cdot a_{T,z} + \Sigma m_i \cdot a_{Mi,z}$

(전체 무게 (정역학) / 토르소만큼 / 링크 관성)

$G_y = m_t \cdot a_{Mt,y} + m_s \cdot a_{MS,y} + m_c \cdot a_{Mc,y}$

② 보조링크 CD

$\Sigma M_D = 0 \;;\; r_{DC} \times F_C + r_{PMc} \times f_C - I_C \cdot \alpha_4 = 0$

③ 종아리

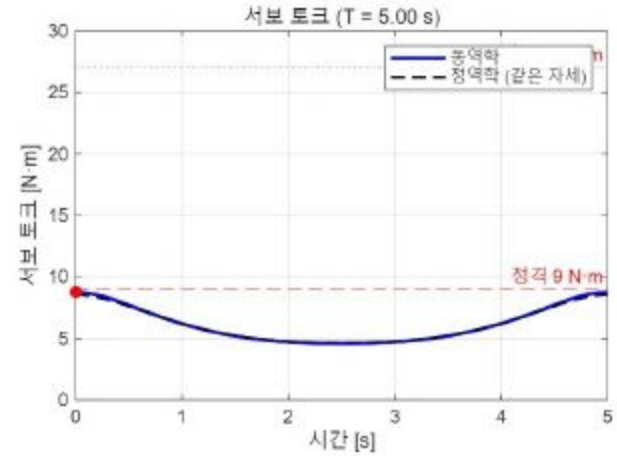
$\Sigma M_B = 0 \;;\; r_{BP} \times G + r_{BC} \times (-F_C) + r_{BMs} \times f_s - I_s \cdot \alpha_3 = 0$

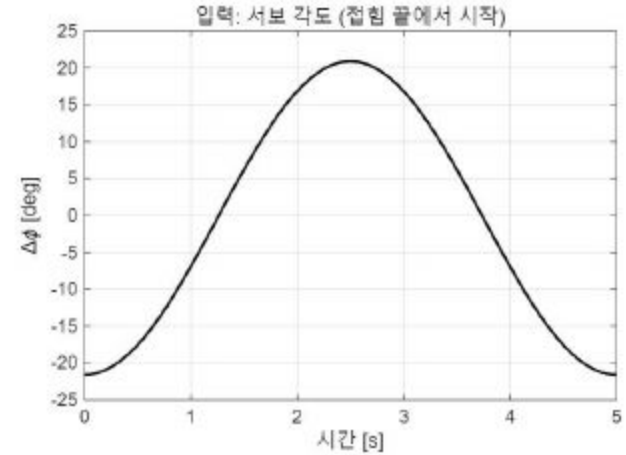
$\Sigma F = 0 \;:\; G - F_C + F_B + f_s = 0$

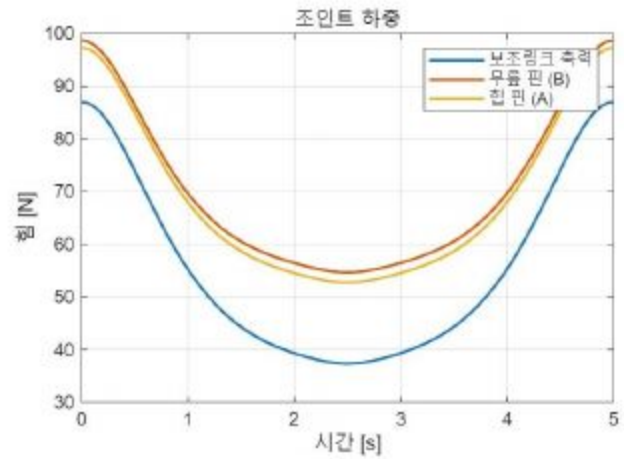
④ 허벅지

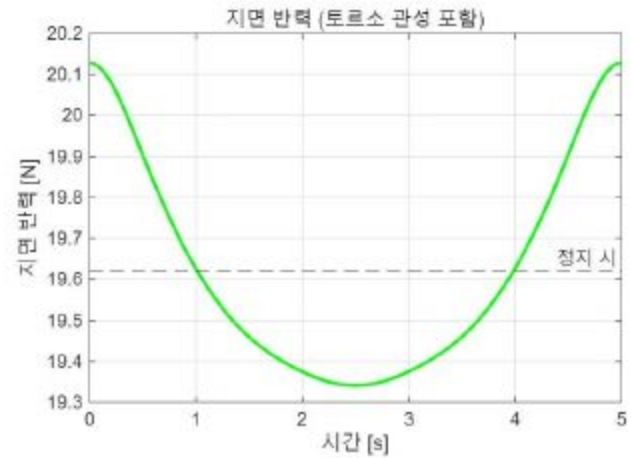
$\Sigma F = 0 \;:\; F_A - F_B + f_t = 0$

$\Sigma M_A = 0 \;;\; \tau + r_{AB} \times (-F_B) + r_{AMt} \times f_t - I_t \cdot \alpha_2 = 0$





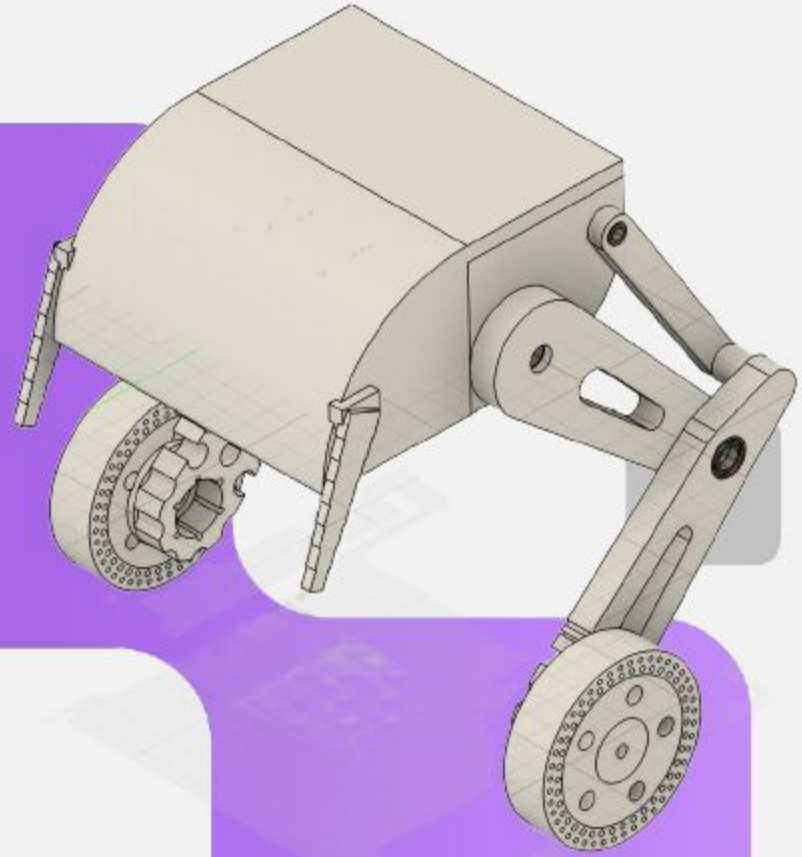




목차

PART 05

ELECTRONICS BRING-UP



전장 구동 테스트

서보 · 인휠모터 단품 구동 , LattePanda 선정 · 부팅 , ODrive 휠 구동 , BNO085 IMU 테스트

서보 · 인휠모터 단품 구동 42–43p

LattePanda 선정 · 부팅 44–46p

LattePanda → ODrive 휠 구동 47–48p

BNO085 IMU 테스트 49–52p

IMU 코드 (Teensy · 호스트) 53–54p

모듈화 – 각모듈 구동 확인

04 토르소의 서브모터 구동



목차

모듈화 – 각모듈 구동 확인

04 다리 모듈의 휠모터 구동





메인보드 부팅 / 환경 테스트

01 라떼판다 선정이유

라떼판다-4G,64GB 윈도우10 엔터프라이즈 버전 (라이센스 포함) [독점, 공식대리점] [DFR0470] ICBANQ

인텔 Cherry trail 1.8GHz 쿼드코어, windows10 기본 탑재, 아두이노 호환, 안드로이드, 우분투 사용 가능, latte panda



추천 무료 국내 당일

소비자가	380,000 원
(vat 포함)	418,000 원
상품코드	P008000066
제 조 사	DFRobot
마일리지	?
배송기간	당일출고
최소구매수량	1개
확인사항	KC인증 번호:R-CMI-ICQ-LP

수량선택 −

- Windows 기반 Python/C++ 개발 환경 활용 가능
- 다양한 센서 및 MCU 와의 높은 확장성
- 소형 · 경량으로 로봇 온보드 탑재 가능
- 고수준 연산과 실시간 제어의 역할 분리

라떼판다-4G,64GB 윈도우10 엔터프라이즈 버전 (라이센스 포함) [독점, 공식대리점] [DFR0470] : 산업용 싱글보드 > AI 산업용 보드 > AI 제품

메인보드 부팅 / 환경 테스트

01 라떼판다 기능

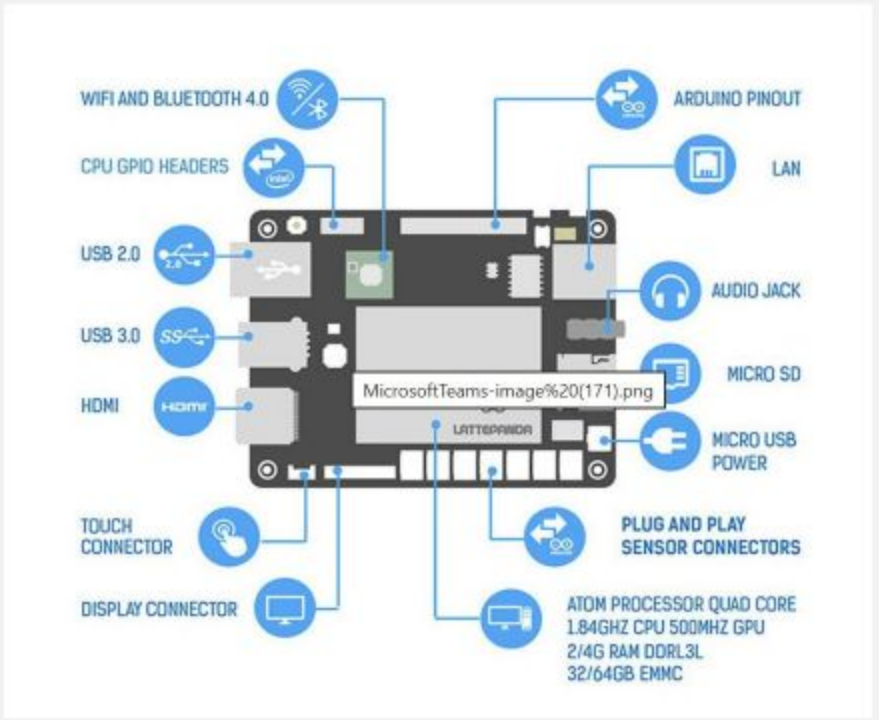
라떼판다 DFR0470 제품 사양

제품 사양 (1)

	DFR0444	DFR0418	DFR0419	DFR0470
On board RAM	2GB	2GB	4GB	4GB
eMMC	32GB	32GB	64GB	64GB
라이센스키 인증	NO	YES	NO	YES

제품 사양 (2)

	라떼판다 오리지널			
	DFR0444	DFR0418	DFR0419	DFR0470
CPU	Intel Cherry Trail Z8350 Quad Core			
RAM	2~4GB			
저장소	32~64GB			
블루투스	Bluetooth 4.0			
전원입력	5V / 2A			
상품코드	P011852166	P007565470	P007542823	P008000066
번들배터리	번들배터리 미포함			
KC인증	KC 인증 완료			



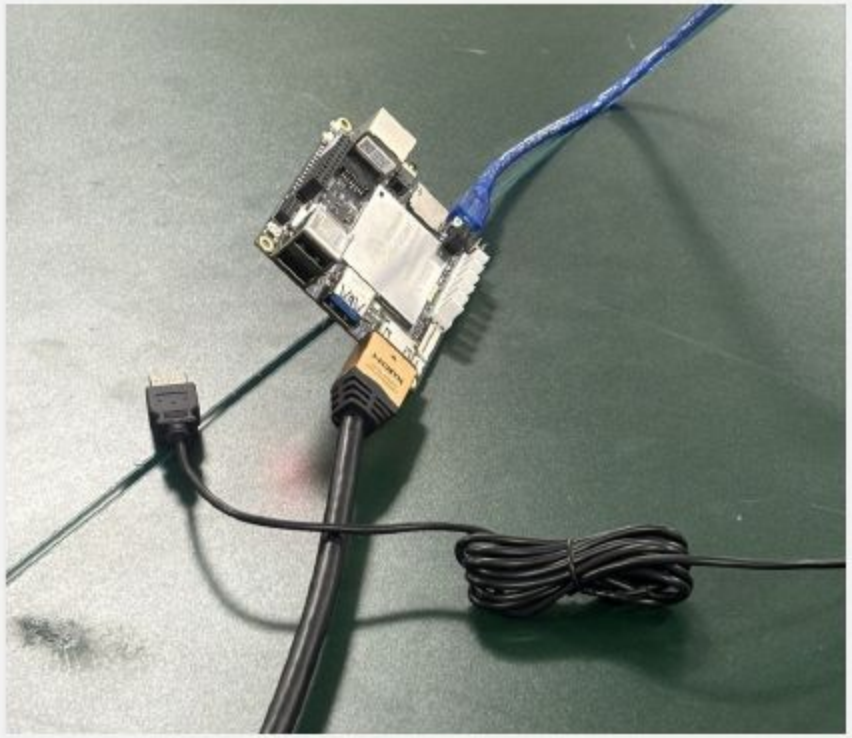
- High-Level Controller
- 로봇 상태 모니터링 및 데이터 처리
- 사용자 명령 및 동작 제어
- Teensy 4.1 과 Serial 통신
- Teensy 에 목표 자세 · 속도 · 동작 명령 전달

라떼판다-4G,64GB 윈도우10 엔터프라이즈 버전 (라이센스 포함) [독점, 공식대리점] [DFR0470] : 산업용 싱글보드 > AI 산업용 보드 > AI 제품

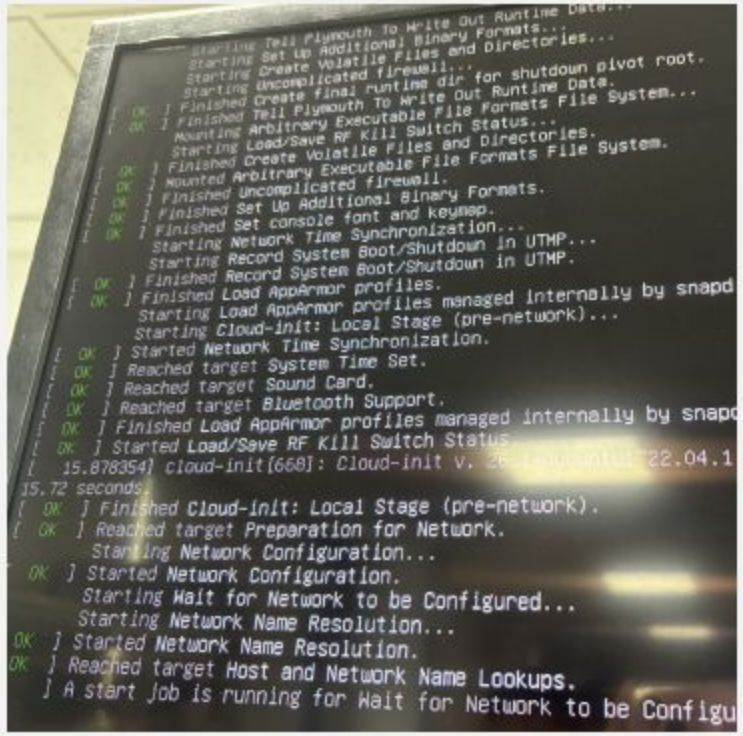
목차

메인보드 부팅 / 환경 테스트

01 구동 테스트







라떼판다-4G,64GB 윈도우10 엔터프라이즈 버전 (라이센스 포함) [독점, 공식대리점] [DFR0470] : 산업용 싱글보드 > AI 산업용 보드 > AI 제품

목차

메인보드 부팅 / 환경 테스트

01 LATTE PANDA 를 통한 휠 구동 테스트



```
# 모터
odrv0.axis0.motor.config.pole_pairs = 15
odrv0.axis0.motor.config.torque_constant = 1
odrv0.axis0.motor.config.calibration_current = 5
odrv0.axis0.motor.config.resistance_calib_max_voltage = 8
odrv0.axis0.motor.config.current_lim = 10

# 엔코더 (J3: A, B)
odrv0.axis0.encoder.config.mode =
ENCODER_MODE_INCREMENTAL
odrv0.axis0.encoder.config.cpr = 3200
odrv0.axis0.encoder.config.use_index = False
odrv0.axis0.config.startup_encoder_index_search = False

# 제어 ( 시작값 , 이후 튜닝 )
odrv0.axis0.controller.config.vel_limit = 3
odrv0.axis0.controller.config.pos_gain = 20
odrv0.axis0.controller.config.vel_gain = 0.5
odrv0.axis0.controller.config.vel_integrator_gain = 1

odrv0.save_configuration()
odrv0.reboot()
```

메인보드 부팅 및 인휠모터 구동

01 인휠모터 구동



```
%% 캘리브레이션 확인 - 이걸 치면 바로 회전 시작 %%
odrv0.axis0.error = 0
odrv0.axis0.motor.error = 0
odrv0.axis0.encoder.error = 0
odrv0.axis0.requested_state = 3


odrv0.axis0.current_state
odrv0.axis0.motor.is_calibrated
dump_errors(odrv0)


%%POS 와 velocity 출력 - 이걸 치면 바로 회전 시작 %%
import time
from collections import deque

odrv0.axis0.controller.config.control_mode = 2
odrv0.axis0.controller.config.vel_limit = 5
odrv0.axis0.requested_state = 8
time.sleep(0.3)
odrv0.axis0.controller.input_vel = 1.5

prev_pos = odrv0.axis0.encoder.pos_estimate
prev_time = time.time()
window = deque(maxlen=5)  # 최근 5 개 값 평균

for i in range(50):
    time.sleep(0.1)
    now = time.time()
    pos = odrv0.axis0.encoder.pos_estimate
    dt = now - prev_time
    vel_calc = (pos - prev_pos) / dt
    window.append(vel_calc)
    smoothed = sum(window) / len(window)
    print(f"[{i}] raw={vel_calc:.3f}, smoothed={smoothed:.3f} turns/s")
    prev_pos = pos
    prev_time = now

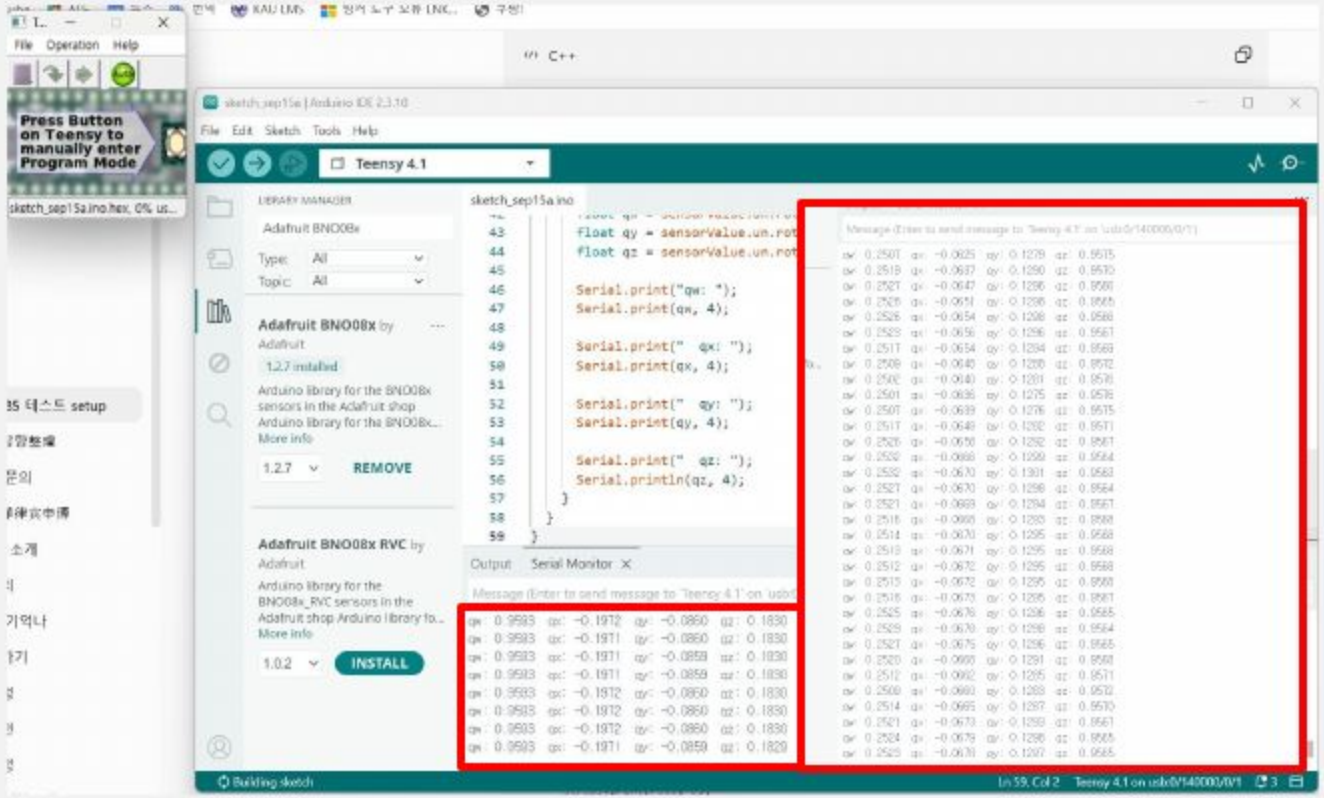
odrv0.axis0.controller.input_vel = 0
odrv0.axis0.requested_state = 1
```

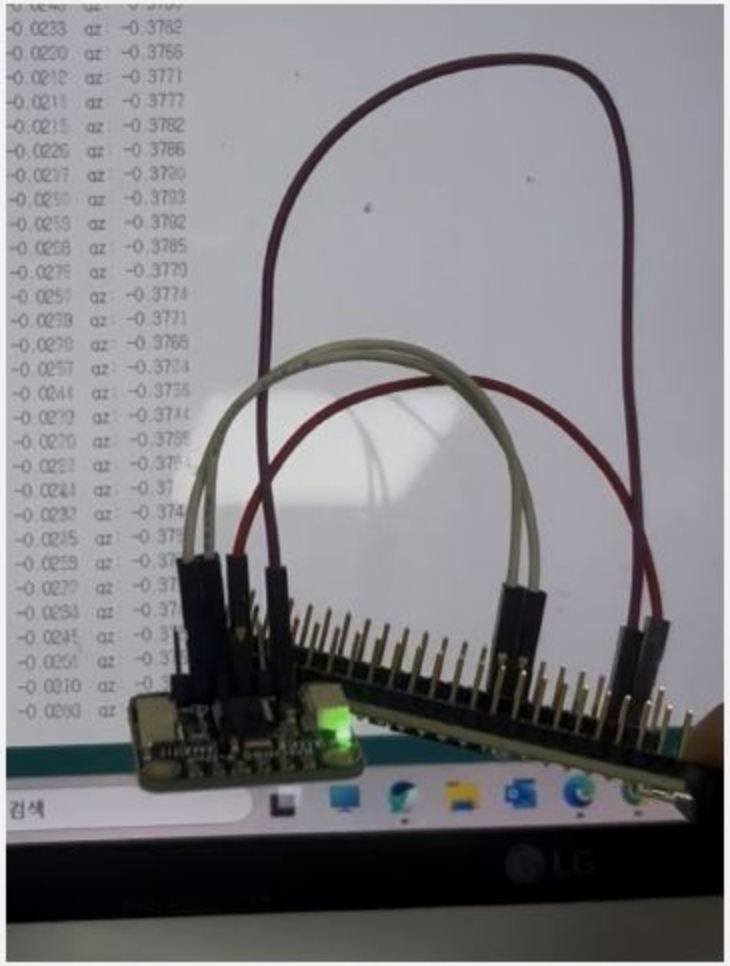
IMU 테스트

01

구동 테스트

- IMU 센서 (BNI085): 3 축 가속도계 +3 축 자이로 +3 축 지자기계 정보 포함



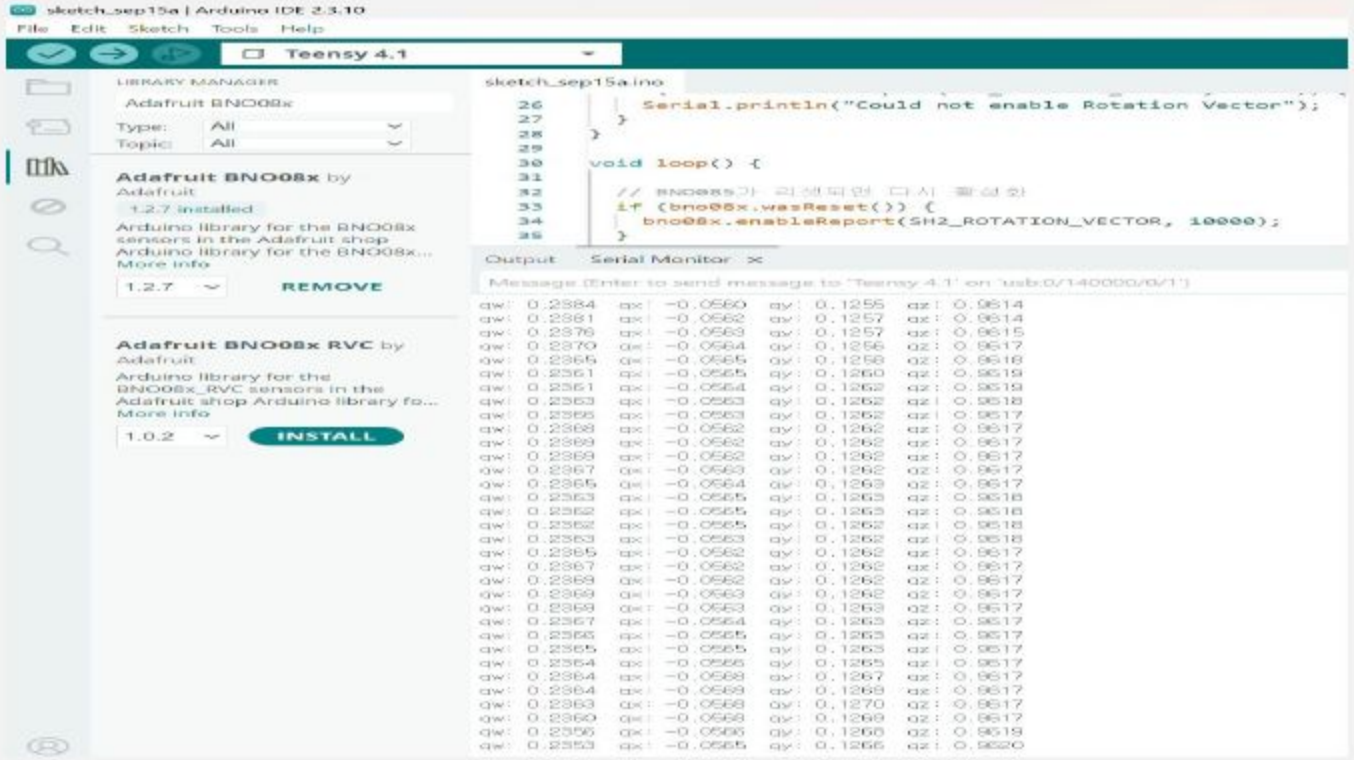


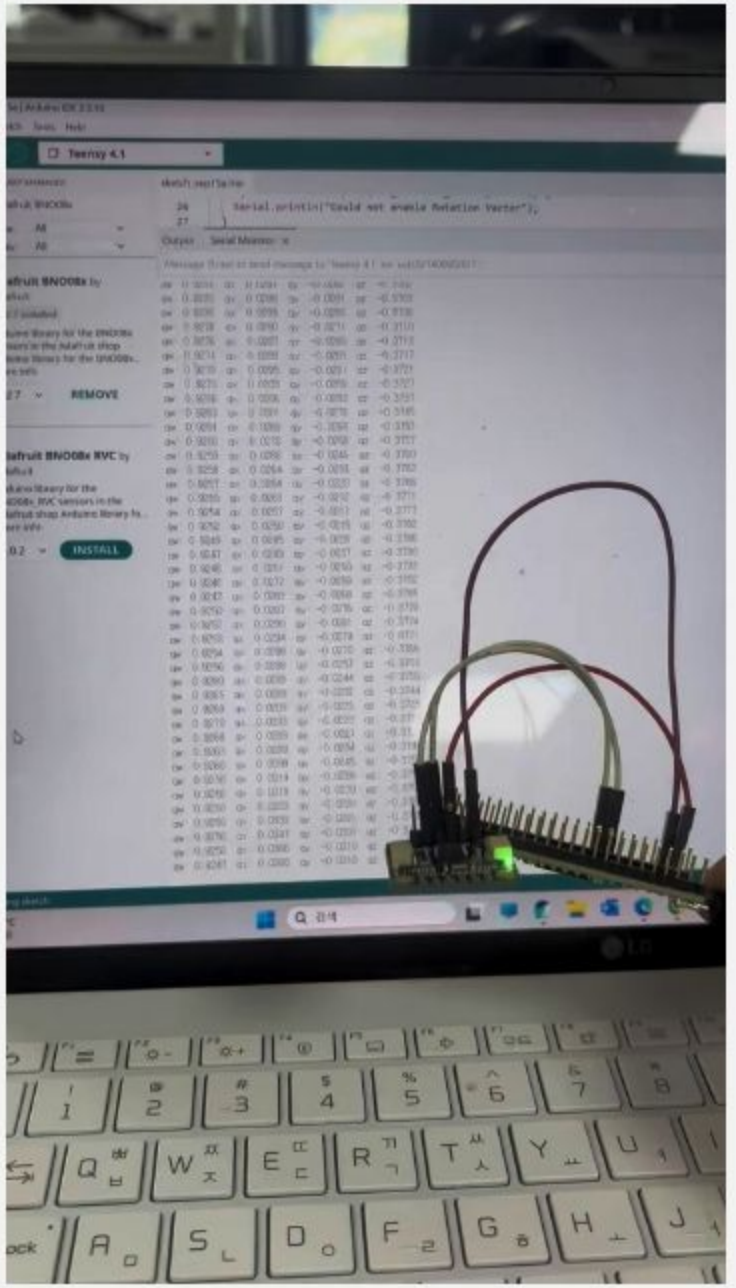
IMU 테스트

01

구동 테스트 - (qw,qx,qy,qz) 쿼터니언 값 출력 테스트

- IMU 센서 (BNI085) 를 Teensy 4.1 에 연결하여 IMU 의 구동성능을 확인함





IMU 테스트

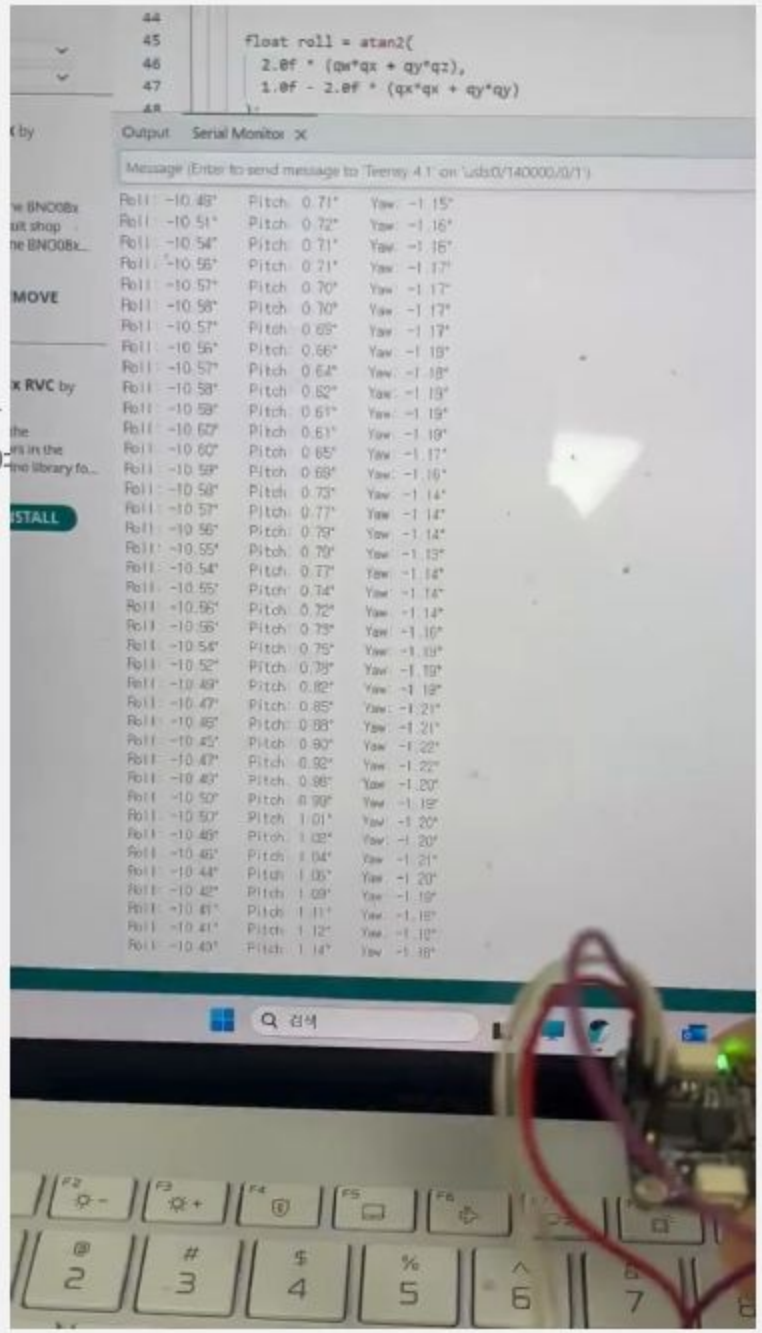
01 **구동 테스트 - Roll, Pitch, Yaw 값 출력 테스트**

- IMU 센서 (BNI085) 를 Teensy 4.1 에 연결하여 IMU 의 구동성능을 확인함
- BNO085 의 센서 퓨전으로 계산된 Quaternion 을 Euler Angle 로 변환하여
  로봇의 3 축 자세 (Roll, Pitch, Yaw) 를 추정

$$Roll = \tan^{-1}\left(\frac{2(q_w q_x + q_y q_z)}{1 - 2(q_x^2 + q_y^2)}\right)$$

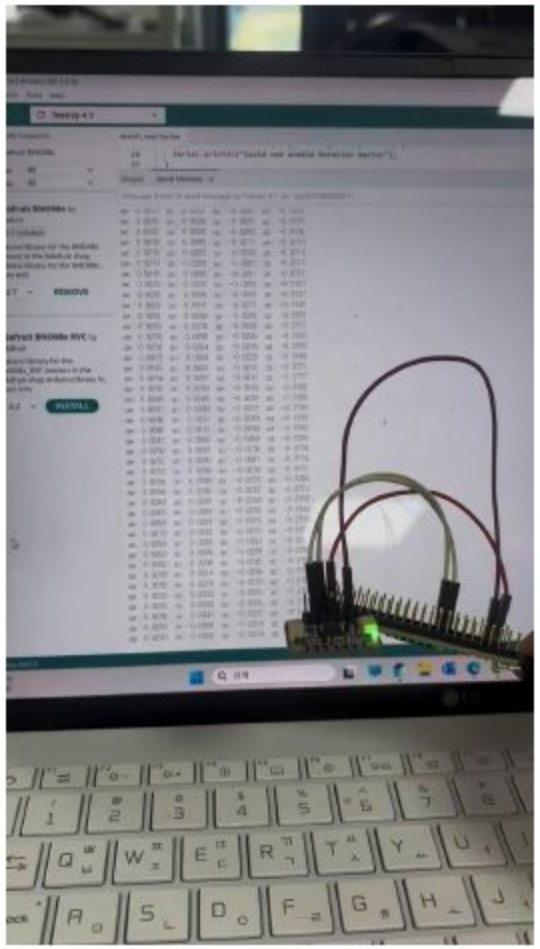
$$Pitch = \sin^{-1}\left(2(q_w q_y - q_z q_x)\right)$$

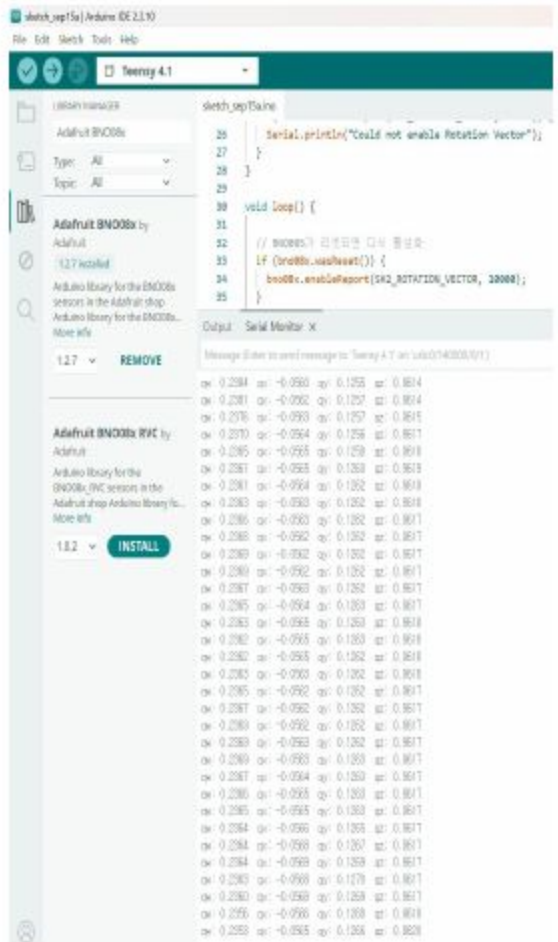
$$Yaw = \tan^{-1}\left(\frac{2(q_w q_z + q_x q_y)}{1 - 2(q_y^2 + q_z^2)}\right)$$

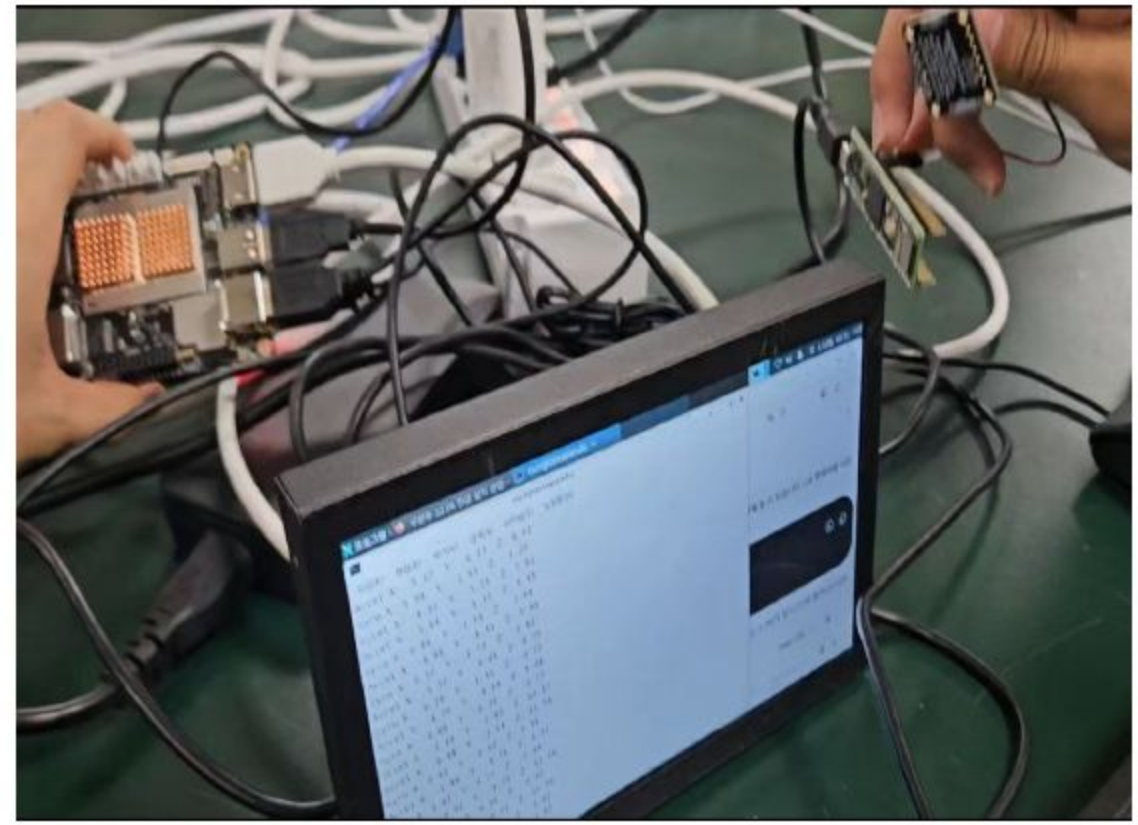


IMU 테스트

02 IMU – 메인보드 테스트







IMU 테스트

03 TEENSY 4.1 센서 데이터 송신 코드 (ARDUINO) / 우분투 호스트 수신 및 파싱 코드

```
#include <Adafruit_BNO08x.h>

Adafruit_BNO08x bno08x;
sh2_SensorValue_t sensorValue;

void setup() {
 Serial.begin(115200);
 while (!Serial) delay(10);

 // BNO085 I2C 초기화
 if (!bno08x.begin_I2C()) {
  while (1) { delay(10); }
 }
 // 게임 회전 벡터 (Game Rotation Vector) 리포트 활성화
 bno08x.enableReport(SH2_GAME_ROTATION_VECTOR);
}

void loop() {
 if (bno08x.getSensorEvent(&sensorValue)) {
  if (sensorValue.sensorId == SH2_GAME_ROTATION_VECTOR) {
   // 쿼터니언 데이터 (i, j, k, real) 를 CSV 형태로 시리얼 출력
   Serial.print(sensorValue.un.gameRotationVector.i, 4);
   Serial.print(",");
   Serial.print(sensorValue.un.gameRotationVector.j, 4);
   Serial.print(",");
   Serial.print(sensorValue.un.gameRotationVector.k, 4);
   Serial.print(",");
   Serial.println(sensorValue.un.gameRotationVector.real, 4);
  }
 }
 delay(10);
}
```

```
import serial
import sys
import time

# 우분투 환경에 맞는 시리얼 포트 설정
SERIAL_PORT = '/dev/ttyACM0'  # 필요시 ttyACM1 로 변경
BAUD_RATE = 115200

def main():
print(f"Connecting to {SERIAL_PORT} at {BAUD_RATE} baud...")
try:
ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
time.sleep(2) # 연결 안정화 대기
except Exception as e:
print(f"Failed to connect to serial port: {e}")
sys.exit(1)

print("Listening for BNO085 IMU data... Press Ctrl+C to exit.")

try:
while True:
if ser.in_waiting > 0:
line = ser.readline().decode('utf-8', errors='ignore').strip()
if line:
# 쉼표 (,) 로 데이터 분리
parts = line.split(',')
if len(parts) == 4:
try:
qx = float(parts[0])
qy = float(parts[1])
qz = float(parts[2])
qw = float(parts[3])

# 수신된 쿼터니언 데이터 출력 ( 제어 알고리즘 및 ROS 2 연동 지점 )
print(f"Quaternion -> X: {qx:.3f}, Y: {qy:.3f}, Z: {qz:.3f}, W: {qw:.3f}")
except ValueError:
pass
except KeyboardInterrupt:
print("\nExiting program...")
finally:
ser.close()

if __name__ == '__main__':
main()
```

IMU 테스트

01 우분투 호스트 수신 및 파싱 코드

```
import serial
import sys
import time

# 우분투 환경에 맞는 시리얼 포트 설정
SERIAL_PORT = '/dev/ttyACM0'  # 필요시 ttyACM1 로 변경
BAUD_RATE = 115200

def main():
    print(f"Connecting to {SERIAL_PORT} at {BAUD_RATE} baud...")
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
        time.sleep(2) # 연결 안정화 대기
    except Exception as e:
        print(f"Failed to connect to serial port: {e}")
        sys.exit(1)

    print("Listening for BNO085 IMU data... Press Ctrl+C to exit.")

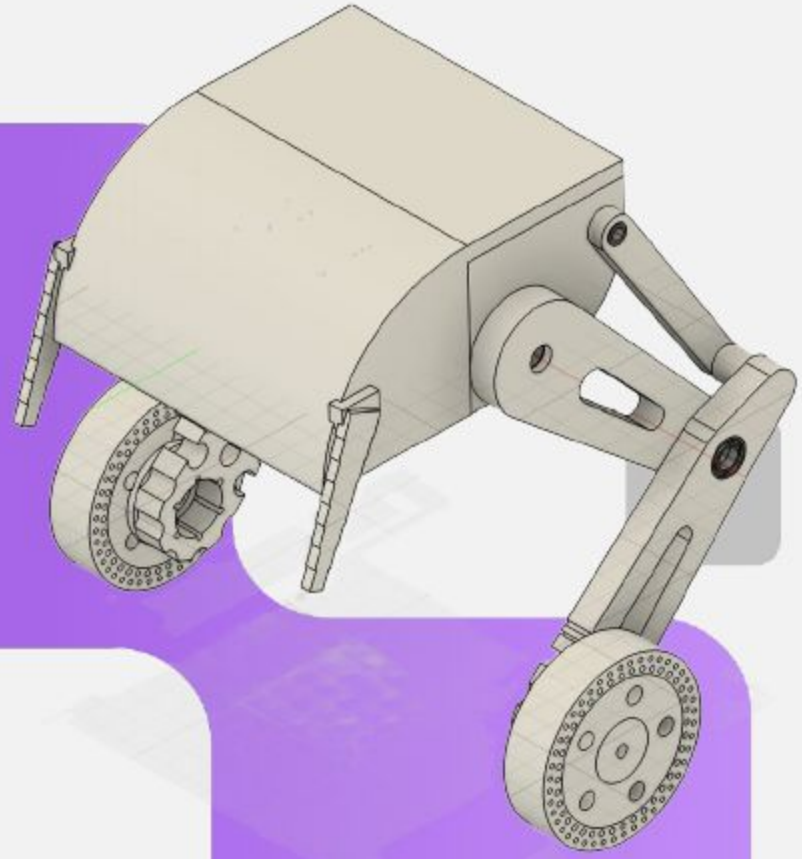
    try:
        while True:
            if ser.in_waiting > 0:
                line = ser.readline().decode('utf-8', errors='ignore').strip()
                if line:
                    # 쉼표 (,) 로 데이터 분리
                    parts = line.split(',')
                    if len(parts) == 4:
                        try:
                            qx = float(parts[0])
                            qy = float(parts[1])
                            qz = float(parts[2])
                            qw = float(parts[3])

                            # 수신된 쿼터니언 데이터 출력 ( 제어 알고리즘 및 ROS 2 연동 지점 )
                            print(f"Quaternion -> X: {qx:.3f}, Y: {qy:.3f}, Z: {qz:.3f}, W: {qw:.3f}")
                        except ValueError:
                            pass
    except KeyboardInterrupt:
        print("\nExiting program...")
```

목차

PART 06

CAN INTEGRATION PIPELINE



CAN 통합 구동 계획

DM4340P · ODrive · IMU 를 Teensy 4.1 의 CAN 두 줄로 묶어 500 Hz 로 제어하고 ROS 2 로 올리는 파이프라인 · 코드 · 일정

현황 · 전장 구성 · 시스템 구조 56–58p

전원 · CAN 버스 설계 59–60p

DM4340P · ODrive 프로토콜과 설정 61–63p

펌웨어 구조 · 상태머신 · 코드 64–69p

ROS 2 브리지 · 코드값 70–72p

Bring-up · 일정 · 안전 73–76p

CAN 통합 구동 — 현황

01 단품 구동은 확인 , 남은 일은 Teensy 하나로 묶는 것

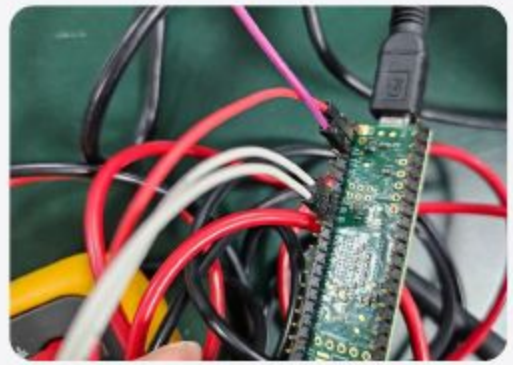


USB 구동 확인

휠 · ODrive

LattePanda → odrivetool(USB)
속도제어 · 15 극쌍 · 3200 CPR

다음 CAN2 전환 , 2 축 동시



I²C 출력 확인

IMU · BNO085

Teensy I²C(18/19) → USB
쿼터니언 · Roll / Pitch / Yaw

다음 400 Hz, 피치축 정의

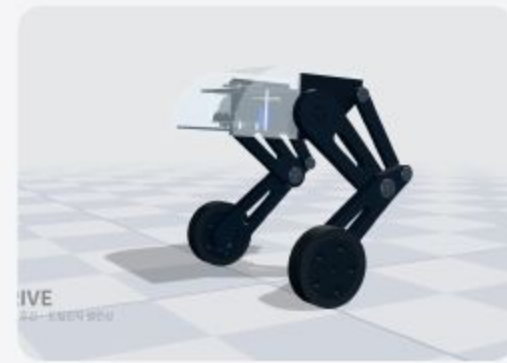


미시험 · 입고 후

고관절 · DM4340P

40:1 · 정격 9 / 피크 27 N·m
CAN 1 Mbps · MIT 모드

다음 ID 설정 → CAN1 구동



설계 단계

통합 · ROS 2

Teensy 500 Hz 루프
+ LattePanda ROS 2 브리지

다음 서포터 위 밸런싱

핵심 Teensy 4.1 이 CAN1(고관절) · CAN2(휠) · I²C(IMU) 를 500 Hz 로 직접 돌리고 , LattePanda 는 ROS 2 로 명령 · 기록만

전장 구성

02 BOM 전장품 — 역할 · 연결 · 설정값

부품	수량	역할	연결	전원	설정값 (코드와 일치)
Damiao DM-J4340P-2EC	2	고관절 (높이 · 롤)	CAN1 1 Mbps · XT30(2+2)	24 V (15–32)	ID 1/2 · 피드백 0x11/0x12 · MIT
누리로봇 WA172E 인휠	2	구동 휠	ODrive 3 상 · 홀 · 엔코더 A/B	24 V 정격	15 극쌍 · 3200 CPR · Kt 1 N·m/A
ODrive v3.6 (보유)	1	휠 FOC 드라이버 2 축	CAN2 1 Mbps · node 0 / 1	6S 직결	토크모드 · 엔코더 2 ms 송신
Teensy 4.1	1	하위 실시간 제어	USB · CAN1 · CAN2 · I²C	USB 5 V	500 Hz · FlexCAN_T4
SN65HVD230 트랜시버	3	3.3 V CAN 물리층	Teensy ↔ CANH / CANL	3.3 V	CAN1 · CAN2 + 예비 1
BNO085 IMU	1	피치 · 롤 · 요레이트	I²C (SDA 18 / SCL 19)	3.3 V	주소 0x4A · 400 Hz
LattePanda (보유)	1	상위 제어 · ROS 2	USB → Teensy · ODrive	5 V	Ubuntu 24.04 · Jazzy
VEGA 6S 5200 mAh	1	주전원 22.2 V	XT90 → WAGO 221-415	19.8–25.2 V	저전압 컷 19.0 V
24 V → 5 V 강압기	1	LattePanda 전원	WAGO 분배	5 V	—

팀 BOM 총계 1,184,795 원 (3D 프린팅 별도) · ODrive · LattePanda 는 기존 보유품 · 루빅 링크 V3 는 서보 변경으로 제외

시스템 구조

03 상위 LattePanda · 하위 Teensy · 구동부 — 3 계층



1 회 설정 도구 Windows PC — Damiao 디버깅 툴 (DM ID · 모드 · 영점) LattePanda — odrivetool, USB (ODrive 모터 · 엔코더 · CAN)

전원 · 배선

04 6S 한 줄에서 분배 · CAN 은 양 끝에 120 Ω

VEGA 6S LiPo 22.2 V · 5200 mAh

↓

XT90 커넥터

↓

퓨즈 30 A + 메인 / 비상 스위치 ← 추가 필요

↓

WAGO 221-415 분배

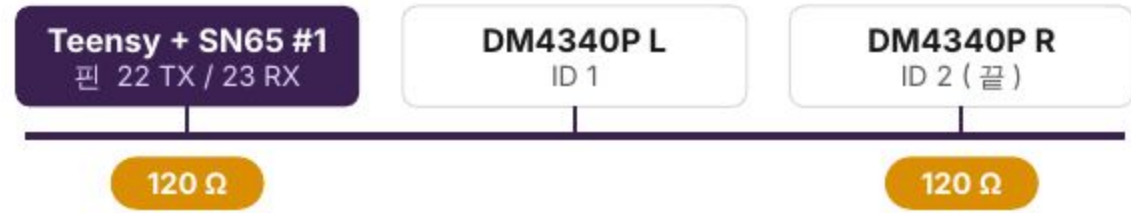
DM4340P × 2
XT30(2+2) 체인
24 V 형 15–32 V

ODrive DC
WA172E × 2
10 A / 축 제한

24→5 V 강압기
LattePanda 5 V
→ USB → Teensy

Teensy 3.3 V → BNO085 · SN65HVD230 × 2 핀은 5 V 비허용

CAN1 — 고관절



CAN2 — 휠



배선 체크

- 전원 OFF: CANH–CANL ≈ 60 Ω
- 트위스트 페어 + GND 함께 배선
- CAN H/L 반전 주의 (케이블 표기)
- 추가 핀 22·23 → CAN1 · 1·0 → CAN2



CAN 버스 설계

05 버스 두 개로 분리 — ID 충돌 방지 · 부하 약 25 %

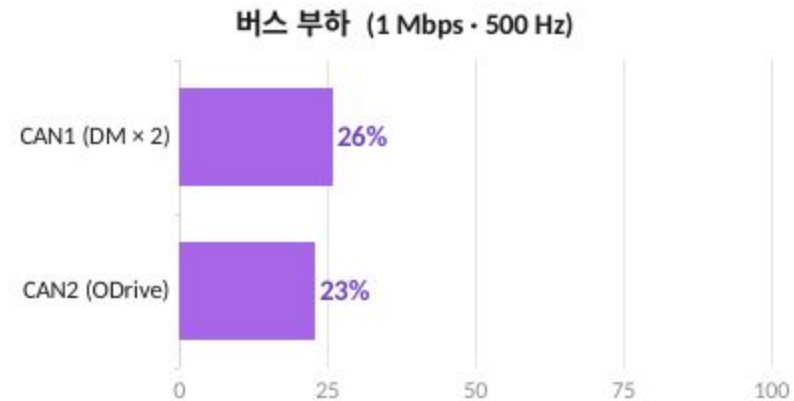
버스	ID	방향	내용	주기
CAN1	`0x001 / 0x002`	Teensy → DM	MIT 명령 (L / R)	500 Hz
CAN1	`0x011 / 0x012`	DM → Teensy	피드백 위치 · 속도 · 토크 · 온도	500 Hz
CAN1	`0x7FF`	Teensy → DM	레지스터 읽기 · 쓰기 · 저장	설정 시
CAN2	`0x00E / 0x02E`	Teensy → ODrive	Set Input Torque (L / R)	500 Hz
CAN2	`0x009 / 0x029`	ODrive → Teensy	Encoder 위치 · 속도	500 Hz
CAN2	`0x001 / 0x021`	ODrive → Teensy	Heartbeat 에러 · 상태	10 Hz
CAN2	`0x007/0B/18`	Teensy → ODrive	상태 · 제어모드 · 에러클리어	이벤트
CAN2	`0x017 / 0x037`	RTR 요청 → 응답	버스 전압 · 전류 (워치독)	20 Hz

ODrive ID = (node_id << 5) | cmd · DM 명령 ID = CAN_ID, 피드백 ID = Master ID

한 버스에 같이 두면 ?

- DM R 명령 `0x002` = ODrive node 0 **E-stop**
- DM L 명령 `0x001` = node 0 Heartbeat

→ CAN1 / CAN2 분리
한쪽 버스 고장이 다른 쪽을 막지 않음

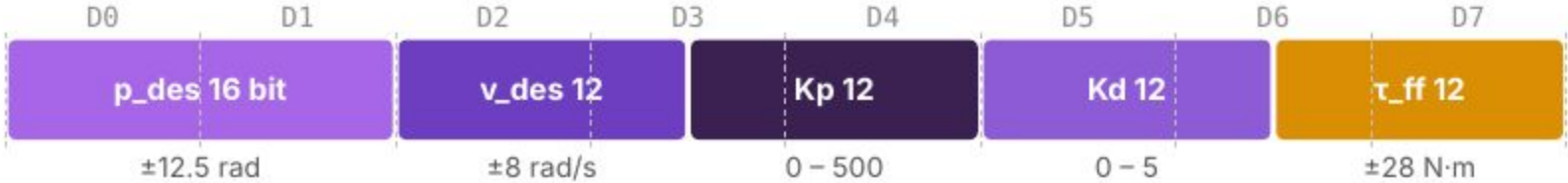


프레임 ≈ 130 bit × 500 Hz × (명령 2 + 응답 2)
= 260 kbit/s → 1 Mbps 의 26 %

DM4340P 프로토콜

06 MIT 모드 — 위치 · 속도 ·Kp·Kd· 토크를 8 바이트 한 프레임에

명령 프레임 ID = CAN_ID (1 / 2)



피드백 프레임 ID = Master ID (0x11 / 0x12)



데이터 (8 B)	동작
FF FF FF FF FF FF FF FC	Enable
FF FF FF FF FF FF FF FD	Disable
FF FF FF FF FF FF FF FE	현재 위치 = 영점
FF FF FF FF FF FF FF FB	에러 클리어 *

* 펌웨어 버전에 따라 지원

state 코드

0 Disable · 1 Enable
8 과전압 · 9 저전압
A 과전류 · B MOS 과열
C 코일 과열 · D 통신두절
E 과부하 → 8↑ FAULT

모터 안에서 계산되는 토크

$$\tau = K_p(p_{des} - p) + K_d(v_{des} - v) + \tau_{ff}$$

- Kp = 0, Kd > 0 → 댐핑만
- Kd = 0 위치제어 → 진동 , 금지
- τ_ff = 높이별 중력보상

정수 변환

$$u = (x - x_{min})(2^n - 1) / (x_{max} - x_{min})$$

mit(1.0, −2.0, 50, 1.5, 5.0) →
8A 3D 60 01 9A 4C D9 6D

PC 단위테스트로 바이트 검증 완료

목차

DM4340P 초기 설정

07 모터마다 ID · 모드 · 영점을 한 번 저장

1 준비
USB-TTL 3.3 V 또는 Damiao USB2CAN
+ 동봉 GH1.25 3 핀 디버그 케이블

2 연결
모터 24 V (전원공급기 전류제한 3 A)
디버그 포트 ↔ USB-TTL, TX↔RX 교차

3 읽기
Damiao 디버깅 툴 → 포트 (921600)
→ Open Port → Read Param

4 쓰기 · 저장
오른쪽 표대로 입력 → Write → 저장
전원 재투입 후 다시 읽어 확인

5 영점
다리를 기준 자세 (Δφ = 0) 에 맞추고
Set Zero (dm_single_test 의 z 키)

레지스터	번호	왼쪽	오른쪽
CAN_ID (ESC_ID)	8	0x01	0x02
Master ID	7	0x11	0x12
Control Mode	10	1 = MIT	1 = MIT
PMAX/VMAX/TMAX	21–23	12.5/8/28	12.5/8/28
CAN Timeout	9	2000	2000

Timeout 2000 × 50 µs = 100 ms 동안 명령이 없으면 모터가 스스로 정지

모터는 한 개씩 연결해서 설정 — 출하 ID 가 같으면 충돌

```
// 툴이 없을 때 : Teensy 에서 레지스터 쓰기 (ID 0x7FF)
tx(m.disable());
tx(m.writeRegU32(dm::reg::MST_ID, 0x12));  // fb ID
tx(m.writeRegU32(dm::reg::ESC_ID, 0x02));  // CAN ID
tx(m.saveParams());                        // 재부팅
```

ODrive CAN 설정

08 odrivetool 로 1 회 설정 → 이후는 CAN 으로만 명령

host/odrive_can_setup.odrivetool.py — 3 주차 axis0 설정을 2 축 + CAN 으로 확장

```
odrv0.can.config.baud_rate = 1000000
odrv0.config.dc_bus_undervoltage_trip_level = 19.0
odrv0.config.brake_resistance = 2.0     # 동봉 저항
odrv0.config.enable_brake_resistor = True
for ax, node in ((odrv0.axis0, 0), (odrv0.axis1, 1)):
    ax.motor.config.pole_pairs = 15     # 30 극
    ax.motor.config.torque_constant = 1 # Kt 1 N·m/A
    ax.motor.config.calibration_current = 5
    ax.motor.config.current_lim = 10
    ax.encoder.config.mode = ENCODER_MODE_INCREMENTAL
    ax.encoder.config.cpr = 3200        # 800 PPR × 4
    ax.encoder.config.use_index = False
    ax.controller.config.control_mode = 1   # TORQUE
    ax.controller.config.input_mode = 1     # PASSTHROUGH
    ax.config.can.node_id = node        # 0 = L, 1 = R
    ax.config.can.encoder_rate_ms = 2   # 500 Hz
    ax.config.can.heartbeat_rate_ms = 100
    ax.config.watchdog_timeout = 0.2
odrv0.save_configuration(); odrv0.reboot()
```

cmd	메시지	페이로드
0x01	**Heartbeat**	axis_error u32 · state u8
0x07	**Set Axis State**	7 캘리브 · 8 폐루프 · 1 IDLE
0x09	**Encoder Estimates**	pos · vel f32 [turn]
0x0B	**Set Controller Mode**	1 토크 · 1 passthrough
0x0E	**Set Input Torque**	f32 [N·m]
0x17	**Bus Voltage/Current**	RTR → f32 V · f32 A
0x18	**Clear Errors**	—

부팅마다 엔코더 오프셋 캘리브레이션

- WA172E 엔코더는 인덱스 없는 증분식 → state 7
- 서포터 위에서 Teensy 가 CAN 으로 자동 요청 (약 3 s)
- 모터 캘리브는 1 회 후 pre_calibrated = True

코드 작성 위치

09 어디서 · 무엇으로 · 어떤 코드를 쓰나

장소	도구 · 언어	작성 / 수정하는 것	언제
Windows PC	Damiao 디버깅 툴	ID · 모드 · 영점 (설정값)	모터별 1회
LattePanda → ODrive USB	odrivetool (Python)	host/odrive_can_setup	1회
개발 노트북 Ubuntu 24.04	Arduino IDE 2 + Teensyduino (C++)	firmware/twlr_teensy	수정 시 업로드
LattePanda Ubuntu 24.04	ROS 2 Jazzy rclpy · colcon	ros2_ws/src/ twlr_bringup	로봇 실행 시
개발 노트북	g++ · pytest	tests/	커밋 전

저장소에 추가할 구조 (★ 신규 · twlr_control.zip)

```
capstone-design_TWLR_KAU/
├ firmware/
│  ├ teensy_bno085_quat/ (기존)
│  ├ twlr_teensy/          ★ 통합
│  │  ├ config.h           설정값
│  │  ├ dm_motor.*         DM4340P
│  │  ├ odrive_can.*       ODrive
│  │  ├ imu.h              BNO085
│  │  ├ control.h          LQR
│  │  ├ bridge.*           패킷
│  │  └ twlr_teensy.ino    루프
│  └ bringup/              ★ 단품
├ host/
│  ├ odrive_can_setup      ★
│  └ dm4340p_setup.md      ★
├ ros2_ws/src/
│  └ twlr_bringup/         ★ ROS 2
└ tests/                   ★ PC 검사
```

Teensy 펌웨어

10 2 ms 마다 : 받기 → 추정 → 제어 → 보내기

CAN 수신	IMU	호스트 명령	상태추정	제어	CAN 송신
FIFO 인터럽트 RX 256 큐	GRV + Gyro 400 Hz	USB 패킷 CRC16	θ, ω (IMU) x, v (휠)	LQR → 휠 고관절 MIT	DM × 2 ODrive × 2

주기	작업	함수
500 Hz	제어 스텝 · CAN 송신 (DM 2 + ODrive 2)	`controlStep()`
400 Hz	IMU 리포트 수신	`imu.poll()`
100 Hz	Telemetry 92 B → LattePanda	`sendTelemetry()`
50 Hz	Command 16 B ← LattePanda	`readHost()`
20 Hz	ODrive 버스전압 요청 (워치독)	`requestBusVI()`
10 Hz	ODrive Heartbeat → 에러 감시	`checkHealth()`

루프 시간 확인

- `loop_us_max` 1 초마다 전송
- 목표 < 2,200 µs
- 넘으면 IMU · 텔레메트리 주기 ↓
- 병목은 계산이 아니라 I²C 읽기

Teensy 펌웨어

11 안전 상태머신 — 순서대로만 켜고, 이상하면 바로 멈춘다



전이 조건

1 ODrive 하트비트 + IMU 수신 확인

2 /twlr/enable = true

3 캘리브 완료 → 휠 페루프 + DM Enable

4 /twlr/balance = true & |θ − θeq| < 10°

← balance = false → READY (서포터 위에서만)

ROS 명령 250 ms 끊김 → v = 0 (균형 유지)
코일 80 °C ↑ → 510 mm 로 올려 토크 ↓

FAULT

들어가는 조건

- DM 피드백 30 ms 끊김 · DM state ≥ 8
- ODrive 하트비트 300 ms 끊김 · axis_error
- |θ − θeq| > 35° (넘어짐)
- 엔코더 캘리브 15 s 초과

동작

- 휠 IDLE · 고관절 1 s 댐핑 (Kp 0, Kd 1) → Disable
- enable = false → 1 s 후 IDLE 로 리셋

코드 ① 고관절 DM4340P

12 MIT 프레임 만들기 · 피드백 해석 (dm_motor.cpp)

명령 만들기

```
CanFrame Motor::mit(float p, float v,
                    float kp, float kd, float t) const {
  uint32_t P  = floatToUint(p, -pmax_, pmax_, 16);
  uint32_t V  = floatToUint(v, -vmax_, vmax_, 12);
  uint32_t KP = floatToUint(kp, 0.f, 500.f, 12);
  uint32_t KD = floatToUint(kd, 0.f, 5.f, 12);
  uint32_t T  = floatToUint(t, -tmax_, tmax_, 12);
  CanFrame f;  f.id = can_id_;  f.len = 8;
  f.data[0] = P >> 8;   f.data[1] = P & 0xFF;
  f.data[2] = V >> 4;
  f.data[3] = ((V & 0xF) << 4) | (KP >> 8);
  f.data[4] = KP & 0xFF;
  f.data[5] = KD >> 4;
  f.data[6] = ((KD & 0xF) << 4) | (T >> 8);
  f.data[7] = T & 0xFF;
  return f;
}
```

피드백 해석

```
bool Motor::parse(const CanFrame& f, uint32_t now) {
  if (f.id != mst_id_) return false;
  if ((f.data[0] & 0x0F) != (can_id_ & 0x0F))
    return false;
  fb_.state = f.data[0] >> 4;   // 1 Enable, 8↑ 에러
  uint32_t p = (f.data[1] << 8) | f.data[2];
  uint32_t v = (f.data[3] << 4) | (f.data[4] >> 4);
  uint32_t t = ((f.data[4] & 0xF) << 8) | f.data[5];
  fb_.pos = uintToFloat(p, -pmax_, pmax_, 16);
  fb_.vel = uintToFloat(v, -vmax_, vmax_, 12);
  fb_.tau = uintToFloat(t, -tmax_, tmax_, 12);
  fb_.t_mos = f.data[6];
  fb_.t_rotor = f.data[7];
  fb_.stamp_ms = now;           // 30 ms 끊기면 FAULT
  return true;
}
```

```
// 500 Hz 루프에서 사용 — 좌우 모터가 마주 보므로 방향 부호 DM_DIR = +1 / -1
tx1(hipL.mit(DM_DIR_L * phi_des, 0.f, HIP_KP, kd, DM_DIR_L * tau_ff));  // Kp 40 · kd 공중 1.5 / 서기 3.0
tx1(hipR.mit(DM_DIR_R * phi_des, 0.f, HIP_KP, kd, DM_DIR_R * tau_ff));
```

코드 ② 휠 ODrive

13 CAN Simple 메시지 + 부팅 시퀀스

odrive_can.cpp

```
uint32_t arbId(uint8_t node, uint8_t cmd) {
  return (node << 5) | (cmd & 0x1F);
}
CanFrame Axis::setInputTorque(float nm) const {
  CanFrame f = make(SET_INPUT_TORQUE, 4); // 0x0E
  putF32(&f.data[0], nm);   // little-endian
  return f;
}
bool Axis::parse(const CanFrame& f, uint32_t now) {
  if ((f.id >> 5) != node_) return false;
  switch (f.id & 0x1F) {
  case HEARTBEAT:
    st_.axis_error = getU32(&f.data[0]);
    st_.state = f.data[4];  break;
  case GET_ENCODER_ESTIMATES:
    st_.pos = getF32(&f.data[0]);   // turn
    st_.vel = getF32(&f.data[4]);   // turn/s
  } ...
```

twlr_teensy.ino

```
// IDLE → CALIB  (/twlr/enable 수신)
tx2(wheelL.clearErrors());
tx2(wheelL.setState(ENCODER_OFFSET_CALIBRATION));
// CALIB → READY  (state 가 다시 IDLE 이면)
tx2(wheelL.setControllerMode(TORQUE_CONTROL,
                             PASSTHROUGH));
tx2(wheelL.setInputTorque(0));
tx2(wheelL.setState(CLOSED_LOOP_CONTROL));
tx1(hipL.enable());            // DM: FF..FC
// BALANCE (500 Hz)
tx2(wheelL.setInputTorque(WHEEL_DIR_L * tau_L));
tx2(wheelR.setInputTorque(WHEEL_DIR_R * tau_R));
//  휠 각속도 = dir · vel[turn/s] · 2π
```

clearErrors → **state 7** 오프셋 캘리브 → **IDLE 복귀** 확인 → **토크 모드** passthrough → **state 8** 폐루프 → **토크 명령** 500 Hz

3 주차에 odrivetool 에서 손으로 치던 requested_state = 3 / 8, control_mode = 2 를 그대로 CAN 프레임으로 옮긴 것

코드 ③ 밸런싱 · 고관절

14 LQR 휠 토크 + 높이별 중력보상 (control.h)

휠 : 전진 토크 u 를 좌우로 분배 (|τ| ≤ 8 N·m)

$$u = K_x(x - x_{ref}) + K_v(v - v_{cmd}) + K_\theta(\theta - \theta_{eq}) + K_\omega\,\omega$$

$$\tau_{L,R} = u/2 \mp K_\psi(r_{cmd} - r)/2$$

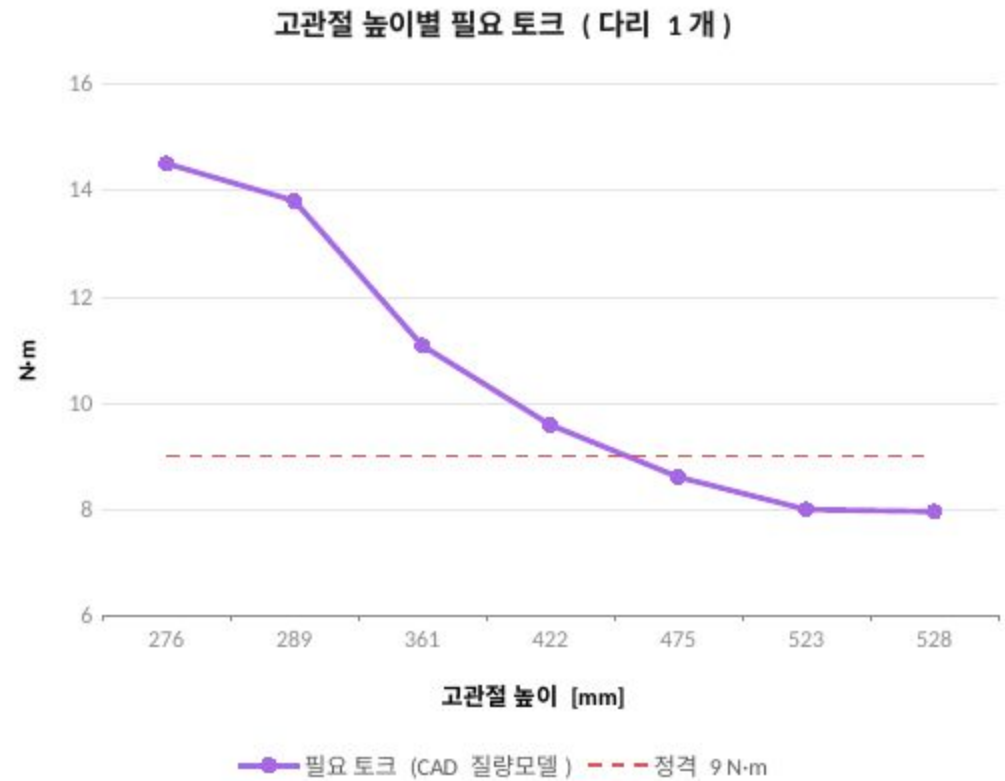
ω 피치 각속도 · r 요레이트 · x, v 휠 이동거리 · 속도

게인	값	단위
K_x	2.24	N·m / m
K_v	4.10	N·m·s / m
K_θ	28.4	N·m / rad
K_ω	4.59	N·m·s / rad
K_ψ	0.50	N·m·s / rad

초기값 계산 조건

총질량 11.24 kg (CAD)
몸체 6.84 kg, 휠축 위 0.305 m
휠 반경 0.086 m
Q = diag(5, 2, 200, 5), R = 1
극점 − 20.8, −6.4, −0.74 ± 0.6j
→ Simulink 모델 lqr() 로 재계산

```
float phi_des = hipTraj.step(heightToPhi(h_cmd),
                             0.35f /*rad/s*/, dt);
float tau_ff  = 0.8f * lut(PHI_DEG, TAU_NM, phi);
```



기본 455 mm — 정격 연속 영역 (≥ 452 mm). 낮게 앉을수록 토크 ↑

Teensy ⇄ ROS 2

15 USB 시리얼 패킷 — CRC16 으로 깨진 프레임은 버린다



CRC16-CCITT
poly 0x1021

type 0x01 Telemetry · 0x02 Command · 0x03 로그 문자열

Command 16 B · 50 Hz	형식
seq	u16
enable · balance	u8 u8
v_cmd ← /cmd_vel.linear.x	f32
yaw_rate ← /cmd_vel.angular.z	f32
height_mm ← /twlr/height	f32

Telemetry 92 B · 100 Hz	→ 토픽
t_ms · mode · faults	/twlr/mode
quat[4] · gyro[3]	/imu/data
pitch · pitch_rate	밸런싱 확인
hip pos · vel · τ [2]	/joint_states
wheel pos · vel [2]	/odom
vbus · 온도 · loop_us	/twlr/battery

지금 : 시리얼 브리지

- 3 주차 pyserial 코드 확장
- 추가 빌드 도구 없음
- 바이트 단위 테스트

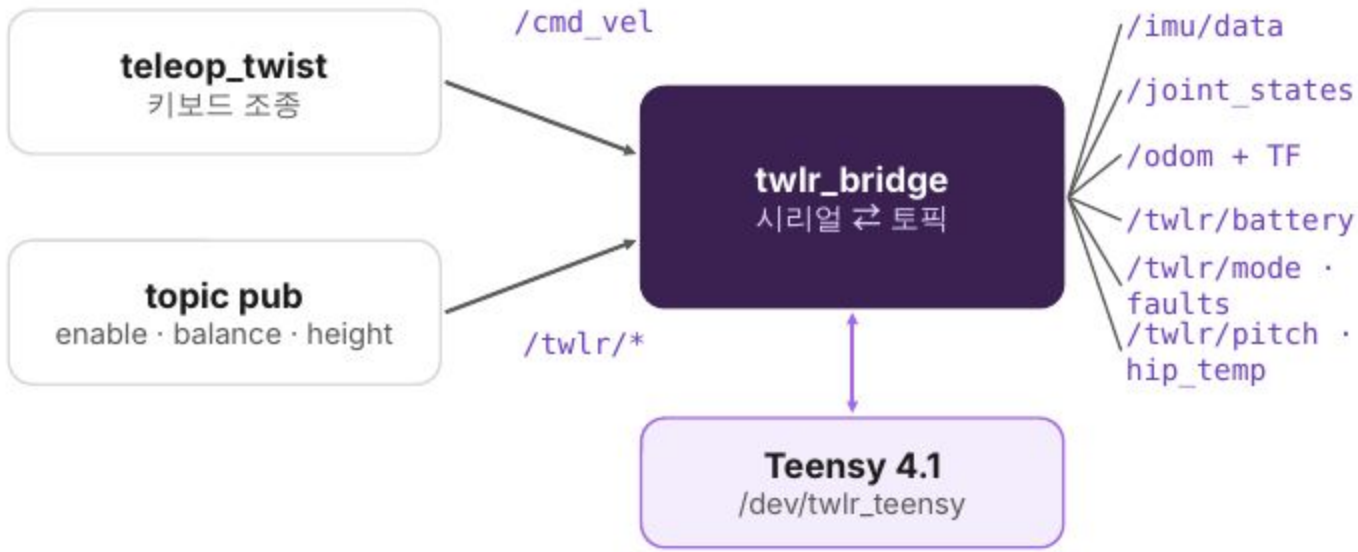
나중 : micro-ROS (선택)

- teensy41 + jazzy 지원
- 에이전트 · 빌드 추가 필요

C++ (bridge.cpp) 와 Python (protocol.py) 이 같은 바이트를 만드는지 교차 테스트 통과

ROS 2 (LattePanda)

16 노드 · 토픽 구성과 실행 순서



토픽을 쓰는 쪽

rviz2	rosbag2	slam_toolbox
시각화	기록 → Simulink 비교	향후 SLAM

/odom 은 휠 엔코더 + 피치 보정으로 계산 → 나중에 라이다만 붙이면 SLAM 입력으로 사용

터미널 명령

```
# 설치 (1회)
sudo apt install ros-jazzy-teleop-\
  twist-keyboard python3-serial
cd ~/twlr_ws
colcon build --symlink-install
source install/setup.bash

# 실행 (서포터 위)
ros2 launch twlr_bringup \
  twlr_bringup.launch.py
ros2 topic pub --once /twlr/enable \
  std_msgs/msg/Bool "{data: true}"
ros2 topic pub --once /twlr/balance \
  std_msgs/msg/Bool "{data: true}"
ros2 run teleop_twist_keyboard \
  teleop_twist_keyboard
ros2 bag record /imu/data \
  /joint_states /odom
```

코드값 정리

17 config.h — 실물에서 바꿀 값은 이 파일 한 곳

항목	값	상태
CTRL_PERIOD_US	2000 (500 Hz)	확정
CAN1/CAN2_BAUD	1 000 000	확정
DM CAN_ID L / R	0x01 / 0x02	확정
DM MST_ID L / R	0x11 / 0x12	확정
DM P/V/T MAX	12.5 / 8 / 28	툴 일치
DM_FB_TIMEOUT_MS	30	확정
OD_NODE_L / R	0 / 1	확정
OD_HB_TIMEOUT_MS	300	확정
WHEEL_RADIUS_M	0.086	확정
WHEEL_TORQUE_MAX	8.0 N·m	확정
IMU_ADDR · 주기	0x4A · 400 Hz	확정

항목	값	상태
HIP_MIN / MAX	−20.6° / +19.9°	확정
HIP_KP · KD (공중 / 서기)	40 · 1.5 / 3.0	튜닝
HIP_FF_SCALE	0.8	튜닝
HIP_RATE_MAX	0.35 rad/s	튜닝
HEIGHT_DEFAULT	455 mm	확정
K_X · K_V	2.24 · 4.10	튜닝
K_TH · K_W · K_YAW	28.4 · 4.59 · 0.5	튜닝
THETA_EQ_RAD	실측 평균	측정
DM_DIR · WHEEL_DIR	+1 / −1	측정
IMU 피치 축 · 부호	pitch · +1	측정
FALL · BALANCE_ENTER	35° · 10°	확정

측정 = 실물에서 한 번 재서 넣는 값 · 튜닝 = 실험하며 조정 · 근거 : 3 주차 운용범위 · 05 DM4340P 토크표 · WA172E 데이터시트

Bring-up 계획

18 8 단계 — 합격 기준을 넘어야 다음 단계로

1 전원 · 배선

퓨즈 · 스위치 , 트위스트 배선

합격
CANH–CANL ≈ 60 Ω

멀티미터

2 DM ID 설정

1 개씩 ID · 모드 · 영점

합격
ID 1 / 2 응답

Damiao 툴

3 DM CAN 구동

dm_single_test e → h → s

합격
사인 추종 오차 < 2°

Teensy CAN1

4 ODrive CAN 전환

odrive_can_test c → l → v

합격
두 휠 0.5 turn/s

Teensy CAN2

5 방향 · 부호 측정

DM_DIR · WHEEL_DIR · IMU

합격
앞으로 기울면 pitch +

config.h

6 통합 + ROS 2

twlr_teensy + twlr_bridge

합격
loop < 2.2 ms

LattePanda

7 서포터 위 밸런싱

θ_eq 측정 → BALANCE

합격
손 놓고 10 s 유지

rosbag 기록

8 주행 · 높이 · 선회

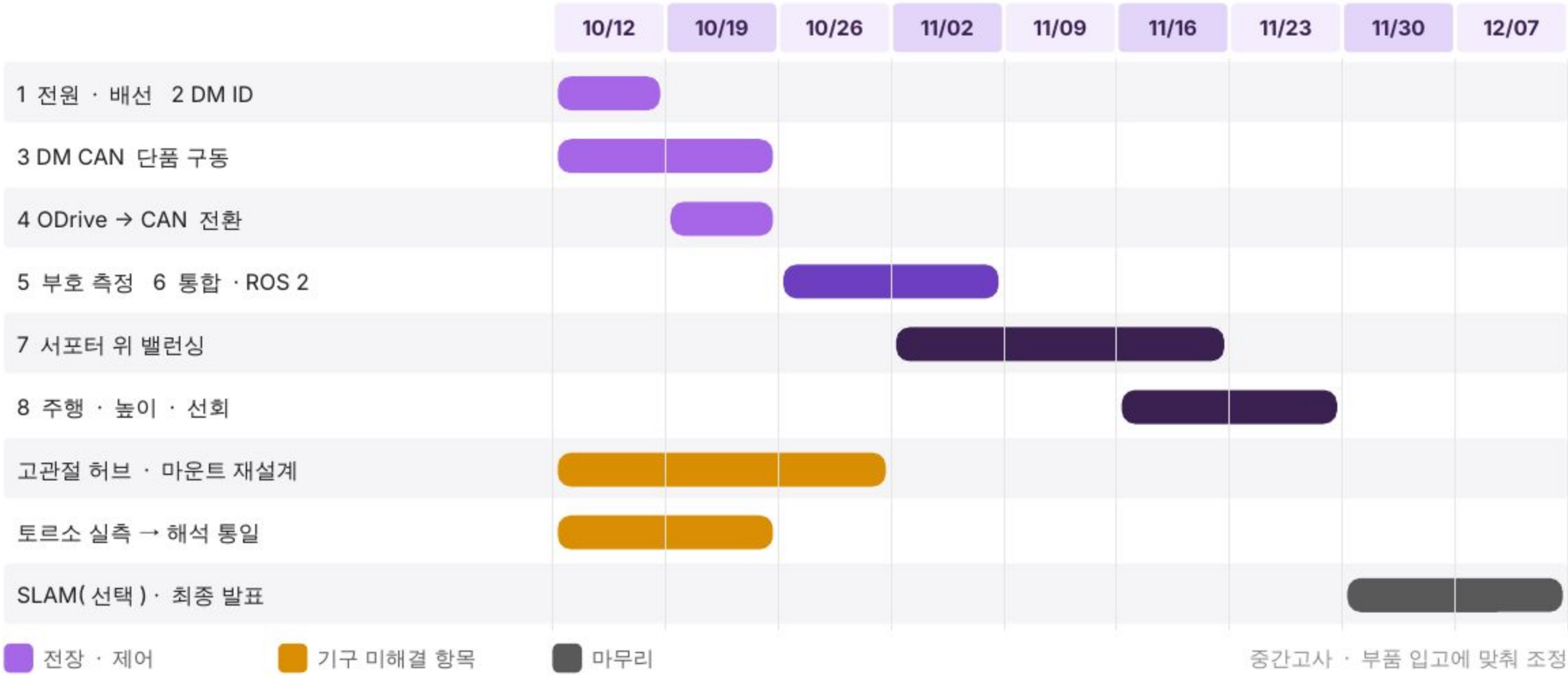
/cmd_vel · /twlr/height

합격
0.3 m/s · 300↔500 mm

쇼케이스 재현

일정

19 10/12 ~ 12/13 주차별 계획 (안)



안전 · 추가 구매

20 미리 막아야 할 6 가지 + BOM 에 없는 부품

위험	대응
Teensy 핀 5 V 비허용	3.3 V 트랜시버 SN65HVD230 사용 (구매품 OK)
CAN ID 충돌	버스 2 개 분리 · DM 은 한 개씩 ID 설정
고관절 연속 토크 초과 (직립 9.59 N·m = 정격 107 %)	기본 455 mm · 중력보상 · 80 °C ↑ → 510 mm
넘어짐	35° ↑ FAULT · 서포터 위 시험 · 휠 8 N·m 제한
회생 전압 (DM 24 V 형 보호 32 V)	ODrive 브레이크 저항 사용
통신 끊김	ROS 250 ms → v = 0 · DM 100 ms · ODrive 0.2 s

BOM 추가 구매

- ☐ 30 A 퓨즈 + 홀더 **필수**
- ☐ 메인 · 비상 스위치 **필수**
- ☐ 120 Ω 저항 (CAN1 끝) **필수**
- ☐ 트위스트 페어 AWG22~24 **필수**
- ☐ USB-TTL 3.3 V **DM 설정**
- ☐ XT30(2+2) 케이블 여분 **권장**
- ☐ Damiao USB2CAN **선택**
- ☐ ODrive 2 Ω 브레이크 저항 **보유 확인**

정리

21 이번 주부터 할 일

DM ID 설정 → CAN1 단품 → CAN2 전환 → 통합 펌웨어 → ROS 2 브리지 → 서포터 밸런싱 → 주행 · 높이

이번 주

- 퓨즈 · 스위치 · 120 Ω · 케이블 · USB-TTL 주문
- DM4340P 입고 → 1개씩 ID 설정
- Teensy 22·23 / 1·0 핀 트랜시버 배선

다음 주

- dm_single_test 로 고관절 사인 추종
- odrive_can_setup → odrive_can_test
- 토르소 실측 질량 측정

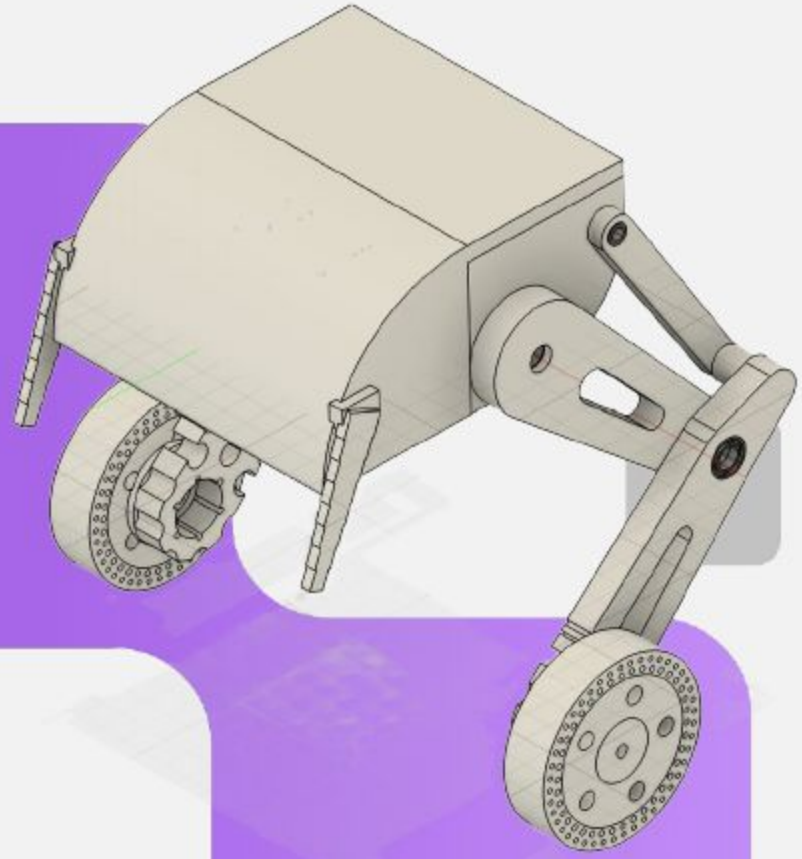
함께 드리는 코드

- twlr_control.zip — 펌웨어 · 단품 시험 · ROS 2
- PC 검사 통과 (CAN · 패킷 · 제어)
- Teensy 빌드는 Teensyduino 에서 확인

목표 쇼케이스의 **SQUAT · ROLL · DRIVE · TURN** 을 실물 **TWLR** 로 재현

PART 07

CODE, PRINCIPLES & THEORY



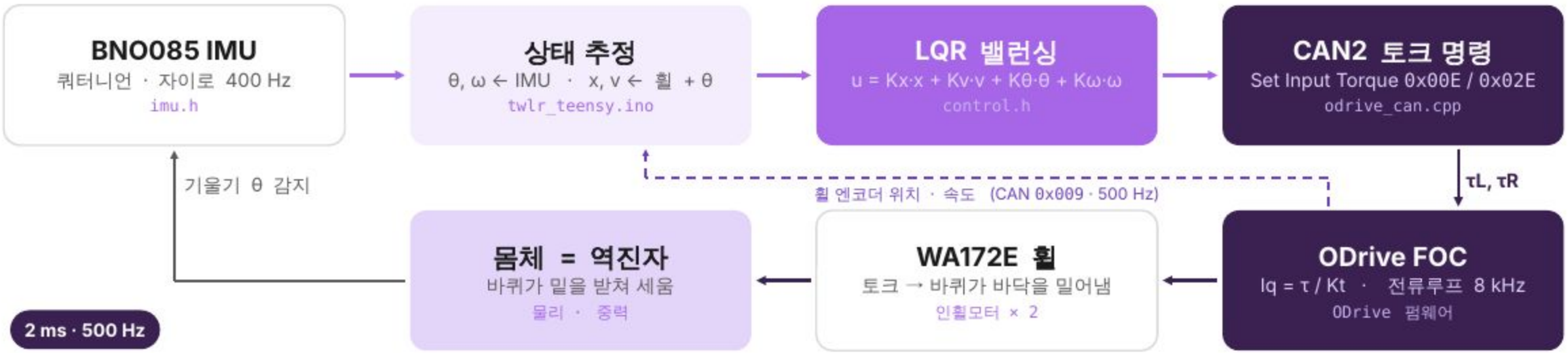
코드 해설 · 원리 · 이론

CAN · MIT 모드 · FOC · 역진자 · LQR · CRC · ROS 2 — 각 코드가 무엇을 뜻하고 어떤 원리로 동작하는지

전체 신호 흐름 · 파일 지도	78–79p
CAN 통신 원리 · FlexCAN 코드	80–82p
고관절 — 비트 패킹 · MIT · 중력보상	83–85p
휠 — FOC · ODrive 메시지 · 설정 스크립트	86–88p
자세 추정 · 역진자 · LQR · 밸런싱 코드	89–92p
메인 루프 · 안전 · 시리얼 · ROS 2	93–97p
단품 시험 스케치	98p

제어 루프 한눈에

01 센서 → 추정 → 제어 → 구동 → 몸체 — 2 ms 마다 한 바퀴



고관절 루프 — 목표와 게인만 보내면 모터 안에서 위치 루프가 닫힘 (MIT 모드)

/twlr/height	heightToPhi	RateLimiter	MIT 프레임	DM4340P	다리 높이
ROS 명령 [mm]	LUT 보간 → Δφ	0.35 rad/s 제한	Kp · Kd · τff	내부 PD + 감속 40:1	290 – 520 mm

핵심 휠은 Teensy 가 직접 루프를 닫고 (LQR, 500 Hz) · 고관절은 목표와 게인만 주면 DM4340P 내부에서 위치 루프가 닫힌다

코드 파일 지도

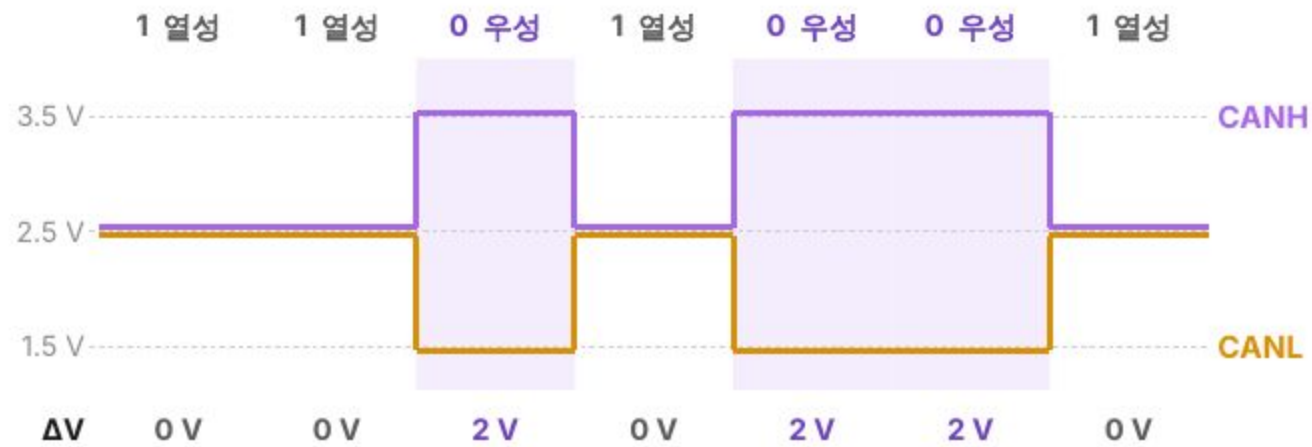
02 파일마다 하는 일이 하나 — 어디서 돌고, 무엇을 부르고, 어느 슬라이드에서 설명하나

파일	실행 위치	하는 일	핵심 이름	해설 슬라이드
config.h	Teensy	모든 숫자 한 곳 (ID · 주기 · 게인 · 한계)	cfg::HIP_KP …	6 장 코드값 정리
can_frame.h	Teensy · PC	하드웨어와 무관한 CAN 프레임 · 리틀엔디안 변환	CanFrame, putF32	ODrive CAN 메시지
dm_motor.h / .cpp	Teensy	DM4340P 명령 만들기 · 피드백 해석 · 레지스터	mit(), parse()	고정소수점 · MIT 모드
odrive_can.h / .cpp	Teensy	ODrive CAN Simple 메시지 만들기 · 해석	setInputTorque()	FOC · ODrive 메시지
imu.h	Teensy	BNO085 → 피치 · 피치 각속도 · 요레이트	poll(), pitch()	IMU 자세 추정
control.h	Teensy	LQR 밸런서 · 높이 → 각도 LUT · 변화율 제한	Balancer::step()	역진자 · LQR · 중력보상
bridge.h / .cpp	Teensy	USB 패킷 구조체 · CRC16 · 바이트 파서	encode(), Parser	시리얼 패킷 · CRC
twlr_teensy.ino	Teensy	setup · loop · 500 Hz 제어 · 상태머신 · 안전	controlStep()	메인 루프 · 상태머신
bringup/dm_single_test	Teensy	DM4340P 1 개 키보드 시험	키 e d z h s 0 p	단품 시험 스케치
bringup/odrive_can_test	Teensy	ODrive 2 축 CAN 시험	키 c l v s t i x	단품 시험 스케치
host/odrive_can_setup	LattePanda	ODrive 1 회 설정 (odrivetool 붙여넣기)	node_id, cpr	ODrive 설정 스크립트
twlr_bringup/protocol.py	LattePanda	시리얼 패킷의 Python 판 (C++ 과 바이트 일치)	encode_command()	시리얼 패킷 · CRC
twlr_bringup/bridge_node.py	LattePanda	ROS 2 토픽 ⇄ 시리얼 · 오도메트리	send_command()	ROS 2 브리지 코드
launch · config · udev 규칙	LattePanda	실행 · 파라미터 · /dev/twlr_teensy 이름 고정	bridge.yaml	8 장 운용 방법
tests/	개발 PC	g++ · pytest 로 프레임 바이트 검증	build_check.sh	—

Teensy 파일은 Arduino IDE 에서 twlr_teensy.ino 하나만 열면 같은 폴더의 .h / .cpp 가 함께 컴파일됨

CAN 물리층

03 두 선의 전압 차이로 0 / 1 — 잡음에 강한 이유



왜 두 선인가

꼬아 놓은 두 선에는 모터 잡음 n 이 똑같이 실린다 → 수신기는 차이만 본다

$$(V_H + n) - (V_L + n) = V_H - V_L$$

우성 (0) 이 이긴다

아무도 안 당기면 1(열성), 한 노드라도 당기면 버스 전체가 0

→ "Wired-AND" — 다음 장 중재 (누가 먼저 말할지) 의 원리

트랜시버 SN65HVD230

- Teensy 22 CTX1 → D · 23 CRX1 ← R
- 3.3 V 논리 ↔ CANH / CANL 차동 변환
- Rs 핀 = GND → 고속 모드 (모듈 기본)

종단저항 120 Ω

- 선의 특성 임피던스 (≈120 Ω) 와 같은 저항으로 끝을 막으면 신호가 되돌아오지 (반사) 않음
- **양 끝 2 개 병렬 → 전원 OFF 에서 60 Ω**

```
can1.setBaudRate(1000000);  // 1 bit = 1 µs
```

1 Mbps = 비트 하나 1 µs

1 Mbps 권장 버스 길이 ≤ 40 m

→ 로봇 안 배선은 1 m 이내라 여유

CAN 프레임 · 중재

04 ID 가 작을수록 우선 — 동시에 보내도 데이터가 깨지지 않는다



중재 — 두 모터가 동시에 피드백을 보내면 ?

	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
DM L 피드백 0x011	0	0	0	0	0	0	1	0	0	0	1
DM R 피드백 0x012	0	0	0	0	0	0	1	0	0	1	·
버스 (= 우성 우선)	0	0	0	0	0	0	1	0	0	0	1

- b1 에서 R 은 1(열성) 을 냈는데 버스는 0 → R 은 즉시 송신을 멈추고 수신으로 전환
- L 프레임이 끝나면 R 이 바로 재전송 → 데이터 손실 없음
- **그래서 ID 가 작을수록 우선순위가 높다**

수신 확인 (ACK)

- 받은 노드가 ACK 칸을 0 으로 당겨 " 받았다 " 를 알림
- 받을 노드가 하나도 없으면 ACK 오류 → 같은 프레임을 계속 재전송
- **→ 모터 전원이 꺼져 있으면 송신이 막히고 피드백이 끊겨 30 ms 뒤 FAULT**

비트 스터핑 같은 값이 5 개 연속이면 반대 비트 1 개 삽입 → 수신기 클럭 동기 유지

부하 계산 8 B 프레임 = 111 bit + 스터핑 (최대 24) ≈ 130 bit = 130 µs → CAN1: 4 프레임 × 500 Hz × 130 µs = **26 %**

FlexCAN_T4 코드 해설

05 인터럽트가 받아 큐에 쌓고 , loop 가 꺼내서 해석한다

twlr_teensy.ino (CAN1 부분 · CAN2 도 같은 구조)

```
FlexCAN_T4<CAN1, RX_SIZE_256, TX_SIZE_16> can1; // [1]

void canBegin() {
  can1.begin();
  can1.setBaudRate(1000000);      // [2]
  can1.setMaxMB(16);              // [3]
  can1.enableFIFO();              // [4]
  can1.enableFIFOInterrupt();     // [5]
  can1.onReceive(onCan1);         // [6]
}
void onCan1(const CAN_message_t& m) {   // [7]
  const CanFrame f = fromMsg(m);
  if (!hipL.parse(f, millis())) hipR.parse(f, millis());
}
void loop() {
  while (can1.events() >> 12) {}  // [8]
  ...
  tx1(hipL.mit(phi, 0, KP, KD, tff));  // [9]
}
```

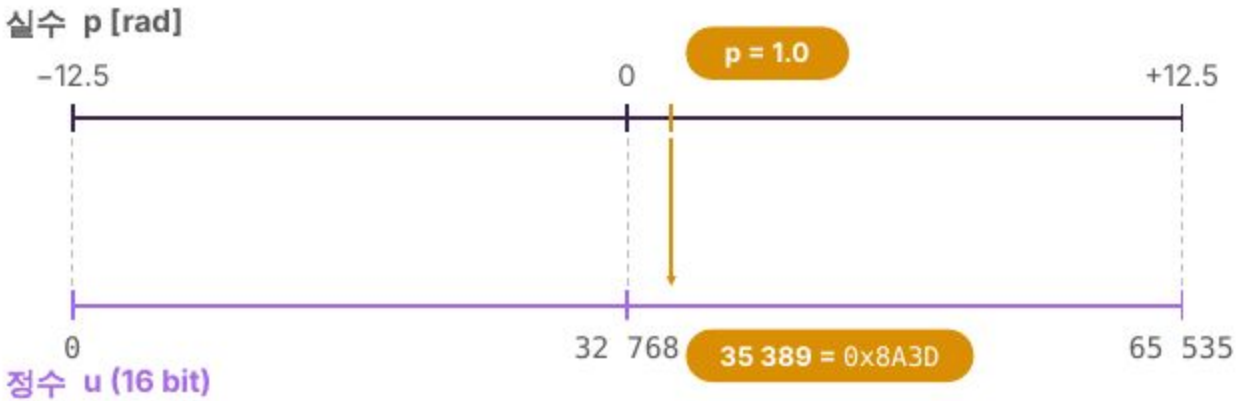


인터럽트 안에서는 복사만 , 해석은 loop 안에서 → 제어 계산이 끊기지 않음

1. `FlexCAN_T4<CAN1, …>`
   CAN1 컨트롤러 사용 · 수신 큐 256 · 송신 큐 16 프레임
2. `setBaudRate(1000000)`
   1 Mbps — 모터 · ODrive 설정과 반드시 같아야 함
3. `setMaxMB(16)`
   메일박스 (하드웨어 송수신 칸) 16 개 사용
4. `enableFIFO()`
   수신 FIFO — 하드웨어가 도착 순서대로 보관
5. `enableFIFOInterrupt()`
   도착 즉시 인터럽트 → 소프트웨어 큐로 복사
6. `onReceive(onCan1)`
   큐에서 꺼낼 때 부를 함수 등록
7. `onCan1()`
   ID 로 주인 찾기 — L.parse 가 아니면 R.parse
8. `events() >> 12`
   1 개 처리 후 남은 개수 (상위 비트) 반환 → 0 이 될 때까지 반복
9. `tx1() → can1.write()`
   빈 메일박스로 바로 송신 , 차 있으면 TX 큐에 대기

고정소수점 변환 · 비트 패킹

06 실수를 n 비트 정수로 — 해상도 = 범위 ÷ $(2^n - 1)$



$$u = \mathrm{round}\left((x - x_{min}) \cdot (2^n - 1) \,/\, (x_{max} - x_{min}) \right)$$

p = 1.0 → 13.5 × 65 535 / 25 = 35 388.9 → 35 389 = 0x8A3D
→ 상위 바이트 먼저 : **D0 = 0x8A, D1 = 0x3D**

12 bit 두 개가 바이트를 나눠 쓰는 법 `mit(1, −2, 50, 1.5, 5)`




```
f.data[3] = ((V & 0xF) << 4) | (KP >> 8);   //  반반 나눠 담기
```

필드	비트	범위	1 LSB (해상도)
p 위치	16	±12.5 rad	0.00038 rad (0.022°)
v 속도	12	±8 rad/s	0.0039 rad/s
Kp	12	0 – 500	0.122 N·m/rad
Kd	12	0 – 5	0.0012 N·m·s/rad
τ 토크	12	±28 N·m	0.014 N·m

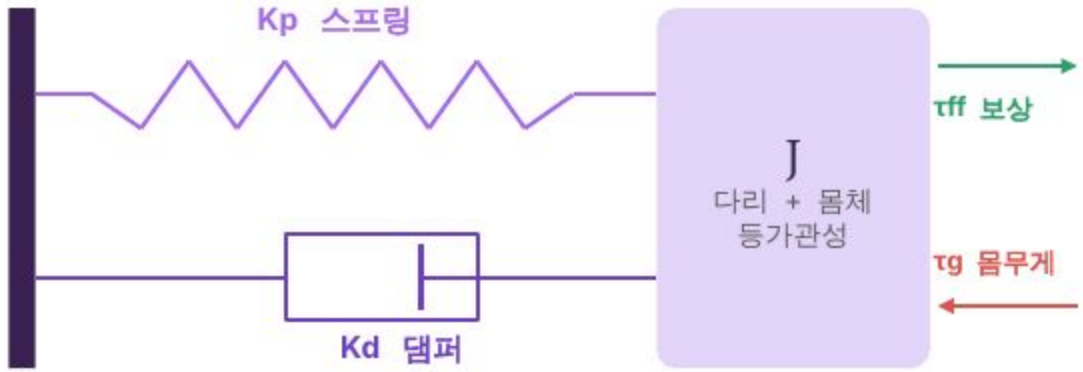
dm_motor.cpp

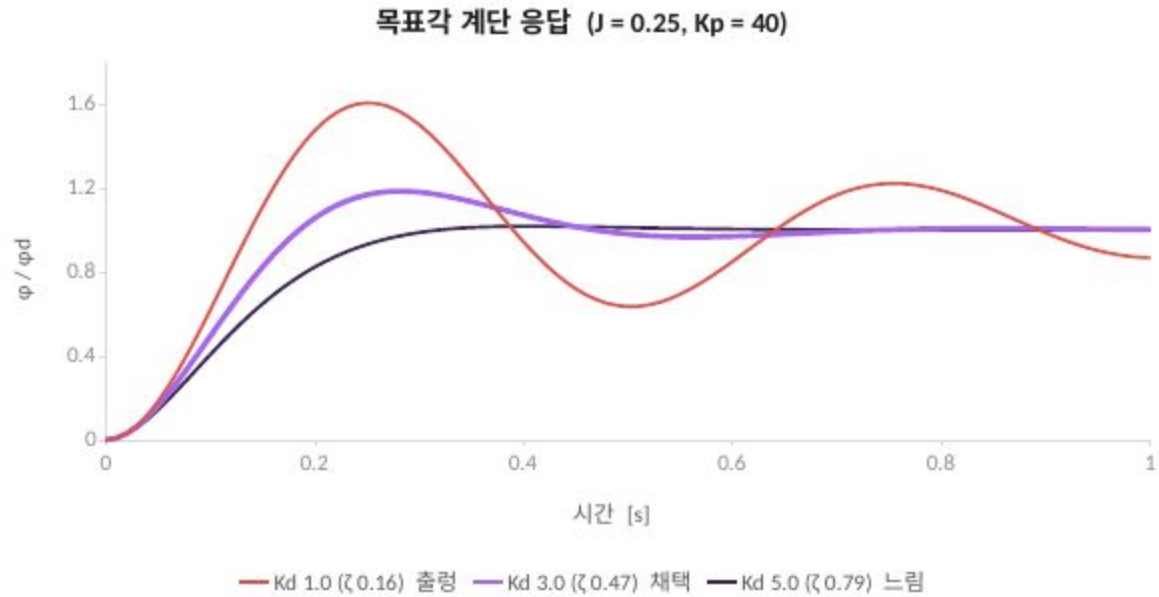
```
uint32_t floatToUint(float x, float lo,
                     float hi, uint8_t bits) {
  x = clampf(x, lo, hi);        //  범위 밖은 자름
  return (x - lo) * ((1u << bits) - 1)
         / (hi - lo) + 0.5f;    //  반올림
}
float uintToFloat(uint32_t u, ...)  //  역변환
{ return u * (hi - lo) / ((1u << bits) - 1) + lo; }
```

PMAX · VMAX · TMAX 가 모터 설정과 다르면 모든 값이 비례해서 틀어짐 → **config.h = 디버깅 툴 (12.5 / 8 / 28)**

MIT 모드 = 가상 스프링 · 댐퍼

07 Kp 는 스프링 , Kd 는 댐퍼 — 고관절 PD 제어의 원리





모터 안에서

$$\tau = K_p(\varphi_d - \varphi) - K_d\omega + \tau_{ff}$$

$$J\alpha = \tau - \tau_g \Rightarrow \omega_n = \sqrt{(K_p/J)},\ \zeta = K_d / 2\sqrt{(K_pJ)}$$

상태	J [kg·m²]	Kd	ζ (감쇠비)	ωn
공중 READY	≈ 0.02	1.5	0.84	45 rad/s · 7 Hz
서 있음 BALANCE	0.21 – 0.28	3.0	0.45 – 0.52	12 – 14 rad/s · 2 Hz

Kp = 40 · J ≈ 다리 1개가 받는 질량 × (dh/dφ ≈ 0.30 m/rad)² → 실물에서 튜닝

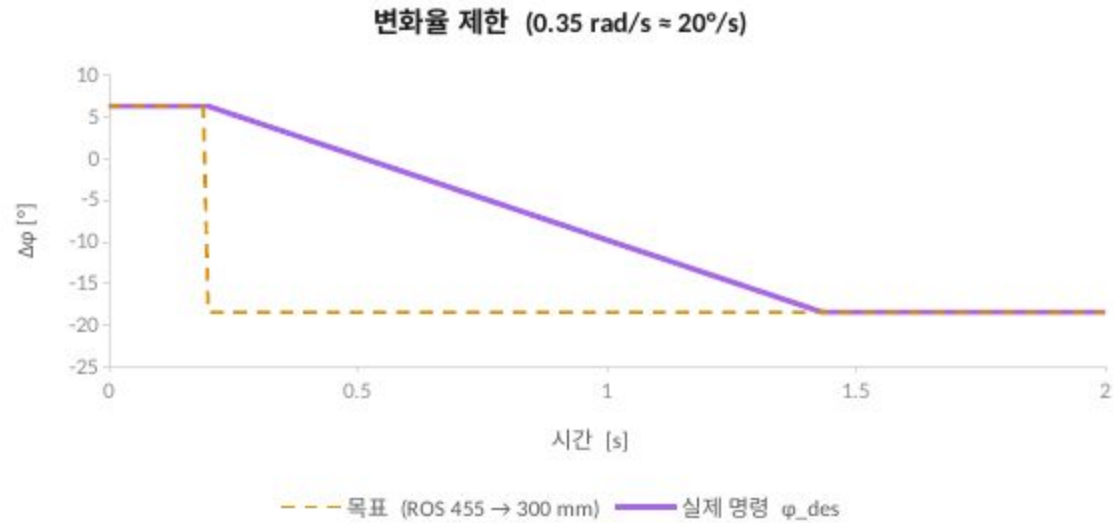
중력보상이 필요한 이유

- τff 없이 정지 오차 = τg / Kp = 8.98 / 40 = 0.22 rad → **12.9°**
- τff = 0.8 τg → 남는 오차 1.80 / 40 = 0.045 rad → **2.6°**
- Kp 만 키우면 오차는 줄지만 떨림 · 소음 ↑ → 보상으로 해결

중력보상 · LUT · 변화율 제한

08 필요한 토크는 미리 넣고, 목표는 천천히 움직인다





선형보간 예 : 455 mm

t = (455 − 422) / (475 − 422) = 0.623

Δφ = 0° + 0.623 × 10° = 6.23°

τ = 9.59 + 0.623 × (8.61 − 9.59) = 8.98 → × 0.8 = 7.18 N·m

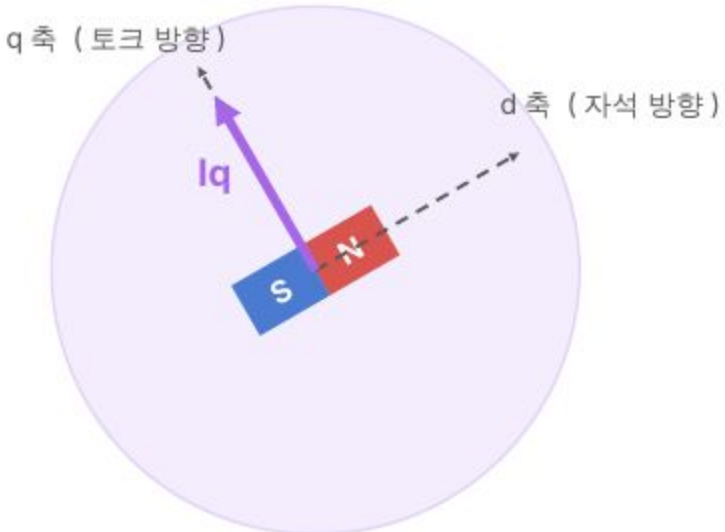
왜 천천히 ?

- 한 스텝 (2 ms) 최대 0.35 × 0.002 = 0.0007 rad (0.04°)
- 455 → 300 mm: 24.7° ÷ 20°/s ≈ 1.2 s — 급하면 몸체가 튐
- BALANCE 진입 시 τff 도 0 → 100 % 를 0.5 s 램프

```
phi_des = hipTraj.step(heightToPhi(h), HIP_RATE_MAX, dt);   // ① 높이 → 각도 → 천천히
ff_ramp = loaded ? clampf(ff_ramp + dt / 0.5f, 0, 1) : 0;    // ② 서 있을 때만 0.5 s 에 걸쳐 켬
tffL    = ff_ramp * gravityTau(hipPhi(hipL, DM_DIR_L));      // ③ 지금 각도의 중력 토크 × 0.8
```

휠 토크 제어 원리 (FOC)

09 τ = Kt · Iq — 전류를 회전자 자석과 90° 로 흘린다



힘 = 전류 × 자기장 → 자석과 90° 일 때 최대 · Id = 0

$$\tau = K_t \cdot I_q$$

- WA172E Kt = 1 N·m/A → 2 N·m = 2 A
- current_lim 10 A → 휠 한계 8 N·m

ODrive 안 (토크 모드)

`setInputTorque(τ)`
CAN 0x00E · 500 Hz

↓

Iq* = τ / Kt
목표 전류

↓

전류 PI 제어
8 kHz · Clarke / Park 변환

↓

SVM PWM
3 상 전압 출력

↓

모터 3 상 전류
→ 토크 → 바퀴 회전

ODrive 의 위치 · 속도 루프는 쓰지 않음
→ Teensy LQR 이 그 역할

왜 켤 때마다 캘리브레이션 ?

$\theta_e = 15 \times \theta_m +$ 오프셋

- FOC 는 자석의 전기각 θe 를 알아야 Iq 방향을 잡음
- 증분 엔코더 (인덱스 없음) 는 켜진 순간 위치를 모름
- **state 7: 모터를 천천히 돌려 오프셋 측정 (≈ 3 s)**

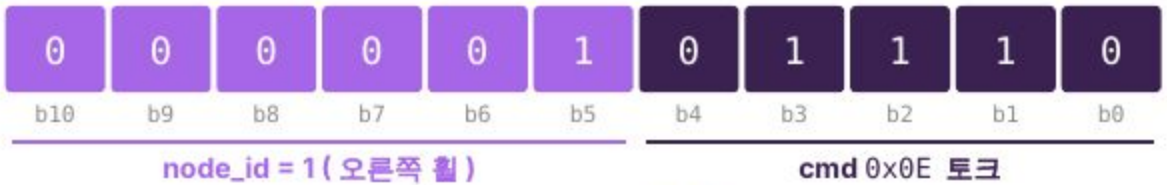
오프셋 오차	전기각	토크
1°	15°	97 %
3°	45°	71 %
6°	90°	0 % — 안 돎

3200 CPR → 기계각 0.11° · 전기각 1.7° · 바퀴 이동 0.17 mm 단위

ODrive CAN 메시지 해설

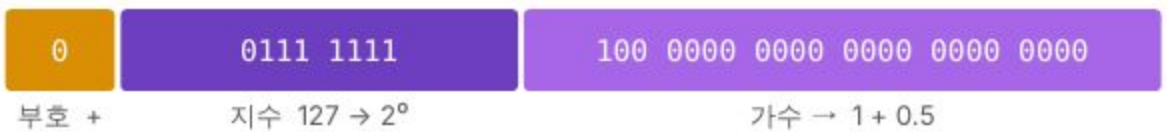
10 ID = node · cmd 비트 합치기 , 값 = IEEE 754 float

ID 11 bit = node 6 bit | cmd 5 bit



(1 << 5) | 0x0E = 32 + 14 = 46 = 0x02E

float 1.5 → 4 바이트 (IEEE 754 · 리틀엔디안)



$(-1)^0 \times 1.5 \times 2^{127-127} = 1.5$ → 0x3FC00000

CAN 데이터



작은 자리 바이트가 먼저 (리틀엔디안)

can_frame.h

```
void putF32(uint8_t* d, float v) { memcpy(d, &v, 4); }
float getF32(const uint8_t* d) {
  float v; memcpy(&v, d, 4); return v; }  // Teensy 도 리틀엔디안
```

메시지	방향	내용 → 코드에서
Heartbeat 0x01	OD → T	axis_error · state → FAULT 판단
Encoder 0x09	OD → T	pos [turn] × 2π → rad · vel
Set State 0x07	T → OD	u32: 7 캘리브 · 8 폐루프 · 1 IDLE
Input Torque 0x0E	T → OD	f32 N·m → Iq = τ / Kt
Bus V/I 0x17	RTR	빈 요청 → V · A 응답
Clear Errors 0x18	T → OD	캘리브 전에 에러 지움

ODrive 워치독

- 그 축으로 오는 CAN 메시지마다 타이머를 0 으로
- 0.2 s 동안 아무것도 안 오면 → 축 IDLE (토크 0)
- READY 에서도 버스전압 요청 (20 Hz) 이 워치독을 먹여 줌
- **CAN 구동 확인 후 enable_watchdog = True**

ODrive 설정 스크립트 해설

11 odrivetool 한 줄 한 줄 — 무엇을 정하고, 왜 그 값인가

코드 (odrv0. …)	값	뜻 · 이유
`can.config.baud_rate`	**1000000**	CAN 속도 — Teensy CAN2 와 같아야 서로 알아들음
`config.dc_bus_undervoltage_trip_level`	**19.0 V**	6S 하한 (≈3.2 V/셀) 아래면 정지 → 배터리 과방전 보호
`config.brake_resistance · enable_brake_resistor`	**2.0 Ω · True**	감속 때 생기는 회생 에너지를 열로 → 버스 과전압 방지 (DM 32 V 보호)
`axis.motor.config.pole_pairs`	**15**	30 극 → 전기각 = 15 × 기계각 (FOC 가 자석 방향을 계산하는 기준)
`motor.config.torque_constant`	**1**	Kt [N·m/A] — 토크 명령 τ 를 전류 Iq = τ / Kt 로 바꿈
`motor.config.calibration_current`	**5 A**	상저항 · 인덕턴스를 잴 때 흘리는 전류
`motor.config.resistance_calib_max_voltage`	**8 V**	측정 전압 한계 — 5 A × 1.37 Ω ≈ 6.9 V 보다 커야 측정 성공
`motor.config.current_lim`	**10 A**	최대 전류 = 최대 토크 10 N·m (펌웨어는 8 N·m 에서 먼저 자름)
`encoder.config.mode · cpr`	**INCREMENTAL · 3200**	A/B 상 800 PPR × 4 체배 = 한 바퀴 3200 카운트 (0.11°)
`encoder.config.use_index · startup_…_index_search`	**False**	인덱스 펄스 없음 → 부팅마다 오프셋 캘리브 (state 7) 필요
`controller.config.control_mode · input_mode`	**1 · 1**	토크 제어 · passthrough (명령을 필터 없이 바로 사용)
`controller.config.vel_limit`	**3 turn/s**	토크 모드에서도 이 속도를 넘으면 토크를 줄임 (≈ 1.6 m/s)
`config.can.node_id`	**0 / 1**	축 = CAN 노드 → 메시지 ID = node << 5 | cmd
`config.can.encoder_rate_ms · heartbeat_rate_ms`	**2 · 100**	위치 · 속도 500 Hz, 상태 · 에러 10 Hz 를 묻지 않아도 자동 송신
`config.watchdog_timeout · enable_watchdog`	**0.2 s · True**	CAN 명령이 0.2 s 끊기면 축 정지 (CAN 구동 확인 후 켬)
`motor.config.pre_calibrated → save_configuration()`	**True**	잰 R · L 을 플래시에 저장 → 다음 부팅엔 모터 캘리브 생략

IMU 자세 추정

12 자이로 (빠르지만 드리프트) + 가속도 (느리지만 기준) → 쿼터니언



쿼터니언 q = (w, x, y, z)

축 n 으로 α 만큼 회전 = (cos α/2, n · sin α/2) — 짐벌락 없음

$$\text{pitch} = \text{asin}(2(wy - zx))$$

$$\text{roll} = \text{atan2}(2(wx + yz), 1 - 2(x^2 + y^2))$$

imu.h

```
void updateEuler() {                    // ZYX
  roll_ = atan2f(2*(qw*qx + qy*qz),
                 1 - 2*(qx*qx + qy*qy));
  float s = 2*(qw*qy - qz*qx);
  pitch_ = asinf(clamp(s, -1, 1));
}
float pitch()     { return SIGN * pitch_; }
float pitchRate() { return SIGN * gy; }  // 자이로
```

ω 는 왜 자이로 직접 ?

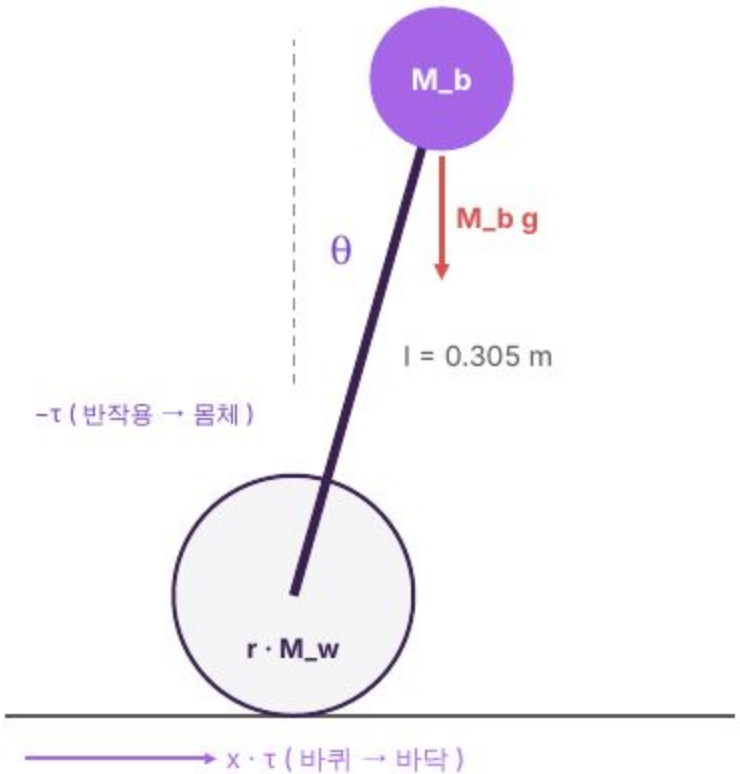
- θ 를 2 ms 마다 미분하면 잡음이 500 배로 커짐
- 자이로는 각속도 그 자체 · 지연 없음
- LQR 의 Kω 항이 깨끗해야 떨림이 없음

θeq — 균형각

- 무게중심이 휠축 바로 위일 때의 IMU 피치
- 장착 기울기 · 무게 배치 때문에 0 이 아님
- **/twlr/pitch 로 실측 평균 → config.h**

바퀴 달린 역진자 모델

13 넘어지려는 몸체를 바퀴가 밑에서 받쳐 세운다



운동방정식 (sin θ ≈ θ 로 선형화)

$$(M_w + M_b + I_w/r^2)\, a + M_b l\, \alpha = \tau / r$$

$$M_b l\, a + (I_b + M_b l^2)\, \alpha - M_b g\, l\, \theta = -\tau$$

a = 전진 가속도 , α = 기울기 각가속도

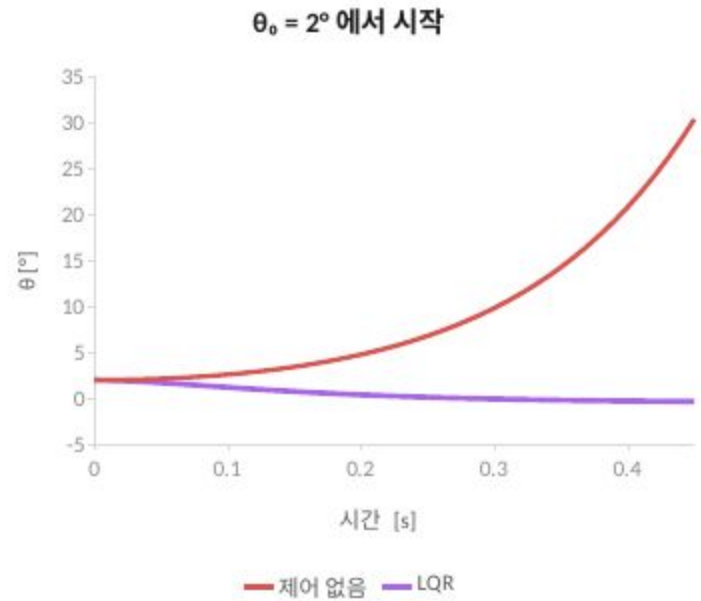
우리 로봇 숫자를 넣으면

$$a = -9.71\,\theta + 2.34\,\tau$$

$$\alpha = 57.6\,\theta - 8.33\,\tau$$

열린 루프 극점 ±7.59 rad/s → 불안정

M_b 몸체	6.84 kg
l 무게중심 높이	0.305 m
M_w 휠 + 다리	4.40 kg
r 휠 반경	0.086 m
I_w · I_b (추정)	0.0085 · 0.07 kg·m²



0.09 s 마다 기울기 2 배

$e^{7.59\,t} = 2 \rightarrow t = \ln 2 / 7.59 = 0.091\,s$

→ 제어 주기 2 ms (45 배 빠름)

IMU 2.5 ms · CAN 왕복 < 1 ms

목차

LQR — 게인을 고르는 방법

14 오차와 토크의 "비용"을 최소로 하는 K를 리카티 방정식으로 푼다

비용 함수

$$J = \int (x^T Q\, x + u^T R\, u)\, dt$$

x = [x, v, θ, ω] 오차 · u = 휠 토크
Q = diag(5, 2, 200, 5) — θ 오차가 가장 비쌈
R = 1 — 토크를 쓰는 비용

리카티 방정식

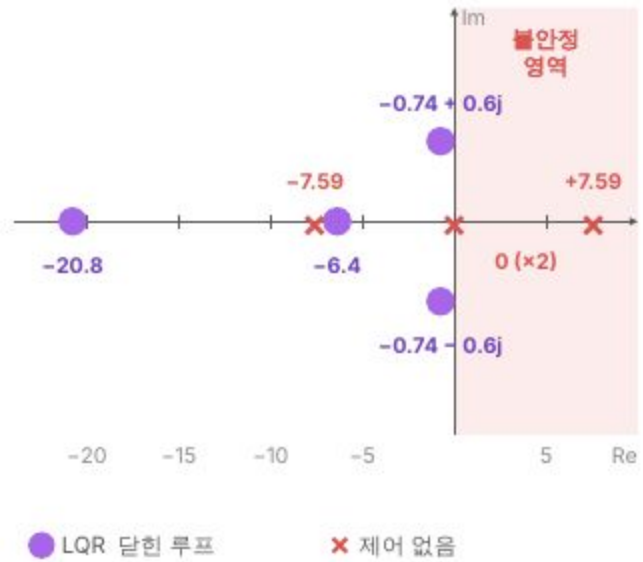
$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$

$$K = R^{-1}B^T P, \quad u = -K\,x$$

키우면	효과
Q_θ	기울기 빨리 복원 · 토크↑ · 떨림 위험
Q_x	제자리 유지 강함 · 흔들림↑
R	토크 아낌 · 반응 느려짐

K_x = √(Q_x / R) = √5 = 2.24
— 위치 게인은 Q_x 로 바로 정해짐

극점 지도 (복소평면)



+7.59 극점 (오른쪽) = 불안정
→ **−20.8 · −6.4 · −0.74 ± 0.6j**
모두 왼쪽 → 안정, 가장 느린 − 0.74 = 위치 복귀 (≈1.4 s)

MATLAB · Simulink 재계산

```
A = [0 1  0     0
     0 0 -9.71  0
     0 0  0     1
     0 0  57.6  0];
B = [0; 2.34; 0; -8.33];
Q = diag([5 2 200 5]); R = 1;
K = lqr(A, B, Q, R)
% -2.24 -4.10 -28.4 -4.59
```

앞으로 가려면 먼저 뒤로

- Kx 와 Kθ 부호가 같다 (비최소위상)
- 바퀴가 살짝 뒤로 → 몸체가 앞으로 기울고 → 따라서 전진

예 : θ = 5° (0.0873 rad)
u = 28.4 × 0.0873 = 2.48 N·m
PC 테스트 출력과 일치

목차

밸런싱 코드 한 줄씩

15 Balancer::step() — 수식이 코드가 되는 과정

```
// twlr_teensy.ino — 상태 추정
x = R_W * (phiw + theta);           // [1]
v = R_W * (wv + theta_dot);

// control.h — Balancer::step()
v_cmd = clampf(in.v_cmd, -0.6f, 0.6f);     // [2]
x_ref_ += v_cmd * in.dt;                   // [3]
x_ref_ = clampf(x_ref_, x - 0.3f,
                        x + 0.3f);         // [4]
u = K_X  * (x - x_ref_) + K_V * (v - v_cmd)
  + K_TH * (theta - THETA_EQ)
  + K_W  * theta_dot;                      // [5]
diff  = K_YAW * (w_cmd - yaw_rate);        // [6]
tau_L = clampf(0.5f*u - 0.5f*diff, -8, 8); // [7]
tau_R = clampf(0.5f*u + 0.5f*diff, -8, 8);

// twlr_teensy.ino — 송신
tx2(wheelL.setInputTorque(WHEEL_DIR_L * tau_L)); // [8]
```

1. **[1] x = r(φ + θ)**
   엔코더는 몸체 기준 각도 — 몸체가 θ 기울면 안 굴러도 −θ 로 보임 → +θ 보정
2. **[2] 속도 제한**
   /cmd_vel 을 ±0.6 m/s 로 자름
3. **[3] 목표 위치 이동**
   명령 속도로 x_ref 를 계속 밀어 줌 → "따라오기"가 곧 주행
4. **[4] 와인드업 방지**
   밀려서 멀어져도 오차 최대 0.30 m → 복귀 토크 폭주 방지
5. **[5] LQR**
   네 오차 × 게인 = 총 전진 토크 u
6. **[6] 선회**
   요레이트 오차 → 좌우 토크 차이
7. **[7] 분배 · 포화**
   반씩 나누고 ±8 N·m 에서 자름 (전류 10 A 안)
8. **[8] 방향 부호**
   모터가 마주 보고 달려서 한쪽은 −1

메인 루프 · 상태머신 코드

16 twlr_teensy.ino — loop() 가 매번 하는 일과 모드별 동작 (요약)

twlr_teensy.ino (break · 중괄호 일부 생략)

```
void loop() {
  while (can1.events() >> 12) {}          // [1]
  while (can2.events() >> 12) {}
  imu.poll(now);   readHost(now);         // [2]
  if (now_us - last_ctrl_us >= CTRL_PERIOD_US)
    controlStep(el * 1e-6f, now);         // [3]
  if (now - last_tel_ms >= TELEM_PERIOD_MS)
    sendTelemetry(now);                   // [4]
}
void controlStep(float dt, uint32_t now) {
  bool want_enable = cmd.enable;          // [5]
  health = checkHealth(now);              // [6]
  if (critical && active) { enterFault(critical); return; }
  switch (mode) {
  case IDLE:                              // [7]
    tx1(hipL.disable());       // 힘 없이 각도만 받기
    if (want_enable && health == 0)
      tx2(wheelL.setState(ENCODER_OFFSET_CALIBRATION));
  case CALIB:                             // [8]
    if (wheelL.st().state == odrive::IDLE) {   // 캘리브 끝
      tx2(wheelL.setState(CLOSED_LOOP_CONTROL));  // 토크모드
      tx1(hipL.enable());  setMode(READY); }
  case READY:                             // [9]
    hipControl(dt, false);  tx2(wheelL.setInputTorque(0));
    if (cmd.balance && fabsf(theta - THETA_EQ) < 0.17f)
      setMode(BALANCE);
  case BALANCE:                           // [10]
    if (fabsf(theta - THETA_EQ) > 0.61f) enterFault(F_FALL);
    hipControl(dt, true);  out = balancer.step(in);
  case FAULT:                             // [11]
    // 휠 IDLE · 고관절 Kp 0 Kd 1 댐핑 1 s → disable
  }
}
```

1. **[1] CAN 수신 비우기**
   인터럽트가 쌓아 둔 프레임 → parse() (>>12 = 남은 개수)
2. **[2] IMU · 호스트**
   BNO085 새 값 · USB 명령 패킷 (CRC 확인) 읽기
3. **[3] 500 Hz 제어**
   2000 µs 마다 1 회, dt 는 실제 경과시간
4. **[4] 100 Hz 텔레메트리**
   92 B 상태 패킷 → LattePanda
5. **[5] enable 유지**
   ROS 가 끊겨도 마지막 값 유지 → 넘어지지 않음
6. **[6] 매 주기 점검**
   피드백 끊김 · 에러 → 활성 모드면 즉시 FAULT
7. **[7] IDLE**
   enable 오면 휠 엔코더 오프셋 캘리브 시작
8. **[8] CALIB → READY**
   하트비트 state 가 IDLE 로 돌아오면 캘리브 끝
9. **[9] READY**
   다리 높이만 유지 (Kd 1.5), 10° 안이면 밸런싱 진입
10. **[10] BALANCE**
    35° 넘으면 넘어짐, 아니면 Kd 3.0 + 중력보상 + LQR
11. **[11] FAULT**
    휠 끄고 다리는 천천히 힘 빼기, enable false 로 리셋

안전 설계 — 3 중 워치독

17 누가 멈추든 , 이웃이 알아채고 안전한 쪽으로 간다



멈춘 곳	알아채는 곳	시간	그 다음
LattePanda · ROS	Teensy	250 ms	v = 0 · 제자리 균형 유지 (enable · balance 유지)
Teensy	DM4340P	100 ms	모터 스스로 Disable (CAN Timeout 2000 × 50 µs)
Teensy	ODrive	0.2 s	축 IDLE (watchdog) → 휠 토크 0
DM 응답 없음 · state ≥ 8	Teensy	30 ms	FAULT: 휠 IDLE · 고관절 1 s 댐핑 → Disable
ODrive 응답 없음 · 에러	Teensy	300 ms	FAULT
IMU 멈춤 · |θ – θeq| > 35°	Teensy	50 ms · 즉시	FAULT (넘어짐)

시리얼 패킷 · CRC 원리

18 바이트 흐름에서 프레임을 찾고, 깨진 것은 버린다

Command 실제 바이트 (PC 교차 테스트 값 — C++과 Python이 같은 22바이트)

A5	5A	02	10	07	00	01	00	00	00	80	3E	00	00	00	BF	00	80	E3	43	8D	96
sync		type	len 16	seq = 7		en 1	bal 0	v = 0.25 m/s				w = −0.5 rad/s				h = 455 mm				CRC 0x968D	

CRC16-CCITT

$G(x) = x^{16} + x^{12} + x^5 + 1$

- 데이터를 G로 나눈 나머지 16 bit를 덧붙임
- 받는 쪽이 다시 계산 → 다르면 버림
- 1·2비트, 홀수 개, 16비트 이하 연속 오류는 100 % 검출
- 그 밖은 1 / 65 536 확률로만 통과

bridge.cpp · protocol.py 동일

```
for (byte b : body) {
  crc ^= b << 8;
  for (int i = 0; i < 8; i++)
    crc = (crc & 0x8000)
        ? (crc << 1) ^ 0x1021
        : (crc << 1);
}  // init 0xFFFF
```

Parser — 다시 동기 맞추기

A5 5A 찾기

↓

type · len 읽기

↓

len + 2 바이트 기다림

↓

CRC 일치?

↓

예 → 패킷 사용

아니오 → 2바이트 버리고 다시 찾기

USB가 바이트를 잘라 보내도 · 중간부터 읽어도 복구

bridge.h

```
#pragma pack(push, 1)    // 패딩 없음
struct Command {         // 16 B
  uint16_t seq;          // H
  uint8_t  enable;       // B
  uint8_t  balance;      // B
  float v_cmd;           // f
  float yaw_rate_cmd;    // f
  float height_mm;       // f
};
```

`struct.pack('<HBBfff', …)`

< 리틀엔디안 · 패딩 없음

H u16 · B u8 · f f32 → 2+1+1+4×3 = 16

ROS 2 브리지 코드

19 bridge_node.py — 토픽을 패킷으로, 패킷을 토픽으로 (요약)

twlr_bringup/bridge_node.py

```
def on_twist(self, msg):                          # [1]
    self.v_cmd = msg.linear.x
    self.w_cmd = msg.angular.z
    self.last_twist = time.monotonic()

def send_command(self):                           # [2] 50 Hz
    alive = (time.monotonic() - self.last_twist) < self.twist_timeout
    v = self.v_cmd if alive else 0.0              # [3]
    pkt = encode_command(self.seq, self.enable, self.balance,
                         v, w, self.height)       # [4]
    with self.lock:
        self.ser.write(pkt)

def read_loop(self):                              # [5] 별도 스레드
    data = self.ser.read(512)
    for ptype, payload in self.parser.feed(data):     # [6]
        if ptype == TYPE_TELEMETRY and len(payload) == 92:
            self.publish(decode_telemetry(payload))

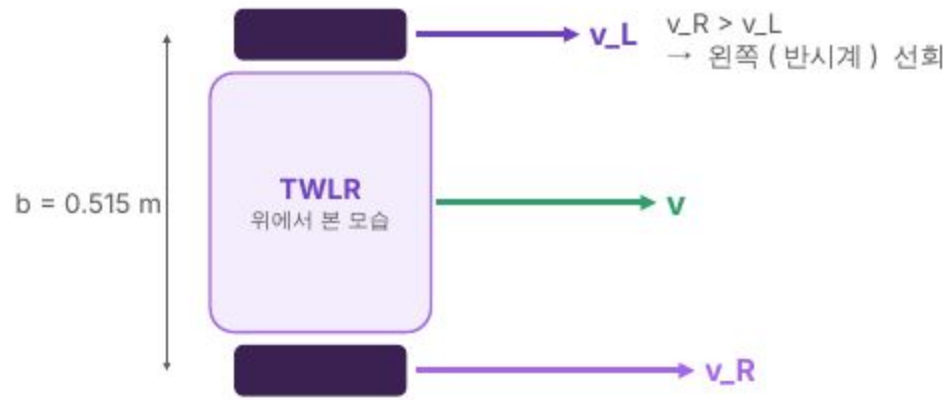
def publish(self, t):                             # [7]
    imu.orientation.w, imu.orientation.x, ... = t.quat
    js.position = [t.hip_pos[0], t.hip_pos[1], t.wheel_pos[0], ...]
    vl = self.r * (t.wheel_vel[0] + t.pitch_rate)     # [8]
    w = (vr - vl) / self.track
    self.yaw += w * dt                            # [9]
    self.x += v * math.cos(self.yaw) * dt
```

1. **[1] /cmd_vel 콜백**
   키보드가 보낸 속도 저장 + 받은 시각 기록
2. **[2] 50 Hz 타이머**
   create_timer(1/50) 가 주기적으로 호출
3. **[3] 키 떼면 정지**
   1 s 동안 새 /cmd_vel 없으면 v = w = 0
4. **[4] 패킷 만들기**
   struct.pack('<HBBfff') + CRC16 → 22 B
5. **[5] 수신 스레드**
   읽기는 기다리는 동작 → ROS 콜백과 분리, 쓰기는 lock
6. **[6] 프레임 찾기**
   A5 5A → 길이 → CRC, 깨진 프레임은 버림
7. **[7] 토픽 발행**
   /imu/data · /joint_states · /twlr/pitch · /twlr/battery
8. **[8] 바닥 기준 속도**
   휠 각속도 + 피치 각속도 (엔코더는 몸체 기준)
9. **[9] 오도메트리**
   차동구동 운동학 적분 → /odom + TF

ROS 2 동작 원리 · 오도메트리

20 토픽으로 느슨하게 연결하고, 바퀴 회전을 적분해 위치를 추정





twlr_bridge 안의 두 흐름

- Timer 50 Hz — 최신 명령을 Command 패킷으로 송신
- Reader 스레드 — 시리얼 읽기 (기다림) → 파싱 → 토픽 발행
- Lock — 두 스레드가 같은 포트에 쓸 때 충돌 방지
- /cmd_vel 1 s 안 오면 v = 0 (키를 떼면 정지)
- QoS sensor_data — /imu/data 는 최신 값 우선, 재전송 없음

바닥 기준 바퀴 속도 = 엔코더 + 피치 각속도

$$v = r\,(\omega_L + \omega_R)\,/\,2, \quad \omega_z = r\,(\omega_R - \omega_L)\,/\,b$$

$$\psi \mathrel{+}= \omega_z \Delta t, \quad x \mathrel{+}= v\cos\psi\,\Delta t, \quad y \mathrel{+}= v\sin\psi\,\Delta t$$

100 Hz 텔레메트리마다 적분 → /odom + TF (odom → base_link)

바퀴 미끄러짐 · 적분 오차가 쌓임 → 나중에 라이다 SLAM 이 보정

단품 시험 스케치 해설

21 키 하나 = CAN 프레임 하나 — 통합 전에 모터를 하나씩 확인

키	dm_single_test 가 보내는 것	확인할 것
e	enable() → ID 0x01: FF×7 FC	state 0 → 1
d	disable() → FF×7 FD	힘이 빠짐 , 각도는 계속 읽힘
z	disable → setZero() FF×7 FE	지금 각도가 0 rad 로 저장
h	mit(현재각 , 0, Kp 10, Kd 0.5, 0)	손으로 밀면 스프링처럼 버팀
s	mit(p_0 + 0.15·sin πt, 0, 20, 1, 0)	±8.6°, 0.5 Hz 추종
0	3 s 동안 0 rad 로 선형 이동	영점으로 천천히 복귀
p	100 ms 마다 상태 출력 on / off	state · pos · τ · 온도 · age

키	odrive_can_test 가 보내는 것	확인할 것
c	setState(7) → 0x007 / 0x027	바퀴가 천천히 돌며 오프셋 측정
l	속도모드 · VEL_RAMP → setState(8)	state = 8 (페루프)
v	setInputVel(±0.5) 0x00D / 0x02D	두 바퀴가 같은 방향으로 회전
s	setInputVel(0)	정지
t	토크모드 → setInputTorque(±0.3)	살짝 미는 힘
i	setState(1) IDLE	힘이 빠짐
x	clearErrors() 0x018 / 0x038	axis_error 0

왜 200 Hz 로 disable 을 계속 보내나 ?

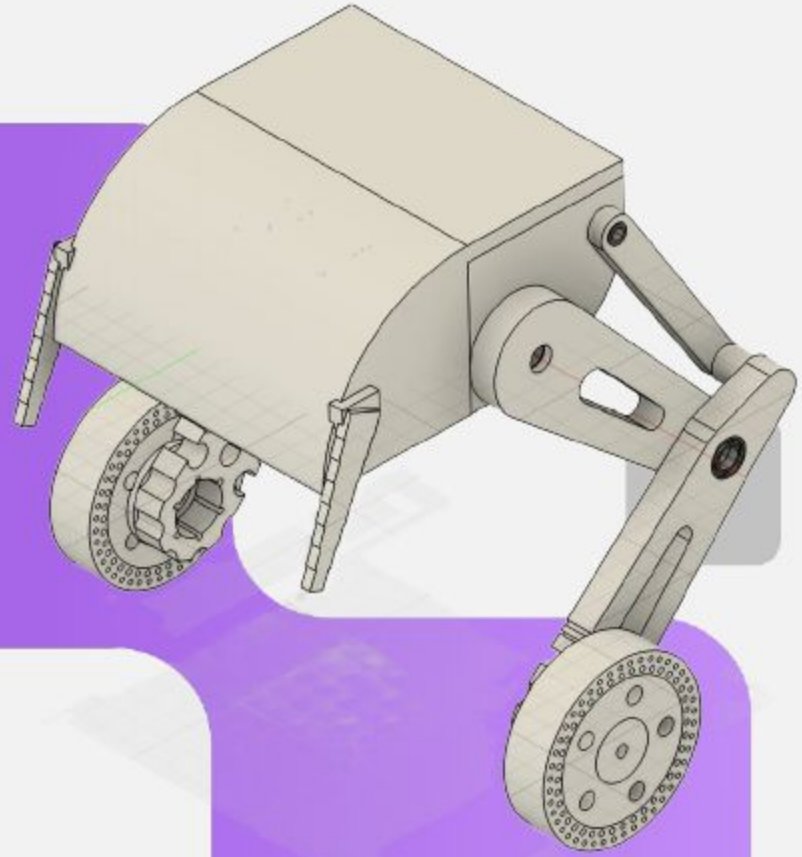
- DM 은 명령 1 개를 받아야 피드백 1 개를 돌려줌
- 힘을 빼 둔 상태에서도 각도 · 온도를 보려면 계속 물어봐야 함
- age 가 계속 커지면 → 배선 · ID · 120 Ω 확인

50 ms 마다 자동으로 하는 일

- requestBusVI() — RTR 0x017 → 버스 전압 · 전류 응답
- 명령이 오가므로 ODrive 워치독도 함께 먹여 줌
- v 의 좌우 부호가 반대인 이유 : 모터가 거울 배치

PART 08

OPERATION MANUAL



운용 방법

BOM 추가 부품을 모두 단 상태를 가정 — 1회 설정 · 켜기 · 밸런싱 · 주행 · 끄기 · 비상 대처 · 튜닝

최종 구성 · 1회 설정 · 부호 측정 100–102p

켜기 · 밸런싱 · 주행 · 끄기 103–105p

비상 대처 · 고장 코드 106–107p

튜닝 · 배터리 · 점검 108–109p

최종 구성 (BOM 추가 반영)

01 이 구성을 기준으로 켜고 · 움직이고 · 끄는 순서를 설명

전원



신호



추가 부품	위치	운용에서의 역할
30 A 퓨즈	배터리 + 직후	단락 · 과전류 시 차단
메인 · 비상 스위치	퓨즈 다음 · 손 닿는 곳	비상 시 전체 전원 차단 (1 순위)
120 Ω (CAN1 끝)	DM R 쪽 끝	반사 방지 — 양 끝 합쳐 60 Ω
트위스트 페어	CAN1 · CAN2	모터 잡음 상쇄
2 Ω 브레이크 저항	ODrive	감속 회생 전압 흡수 (DM 32 V 보호)

운용 준비물

- ☐ 노트북 — SSH 또는 모니터 · 키보드
- ☐ 충전된 배터리 (≥ 24.5 V) + 예비
- ☐ LiPo 방화백 · 멀티미터
- ☐ 서포터 (스탠드) — 바퀴를 띄우는 받침
- ☐ USB-TTL 3.3 V — DM 설정 때만
- ☐ 주행 공간 3 × 2 m 이상
- ☐ 2 인 1 조 — 조작 1 · 안전 1

처음 한 번만 하는 설정

02 모터 ID → ODrive → 펌웨어 → ROS 2 — 순서대로 1회

1 DM4340P

Windows · Damiao 툴

- 모터 1개씩 연결 (24 V · 전원 전류제한 3 A)
- ESC_ID 1 / 2 · MST_ID 0x11 / 0x12 · CTRL_MODE 1 (MIT)
- PMAX 12.5 · VMAX 8 · TMAX 28 · TIMEOUT 2000
- 다리를 기준 자세 (Δφ = 0) 에 두고 영점 저장

확인 dm_single_test → e → 피드백 state 1 · 각도 0 근처

2 ODrive

LattePanda · odrivetool

```
# odrive_can_setup 내용을 블록 단위로 붙여넣기 → 재부팅
odrv0.axis0.requested_state = 4       # 모터 캘리브 ( 휠 공중 )
odrv0.axis0.motor.config.pre_calibrated = True
odrv0.axis1.requested_state = 4
odrv0.axis1.motor.config.pre_calibrated = True
odrv0.save_configuration()
```

확인 dump_errors(odrv0) → 에러 없음 · can.node_id 0 / 1

3 Teensy 펌웨어

노트북 · Arduino IDE 2

- Teensyduino 설치 (FlexCAN_T4 포함)
- 라이브러리 매니저 → Adafruit BNO08x
- 보드 Teensy 4.1 · USB Type Serial · CPU 600 MHz
- firmware/twlr_teensy/twlr_teensy.ino 열기 → 업로드

확인 LattePanda 에서 ls -l /dev/twlr_teensy 가 보임

4 ROS 2 패키지

LattePanda · 터미널

```
sudo apt install ros-jazzy-teleop-twist-keyboard python3-serial
mkdir -p ~/twlr_ws/src && cp -r twlr_bringup ~/twlr_ws/src/
cd ~/twlr_ws && colcon build --symlink-install
echo "source ~/twlr_ws/install/setup.bash" >> ~/.bashrc
sudo cp src/twlr_bringup/config/99-twlr-teensy.rules /etc/udev/rules.d/
sudo udevadm control --reload && sudo udevadm trigger
```

확인 ros2 pkg list | grep twlr → twlr_bringup

방향 · 부호 · 균형각 측정

03 config.h 의 [측정] 값 — 실물에서 한 번 재서 넣고 다시 업로드

config.h 값	어떻게 재나	넣는 값
DM_DIR_L / R	dm_single_test · p → 다리를 손으로 펴 봄	펼 때 pos 증가 → +1
WHEEL_DIR_L / R	odrive_can_test · c → l → v	전진 방향으로 돌면 +1
IMU 피치 축	READY 에서 /twlr/pitch 보며 앞으로 기울임	안 변하면 EULER_ROLL = true
IMU_PITCH_SIGN	앞으로 기울였을 때	pitch 증가 → +1
THETA_EQ_RAD	균형점을 손으로 잡고 5 s 평균	보통 − 0.1 ~ +0.1 rad
DM 영점	기준 자세에서 z 키	재부팅 후에도 0 근처

θ_eq 잡는 법

- READY · 몸체 손잡이를 잡고 바퀴만 바닥에 살짝 닿게
- 앞 · 뒤 어느 쪽으로도 손에 힘이 안 걸리는 각도를 유지
- 밸런싱 후 한쪽으로 계속 흐르면 ±0.01 rad 씩 보정

LattePanda 터미널 → config.h

```
# θ_eq 측정 (100 Hz × 5 s 평균)
ros2 topic echo /twlr/pitch \
  --field data | grep -v -- --- \
  | head -n 500 \
  | awk '{s+=$1} END {print s/NR}'
# → 예 : 0.034

// config.h
constexpr float THETA_EQ_RAD = 0.034f;
```

부호가 틀리면 ?

- 밸런싱 시작 즉시 한쪽으로 튀어 나감
- 그래서 반드시 서포터 위에서 먼저 확인
- 값을 바꾸면 업로드 (≈ 20 s) → 다시 확인

전원 켜기 → READY

04 서포터 위 , 바퀴가 공중에 뜬 상태에서 시작

BOOT	IDLE	CALIB	READY
LED 점등 · 연결 대기	힘 없음 · 각도만 읽음	바퀴가 천천히 앞뒤로 ≈ 3 s	다리가 455 mm 로 천천히 이동

1. **로봇을 서포터에 올림**
   바퀴가 바닥 · 케이블에 닿지 않는지 확인

2. **배터리 연결 → 메인 스위치 ON**
   DM · ODrive LED 확인 · 비상 스위치 위치 확인

3. **LattePanda 부팅 → 터미널 1: launch**
   로그에 'serial connected' · 'mode: IDLE'

4. **터미널 2: /twlr/enable true**
   로그 mode: CALIB → READY (바퀴가 돎 — 손 주의)

5. **faults 비어 있는지 확인**
   문제 있으면 8-08 고장 코드 표

LattePanda 터미널

```
# 터미널 1 — 브리지 실행 ( 로그를 계속 봄 )
ros2 launch twlr_bringup twlr_bringup.launch.py

# 터미널 2 — 기동 : 휠 캘리브 → READY
ros2 topic pub --once /twlr/enable \
  std_msgs/msg/Bool "{data: true}"

# 확인
ros2 topic echo --once /twlr/faults
ros2 topic echo --once /twlr/battery --field voltage
```

CALIB 중 바퀴가 돈다 — 손 · 케이블을 바퀴에서 떨어뜨리기

밸런싱 시작 · 주행

05 균형점 근처에서 balance → 천천히 손을 놓는다

1. **바퀴를 바닥에 내림**
   한 명이 몸체를 잡고 서포터에서 내림 (READY 유지)
2. **몸체를 세워 잡음**
   θ_eq ± 10° 안 — 벗어나면 BALANCE 거부
3. **balance true**
   바퀴가 버티는 힘이 느껴지면 손을 천천히
4. **10 s 제자리 확인**
   그 다음 teleop 으로 주행

높이 바꾸기
- 290 – 520 mm · 약 20°/s 로 천천히
- **452 mm 아래는 정격 초과 → 짧게만**
- 코일 80 °C ↑ → 자동 510 mm

teleop 키

u 전진 좌	i 전진	o 전진 우	x 직진 – 10 %
j 좌회전	k 정지	l 우회전	w 직진 +10 %
m 후진 좌	, 후진	. 후진 우	c 회전 – 10 %

기본 0.5 m/s · 1.0 rad/s
→ x 를 5 번 눌러 ≈ 0.3 m/s 로 시작
- Teensy 가 0.6 m/s · 1.5 rad/s 에서 자름
- 키를 떼면 1 s 뒤 정지 · k = 즉시
- 터미널 창이 선택돼 있어야 키가 먹힘

LattePanda 터미널

```
# 밸런싱 시작 (손으로 세운 채)
ros2 topic pub --once /twlr/balance \
  std_msgs/msg/Bool "{data: true}"

# 키보드 주행 (새 터미널)
ros2 run teleop_twist_keyboard \
  teleop_twist_keyboard

# 높이 [mm]
ros2 topic pub --once /twlr/height \
  std_msgs/msg/Float32 "{data: 400.0}"

# 감시
ros2 topic echo /twlr/hip_temp
ros2 topic hz /imu/data      # ≈ 100
ros2 topic echo /twlr/loop_us
                             # < 2200
```

정지 · 종료 순서

06 반드시 서포터 위에서 끈다 — 순서가 바뀌면 넘어진다

1 k 로 정지
teleop Ctrl+C

→

2 들어서 서포터 위
바퀴 공중

→

3 balance false
→ READY · 휠 토크 0

→

4 enable false
→ IDLE · 다리 힘 풀림

→

5 launch Ctrl+C
sudo poweroff

→

6 스위치 OFF
배터리 분리 · 보관

LattePanda 터미널

```
# 3 → 4 ( 서포터 위에서만 )
ros2 topic pub --once /twlr/balance \
  std_msgs/msg/Bool "{data: false}"
ros2 topic pub --once /twlr/enable \
  std_msgs/msg/Bool "{data: false}"
```

enable false 뒤 다리에 힘이 빠짐

- 서포터가 몸체를 받치고 있는지 먼저 확인
- Ctrl+C 로 브리지를 끄면 enable false 가 자동 전송
- 배터리는 맨 마지막 — 스위치 OFF 후 분리

하면 안 되는 것

바닥에서 balance false
→ 휠 토크 0 → 그대로 넘어짐

바닥에서 enable false · launch Ctrl+C
→ 다리 힘 풀림 + 휠 정지 → 주저앉으며 넘어짐

밸런싱 중 배터리 분리
→ 넘어짐 + LattePanda 비정상 종료 (파일 손상)

예외 : 위험할 때는 넘어지더라도 비상 스위치

비상 상황 대처

07 상황별 첫 동작 — 확신이 없으면 비상 스위치

상황	즉시 할 일	원인 · 다음 조치
앞뒤로 점점 커지는 흔들림	몸체를 잡고 서포터로 → balance false	Kθ · Kω 조정 · θ_eq 재측정
한쪽으로 튀어 나감 (폭주)	**비상 스위치 OFF**	WHEEL_DIR · IMU 부호 → 서포터에서 재확인
넘어짐 (35° 이상)	자동 FAULT → 휠 정지 · 다리 1 s 댐핑	파손 확인 → enable false → true
LattePanda · 터미널 멈춤	로봇은 제자리 균형 유지 (v = 0)	몸체를 잡고 서포터로 → 비상 스위치
고관절 과열 (HIP_TEMP)	자동으로 510 mm 로 올라섬	주행 중단 · 서포터에서 식힌 뒤 재개
타는 냄새 · 연기 · 배터리 부품	**비상 스위치 → 배터리 분리 → 방화백**	원인 확인 전 재가동 금지
배터리 21.0 V 이하	주행 중단 → 종료 순서	19.0 V 에서 ODrive 차단 → 휠 갑자기 정지

원칙 로봇이 넘어지는 것보다 사람이 다치는 것이 더 큰 사고 — 망설이면 비상 스위치

고장 코드 읽는 법

08 /twlr/faults 에 뜨는 이름 → 원인 → 조치

faults 이름	뜻	확인 · 조치
DM_L/R_TIMEOUT	DM 피드백 30 ms 끊김	CAN1 배선 · 120 Ω · DM 전원 · ID 1/2 · 0x11/0x12
DM_L/R_ERR	DM state ≥ 8 (아래 코드)	전원 재투입 · 온도 · 디버깅 툴로 에러 확인
OD_L/R_TIMEOUT	ODrive heartbeat 300 ms 끊김	CAN2 배선 · baud 1 M · node 0/1 · DIP 120 Ω
OD_L/R_ERR	axis_error ≠ 0	USB → odrivetool → dump_errors(odrv0)
IMU	BNO085 50 ms 응답 없음	I²C 18/19 배선 · 3.3 V · 주소 0x4A
FALL	|θ − θeq| > 35°	넘어짐 — 파손 확인 후 리셋
CALIB	휠 엔코더 캘리브 실패 · 15 s 초과	바퀴가 공중인지 · 엔코더 커넥터
HIP_TEMP	코일 80 °C 초과 (경고)	자동 510 mm · 휴식 후 재개

리셋 원인 제거 → /twlr/enable false → 1 s → true

DM state 8 과전압 · 9 저전압 · A 과전류 · B MOS 과열 · C 코일 과열 · D 통신두절 · E 과부하

게인 튜닝 순서

09 θ_eq → 자세 → 위치 · 속도 → 선회 → 고관절 , 한 번에 하나씩

1. **θ_eq**
   제자리에서 한쪽으로 흐르면 ±0.01 rad
2. **Kθ · Kω**
   손 놓고 서 있기 — 떨림 없이 10 s
3. **Kv · Kx**
   살짝 밀었다 놓기 — 출렁임 없이 복귀
4. **K_YAW**
   제자리 회전 — 늦거나 흔들리지 않게
5. **고관절 Kp · Kd · FF**
   높이 바꾸기 — 처짐 · 떨림 없이

plotjuggler: sudo apt install ros-jazzy-plotjuggler-ros

증상	조정
10 Hz 넘는 빠른 떨림 · 소리	Kω ↓ (4.59 → 3.5) 또는 Kθ ↓
느리고 큰 앞뒤 흔들림	Kθ ↑ · Kω ↑
한 방향으로 계속 흘러감	θ_eq 재측정 · Kx ↑
멈출 때 앞뒤로 출렁	Kv ↑
선회가 늦음 / 흔들림	K_YAW ↑ / ↓
다리가 처짐 (높이 부족)	HIP_FF_SCALE 0.8 → 1.0 · HIP_KP ↑
다리가 덜덜 떨림	HIP_KD_LOADED ↑ (3 → 4)

```
ros2 bag record -o run01 /imu/data /joint_states \
  /odom /twlr/pitch /cmd_vel
ros2 run plotjuggler plotjuggler   # bag 열어 그래프
```

한 번에 하나 · 10–20 % 씩 · 바꿀 때마다 기록 · config.h → 업로드 (≈ 20 s)

배터리 · 점검

10 6S LiPo — 전압으로 상태를 판단하고 , 운용 전후를 점검



운용 전

- ☐ 배터리 ≥ 23.0 V
- ☐ 전원 OFF 에서 CANH–CANL ≈ 60 Ω
- ☐ 커넥터 · 나사 · 링크 핀 풀림 없음
- ☐ 서포터 · 비상 스위치 손 닿는 곳
- ☐ 주행 공간 3 × 2 m · 주변 사람 확인
- ☐ 2 인 1 조 — 조작 1 · 안전 1

운용 후

- ☐ /twlr/hip_temp 확인 후 보관
- ☐ 보관 전압 22.8 V 로 충 · 방전
- ☐ 방화백 보관 · 부품 확인
- ☐ rosbag 정리 · 튜닝값 기록

```
# 운용 중 전압 보기
ros2 topic echo /twlr/battery \
  --field voltage
# 21.0 V 아래 → 종료 순서
```

목차

감사합니다

목차